БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики и информатики

ОТЧЕТ

Лабораторная работа №3

“Разработка требований к программной системе”

По курсу “Проектирование человеко-машинных интерфейсов”

Подготовлен: Беглецов Герман
Сидорская Дарья
Яхновец Владислав
студенты группы 10

Минск, 2026 г.

Название и цели лабораторной работы

Выполняется лабораторная работа № 3 по дисциплине “Проектирование человеко-машинных интерфейсов” на тему “Разработка требований к программной системе”.

Основная цель, разработать функциональные и нефункциональные требования к системе “Pinkmine”, спроектировать её архитектуру и требования на основе диаграмм UML (PlantUML, Mermaid), событийных методов EventStorming и Event Modeling, модели C4, и свести результаты в техническое задание по структуре SRS.

Постановка задачи для проектирования

Разработать систему управления проектами “Pinkmine” для повышения эффективности командной работы. Архитектура включает веб-сайт, сервер с API (язык Python, фреймворк Flask) и мобильное приложение для Android на Flutter. В системе можно создавать проекты, добавлять участников и управлять задачами. Задачи отображаются в виде списка и Канбан-доски с возможностью фильтрации по статусу, срокам и типу.

По результатам лабораторных работ №1 и №2 используются уточнённые наборы: статусы задачи, новая, в работе, на проверке, отложена, готова, закрыта; типы задачи, задача, ошибка, улучшение, идея, встреча; роли пользователей, администратор системы, менеджер проекта, участник команды.

Задание 1. Разработка требований в виде пользовательских историй (User Story)

Пользовательские истории

1. Описание приложения и ролей

Pinkmine - кроссплатформенная система управления проектами для небольших команд, созданная для повышения прозрачности и эффективности командной работы.

Система состоит из веб-клиента, мобильного приложения для Android на Flutter и сервера с REST API на Python и Flask. В системе создаются проекты, формируется состав команды, а задачи отображаются в виде списка и Канбан-доски с фильтрацией по статусу, типу и сроку.

В системе предусмотрены следующие роли:

- Администратор системы - настраивает права доступа.
- Менеджер проекта (тимлид) - создаёт проекты, формирует команду и ставит задачи.
- Участник команды (исполнитель) - выполняет задачи и меняет их статусы.
- Новый пользователь - пользователь до регистрации в системе.

2. Список пользовательских историй

№	Пользовательская история	Сценарий	Приоритет
US-1	Как новый пользователь, я хочу зарегистрироваться и войти в Pinkmine, чтобы получить доступ к проектам и задачам моей команды.	Регистрация и вход	Высокий
US-2	Как менеджер проекта, я хочу создать задачу с типом, сроком и исполнителем, чтобы поручить работу команде без лишних совещаний.	Постановка задачи	Высокий
US-3	Как менеджер проекта, я хочу добавлять участников в команду и назначать им роли, чтобы собрать команду для работы над проектом.	Формирование команды	Высокий
US-4	Как участник команды, я хочу менять статус задачи за 1–2 действия, чтобы тимлид видел актуальную картину без вопросов в чате.	Работа над задачей	Высокий
US-5	Как менеджер проекта, я хочу видеть задачи списком и Канбан-доской и фильтровать их по статусу, типу, сроку и исполнителю, чтобы быстро подготовить отчёт для заказчика.	Обзор и фильтрация	Высокий
US-6	Как участник команды, я хочу получать уведомление о назначенной мне задаче и открывать её одним касанием, чтобы не пропустить новую работу.	Работа над задачей	Средний
US-7	Как администратор системы, я хочу настраивать глобальные права доступа ролей, чтобы ограничить действия участников и менеджеров в соответствии с их обязанностями.	Настройка прав	Средний

Обоснование приоритетов

1. US-1 - высокий приоритет. Без регистрации и входа недоступна ни одна другая функция системы.
2. US-2 - высокий приоритет. Постановка задач является основной функцией продукта, а создание задач относится к наиболее частым действиям.
3. US-3 - высокий приоритет. Без сформированного состава команды нельзя назначать исполнителей.
4. US-4 - высокий приоритет. Изменение статуса является частым действием и, согласно профилям персонажей, должно занимать 1–2 действия.
5. US-5 - высокий приоритет. Список задач и Канбан-доска одинаково важны для пользователей.
6. US-6 - средний приоритет. Работать можно и без уведомлений: назначенные задачи видны в разделе «Мои задачи» и на доске. Уведомления снижают риск пропустить задачу, но не блокируют основной сценарий.
7. US-7 - средний приоритет. Права настраивает небольшое число администраторов, а базовый набор прав можно задать заранее.

Зависимости

1. US-1 предшествует всем остальным историям, поскольку все действия в системе требуют авторизации.
2. US-3 и US-2 предшествуют US-4, US-5 и US-6. Смена статуса, фильтрация и уведомления работают с существующими задачами, а назначить задачу можно только участнику проекта.

3. Критерии приёмки

US-1. Регистрация и вход нового пользователя

1. При регистрации обязательны логин, имя, фамилия, email и пароль. Если обязательное поле не заполнено, рядом с ним показывается ошибка, а учётная запись не создаётся.
2. Email должен быть уникальным. При регистрации с уже занятым email показывается сообщение о том, что он используется.
3. Пароль содержит не менее 8 символов и хранится в базе данных только в виде хеша.
4. После регистрации пользователь может войти с указанными логином и паролем. При неверных данных показывается сообщение об ошибке.
5. После входа открывается список проектов. Выход через меню профиля возвращает пользователя на экран входа.
6. Веб-версия: форма отправляется нажатием Enter и корректно отображается в Chrome.

7. Android: поле email вызывает клавиатуру для ввода email; пароль скрыт, доступна кнопка «Показать». Форма помещается на экране шириной 375 px без горизонтальной прокрутки.

US-2. Создание задачи менеджером

1. Форма «Новая задача…» содержит поля: название, описание, тип, срок и исполнитель.
2. Тип выбирается из пяти значений: задача, ошибка, улучшение, идея, встреча. Исполнитель выбирается только из участников проекта.
3. Если название не заполнено, задача не создаётся, а рядом с полем показывается ошибка.
4. Созданная задача получает статус «Новая» и сразу появляется в первой колонке Канбан-доски.
5. Задача появляется в разделе «Мои задачи» назначенного исполнителя.
6. Участнику команды поля «Исполнитель», «Тип» и «Срок» недоступны для редактирования согласно таблице ролей.
7. Веб-версия: задача создаётся из окна на Канбан-доске без перехода на другую страницу. Android: форма открывается на весь экран, а срок выбирается через календарь.

US-3. Состав команды и роли

1. В разделе «Команда» кнопка «Добавить участника…» открывает форму с данными пользователя и ролью.
2. Пользователя можно найти по имени или email.
3. Нельзя добавить пользователя, который уже состоит в проекте. При такой попытке показывается сообщение.
4. После добавления участник появляется в списке команды и получает доступ к просмотру задач проекта.
5. Участнику команды кнопка «Добавить участника…» не показывается.
6. Веб-версия: список участников отображает роль и контактные данные каждого пользователя.

US-4. Смена статуса задачи

1. Доступны шесть статусов: «Новая», «В работе», «На проверке», «Отложена», «Готова», «Закрыта».
2. Android: смена статуса выполняется в два действия - касание текущего статуса в карточке и выбор нового значения в нижней панели.
3. Веб-версия: статус меняется выбором значения в карточке или перетаскиванием карточки между колонками Канбан-доски.
4. Новый статус сразу отображается в карточке и на доске.
5. Статус обозначается иконкой и текстом, а не только цветом.
6. При ошибке связи статус не меняется, а пользователю показывается сообщение об ошибке.

US-5. Просмотр и фильтрация задач

1. В проекте доступны вкладки «Доска» и «Список». Переключение между ними не меняет выбранный проект.
2. Фильтры по статусу, типу, исполнителю, сроку и тегу применяются одновременно.
3. Быстрые фильтры «Мои задачи», «Просроченные» и «На проверке» применяются одним нажатием.
4. Если задачи не найдены, показываются сообщение «По вашему запросу ничего не найдено» и кнопка «Сбросить фильтры».
5. Список поддерживает сортировку по сроку выполнения и по дате обновления.
6. Веб-версия: фильтр по статусу отражается в адресе страницы, например projects/{id}/tasks?status=in_progress.
7. Android: поиск открывается нажатием на иконку лупы в верхней панели в полноэкранном режиме. Панель фильтров помещается на экране 375 × 670 px без горизонтальной прокрутки.

US-6. Уведомление о назначенной задаче

1. При назначении исполнителя создаётся уведомление для этого пользователя и только для него.
2. Текст уведомления содержит название проекта и название задачи.
3. Android: push-уведомление приходит на экран смартфона. Нажатие на него открывает карточку задачи, а не список задач.
4. Веб-версия: иконка колокольчика в верхней панели открывает список последних событий. Нажатие на строку уведомления открывает задачу.
5. Прочитанное уведомление помечается как прочитанное. Непрочитанные уведомления визуально отличаются от прочитанных.

US-7. Глобальные права доступа

1. Раздел «Роли и права» с адресом team/roles доступен только администратору системы.
2. Администратор видит список ролей и права каждой из них.
3. Изменённые права сохраняются и применяются при следующем действии пользователя.
4. После отзыва права пользователь не видит соответствующего действия. Например, участнику недоступно создание проекта.
5. Менеджер и участник при попытке открыть раздел получают сообщение о недоступности действия.
6. Раздел реализован только в веб-версии.

4. Платформенные особенности

Параметр	Веб-интерфейс	Android-приложение

Экран	Монтажная область 1440 px, планшет 980 px, сетка из 12 колонок	Экран 375 × 670 px, сетка 8 pt
Навигация	Верхняя панель: «Проекты», «Мои задачи», «Команда», поиск, уведомления и профиль. Нижняя часть страницы содержит справочную информацию	Нижняя панель: «Главная», «Проекты», «Мои задачи», «Профиль»
Ввод	Мышь и клавиатура	Сенсорный ввод: жесты и касания
Уведомления	Иконка уведомлений в верхней панели	Push-уведомление с прямым переходом в карточку задачи; иконка уведомлений рядом с профилем
Смена статуса	Выбор в карточке или перетаскивание карточки между колонками Канбан-доски	Два действия: нажатие на текущий статус и выбор нового значения в нижней панели
Поиск и фильтры	Сквозная строка поиска в верхней панели, панель фильтров над списком	Иконка лупы в верхней панели, полноэкранный поиск
Доступность	Иконка и текст для статусов, контрастные цвета, шрифт без засечек	Те же требования
Роли и права	Раздел «Роли и права» с адресом team/roles для администратора	Раздел «Роли и права» не реализован

5. Распределение ролей и согласование

Участник	Вклад
Беглецов Герман	Задания 5–6: проектирование на основе EventStorming и Event Modeling
Сидорская Дарья	Задания 1–3: пользовательские истории, диаграммы PlantUML и Mermaid
Яхновец Владислав	Задания 4, 7, 8: расширение спецификации с ИИ, описание архитектуры C4, техническое задание

Задание 2. Разработка спецификации системы с использованием PlantUML

Спецификация включает диаграммы, выполненные на языках PlantUML и Mermaid. Диаграммы построены по результатам лабораторной работы №2: таблицам ролей и задач пользователей, объектной модели, связям между объектами и сценариям персонажей.

Для каждой диаграммы представлены её изображение и исходный код.

1. Диаграмма вариантов использования (PlantUML)

Диаграмма построена по таблице «Проанализировать задачи и роли пользователей» из отчёта лабораторной работы №2. Каждая строка таблицы соответствует варианту использования, а связь между ролью и вариантом использования - знаку «+» в таблице.

Права ролей организованы по принципу **наследования**: менеджер проекта получает все действия участника команды, а администратор системы - все действия менеджера проекта. На диаграмме это показано связями обобщения между ролями.

Участнику команды недоступны создание проекта, добавление участников, назначение исполнителя и типа задачи, установка сроков и добавление тегов. Настройка глобальных прав доступа доступна только администратору системы.

Диаграмма показывает, какие действия может выполнять каждая роль в системе.



2. Диаграмма деятельности: создание и назначение задачи (PlantUML)

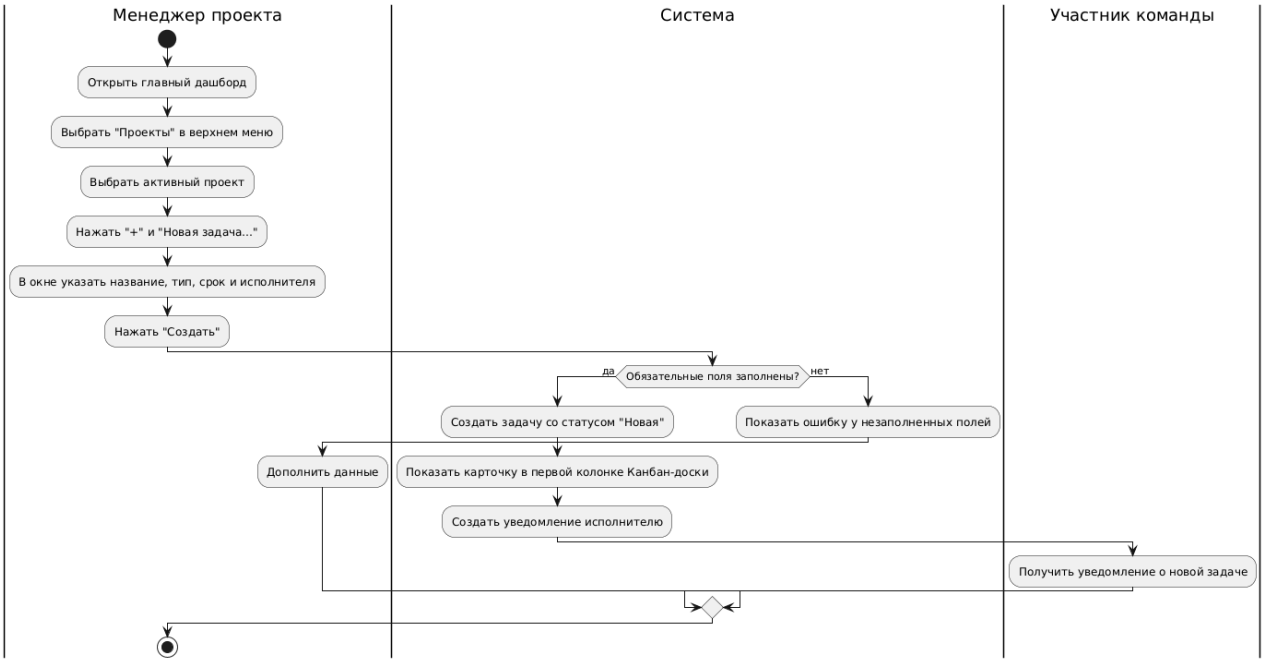
Диаграмма описывает сценарий создания и назначения задачи менеджером проекта. Основой служат «Сценарий 1» тимлида Джесики из лабораторной работы №2 и строка «Создать и назначить задачу» в таблице ключевых сценариев.

Менеджер открывает дашборд, переходит в раздел «Проекты», выбирает активный проект и нажимает «+», затем «Новая задача…». В форме он указывает название, тип, срок и исполнителя и нажимает «Создать».

Система проверяет обязательные поля:

- Если обязательные поля заполнены, система создаёт задачу со статусом «Новая», отображает её карточку в первой колонке Канбан-доски и создаёт уведомление для исполнителя.
- Если обязательные поля не заполнены, система показывает ошибки рядом с соответствующими полями. Менеджер дополняет данные и повторно отправляет форму.

Диаграмма показывает последовательность действий менеджера и системы, а также обработку ошибок при заполнении формы.

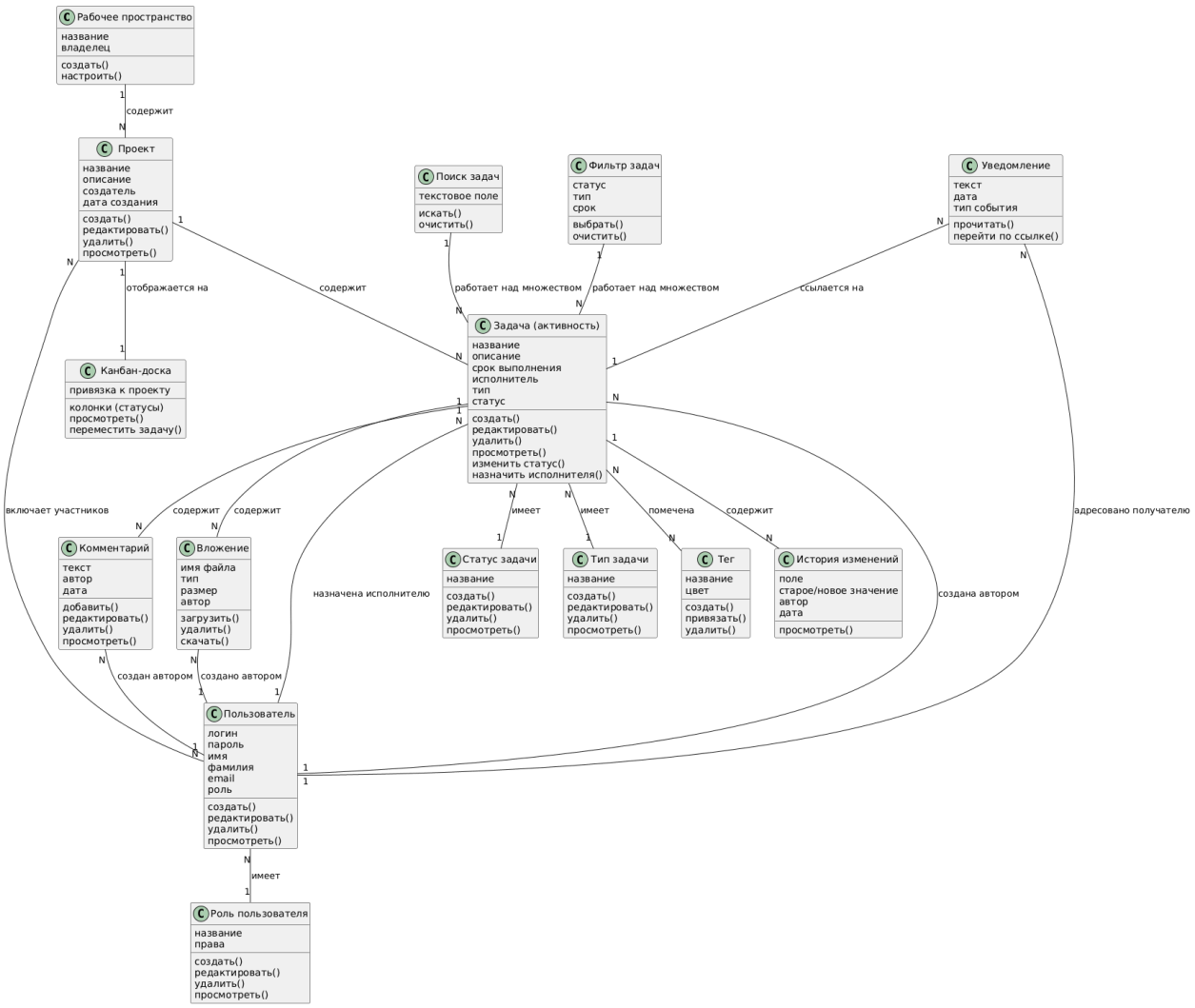


3. Диаграмма классов (PlantUML)

Диаграмма построена по таблицам «Разработать объектную модель» и «Связи между объектами» из лабораторной работы №2.

Каждый класс соответствует объекту объектной модели. Атрибуты взяты из колонки «Атрибуты», операции - из колонки «Действия», а кратности связей - из таблицы связей между объектами.

Диаграмма служит основой для проектирования базы данных и API. Для заинтересованных лиц она показывает, какие объекты хранит система и как они связаны между собой.



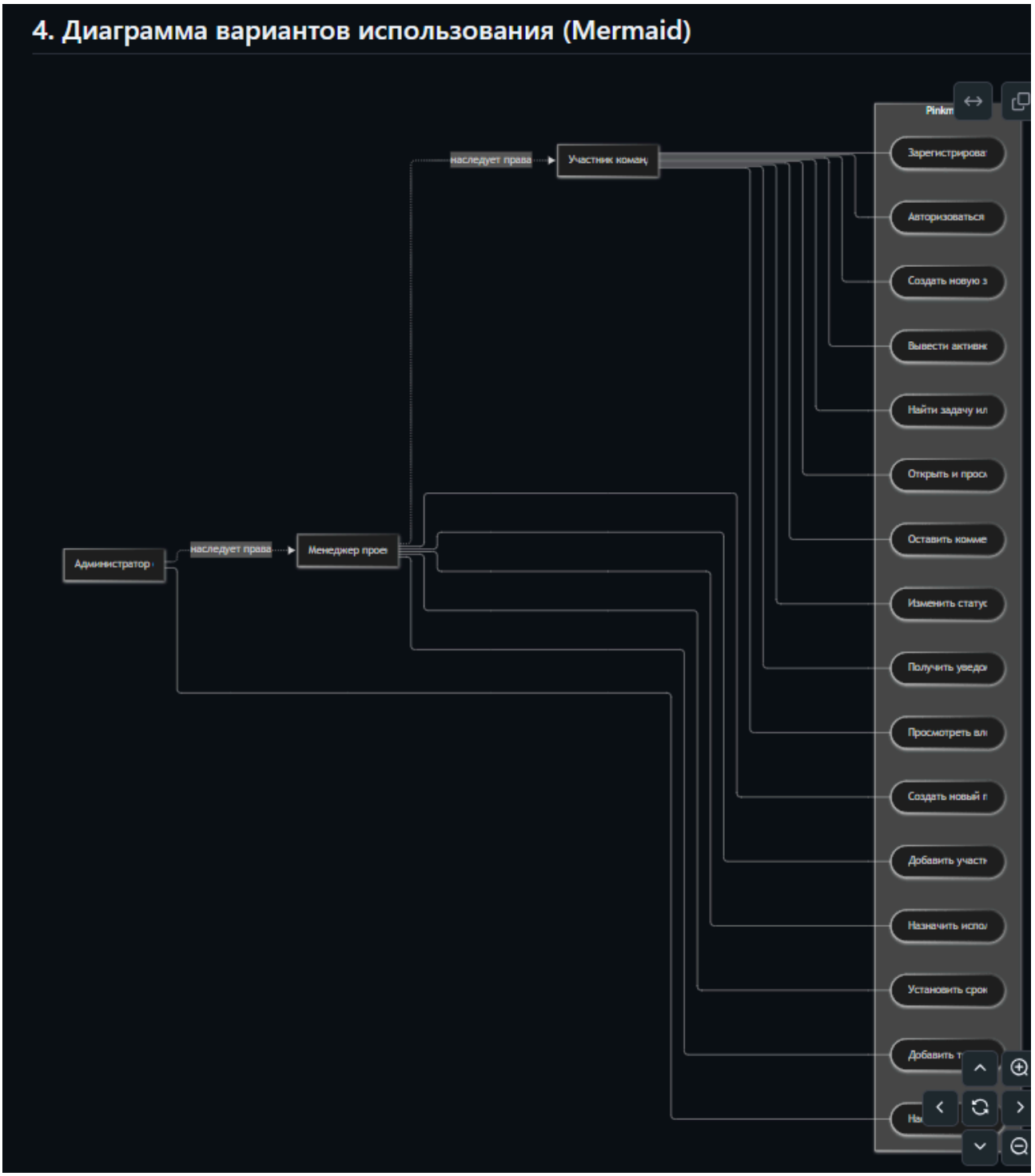
Задание 3. Разработка спецификации системы с использованием Mermaid

4. Диаграмма вариантов использования (Mermaid)

Диаграмма повторяет диаграмму из раздела 1 средствами Mermaid. Она построена по той же таблице «Проанализировать задачи и роли пользователей» из лабораторной работы №2.

Диаграмма оформлена как flowchart: роли представлены прямоугольниками, варианты использования - овалами, а наследование прав обозначено пунктирными стрелками.

Диаграмма сохраняет распределение действий между участником команды, менеджером проекта и администратором системы, представленное в версии PlantUML.



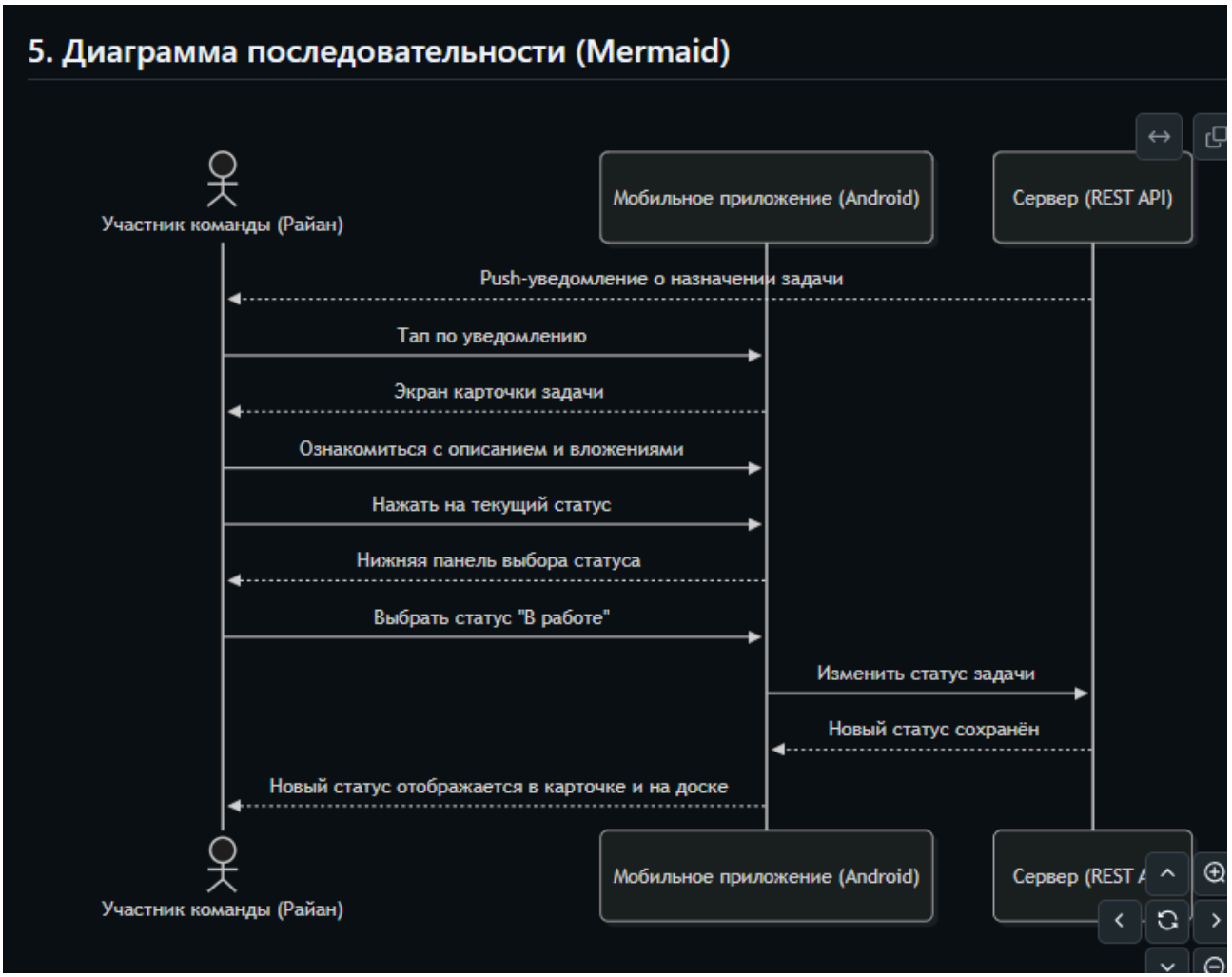
5. Диаграмма последовательности (Mermaid)

Диаграмма описывает «Сценарий 2» исполнителя Райана из лабораторной работы №2.

Исполнитель получает push-уведомление о назначении задачи и переходит по нему в карточку задачи. Затем он нажимает на текущий статус и выбирает в нижней панели значение «В работе».

Мобильное приложение отправляет запрос на изменение статуса на сервер. После успешного выполнения запроса новый статус отображается в карточке задачи и на Канбан-доске.

Диаграмма показывает порядок взаимодействия между пользователем, мобильным приложением и сервером. Она предназначена для согласования работы клиента и API.

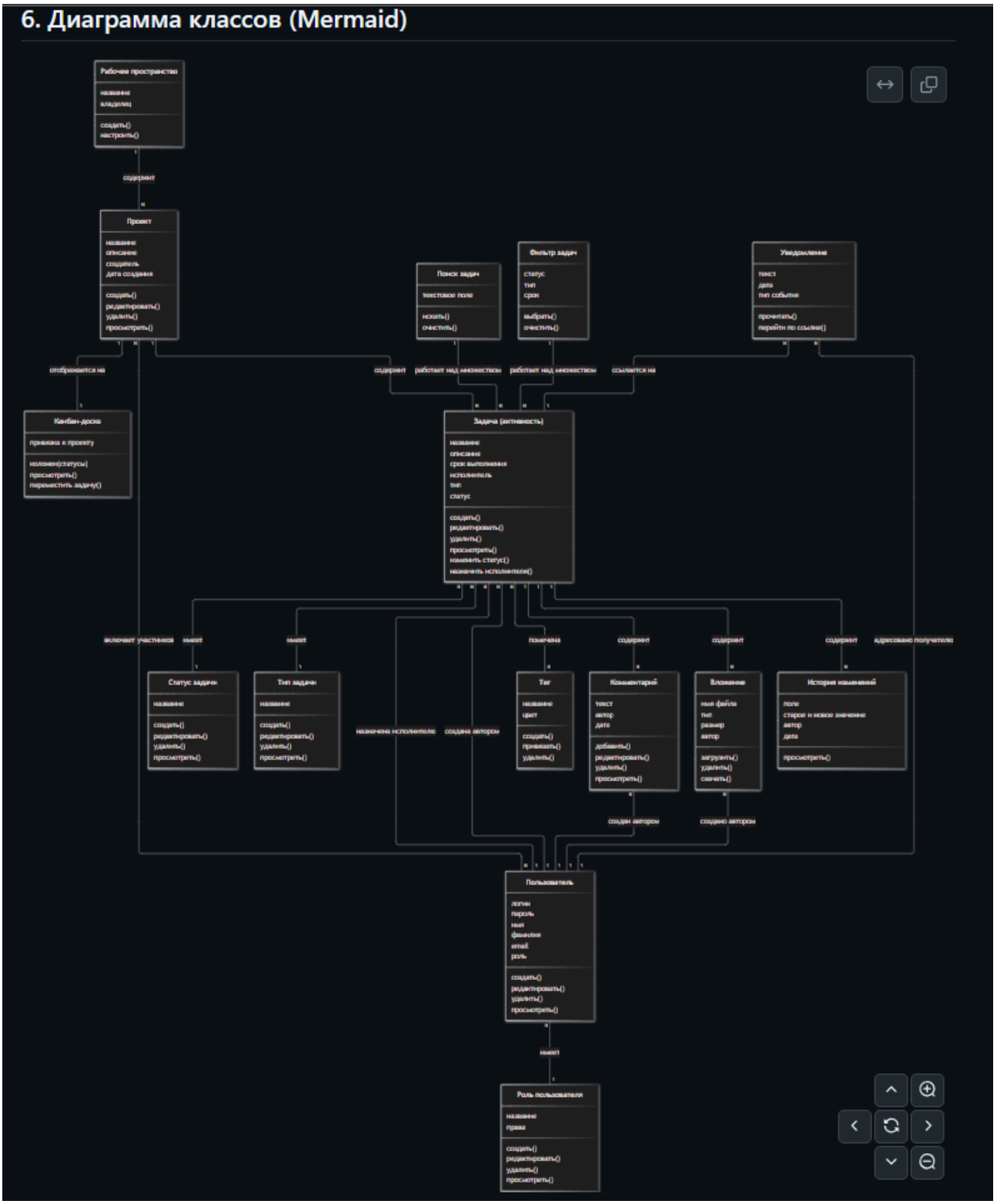


6. Диаграмма классов (Mermaid)

Диаграмма повторяет диаграмму классов из раздела 3 средствами Mermaid.

Она построена по таблицам «Разработать объектную модель» и «Связи между объектами» из лабораторной работы №2. Классы, атрибуты, операции и кратности связей соответствуют этим таблицам.

Диаграмма представляет ту же объектную модель системы, что и версия PlantUML, в другой форме записи.



Задание 4. Разработка спецификации системы с использованием PlantUML, Mermaid и ИИ-агентов

2. Разработанные промпты

Разработано 12 промптов:

- общий блок контекста;

- промпт на пользовательские истории;
- 9 промптов на UML - по одному на каждый вид диаграмм;
- промпт на техническую спецификацию.

Общий блок контекста подставляется во все остальные промпты вместо `{{КОНТЕКСТ}}`. Поля в фигурных скобках `{{…}}` заполняются при запуске. Результат каждого промпта проверялся и сверялся с Event Storming и Event Modeling перед переносом в вики.

№	Промпт	Применён для	Результат в вики
2.1	Общий блок контекста	Все промпты	-
2.2	Пользовательские истории	US-1…US-7, команды и события Event Storming	User-Stories: дополнения к US-1, 2, 4, 6; US-8…US-14
2.3	Диаграмма вариантов использования + сценарии	Вся система; подсистема «Работа с задачами»	Functional-Requirements: `uc_overview`, `uc_tasks`, UC-T1…UC-T5
2.4	Диаграмма деятельности	«Выполнение и проверка задачи», «Добавление участника и приглашение»	Specifications §7: `act_task_review`, `act_invite`
2.5	Диаграмма классов	Доменная модель, модель прав, сервер, мобильное приложение	Specifications §8: `cls_domain`, `cls_rbac`, `cls_backend`, `cls_mobile`
2.6	Объектная диаграмма	Момент отправки задачи на проверку	Specifications §8: `obj_project_review`
2.7	Диаграмма последовательности	6 сценариев: вход, создание и назначение, смена статуса, приглашение, проверка, просрочка	Specifications §9: `seq_*`
2.8	Диаграмма компонентов	Система целиком	Specifications §10: `cmp_system`

2.9	Диаграмма пакетов	Сервер, мобильный и веб-клиент	Specifications §11: `pkg_structure`
2.10	Диаграмма развёртывания	Система целиком	Specifications §12: `dep_system`
2.11	ERD / ORM	Черновая схема БД	Черновик ERD, затем доработан навыком `getanalyst-database-designer` (раздел 3.1) → Database-Schema
2.12	Техническая спецификация (SRS)	Будет применён в задании 8	-

2.1. Общий блок контекста `{{КОНТЕКСТ}}`

```
Проект: Pinkmine - кроссплатформенная система управления проектами для небольших
команд.
Клиенты: веб-клиент (браузер, ПК) и мобильное приложение Android (Flutter).
Сервер: REST API на Python/Flask
(Blueprints: auth, projects, tasks, notifications), реляционная БД, push через
Firebase Cloud Messaging, email через SMTP.
Роли: новый пользователь (не зарегистрирован); участник команды (исполнитель);
менеджер проекта (тимлид,
наследует права участника); администратор системы (наследует права менеджера,
настраивает глобальные права).
Объекты: рабочее пространство, проект, участник проекта с ролью, приглашение,
задача, статус задачи
(новая, в работе, на проверке, отложена, готова, закрыта), тип задачи (задача,
ошибка, улучшение, идея, встреча),
тег, комментарий, вложение, история изменений, уведомление.
Бизнес-правила: исполнитель выбирается только из участников проекта; закрывает
задачу только менеджер;
приглашение действует 7 дней; проект архивируется, физически удаляет только
администратор;
при офлайн-конфликте статуса побеждает состояние сервера; о просрочке - push
исполнителю, email-дайджест менеджеру.
Существующие артефакты: User Stories US-1…US-7, Event Storming (агрегаты: Учётная
запись, Проект, Задача,
Комментарий, Уведомление; контексты: Идентификация и доступ, Управление проектами,
Управление задачами, Уведомления),
Event Modeling (срезы S1–S17 с маршрутами API).
```

2.2. Промпт: пользовательские истории

```
Роль: ты - опытный системный аналитик, пишешь требования в формате User Story по
INVEST.

Контекст:
{{КОНТЕКСТ}}

Входные данные: текущий список User Stories с критериями приёмки:
{{ВСТАВИТЬ User-Stories.md}}
и события/команды Event Storming:
{{ВСТАВИТЬ список команд и событий}}
```

```
Задача:
1. Проверь каждую существующую историю по INVEST и найди пробелы: недостающие
критерии приёмки, граничные случаи,
   платформенные различия (веб/Android), противоречия с бизнес-правилами.
2. Найди команды и события Event Storming, которые не покрыты ни одной историей, и
предложи новые истории,
   начиная с номера US-8. Старые номера не меняй.
3. Для каждой новой истории дай:
   - формулировку «Как <роль>, я хочу <действие>, чтобы <ценность>»;
   - 4–7 проверяемых критериев приёмки без дублирования, платформенные пометить
[Web] / [Android];
   - для 1–2 ключевых критериев - сценарий Given/When/Then;
   - приоритет High/Medium/Low и обоснование в 1–2 предложениях;
   - зависимости от других историй.

Ограничения: не придумывай функции, которых нет в контексте и входных данных; пиши
по-русски;
истории - пользовательские, а не технические задачи.

Формат вывода: Markdown -
(а) таблица «Пробелы в существующих историях»: история | что добавить;
(б) таблица новых историй: № | История | Сценарий | Приоритет;
(в) по каждой новой истории - блок «### US-N. Название» с нумерованными
критериями;
(г) список зависимостей.
Самопроверка в конце: перечисли, какие команды Event Storming теперь покрыты
историями.
```

2.3. Промпт: диаграмма вариантов использования и сценарии

```
Роль: ты - системный аналитик, моделируешь функциональные требования в UML на
языке PlantUML.

Контекст:
{{КОНТЕКСТ}}

Входные данные: User Stories US-1…US-14; таблица ролей и задач из ЛР2;
существующая диаграмма вариантов
использования; ограниченные контексты Event Storming.
Область: {{вся система | подсистема «…»}}

Задача:
1. Построй диаграмму вариантов использования: актёры (роли + внешние системы FCM,
SMTP, планировщик),
   граница системы, варианты использования как цели пользователя (глагол +
объект), наследование ролей,
   <<include>>/<<extend>> только там, где они реально есть. Сгруппируй варианты
использования по ограниченным контекстам.
2. Для каждого варианта использования опиши сценарий по шаблону: ID, название,
актёры (основной и вторичные), цель,
   связанные US, предусловия, триггер, основной поток (нумерованные шаги «актёр
делает - система отвечает»),
   альтернативные потоки, исключения (с номером шага, на котором они возникают),
постусловия, бизнес-правила.

Ограничения:
- синтаксис PlantUML, блок @startuml uc_<имя> … @enduml;
- подписи по-русски;
- используй только роли, системы и функции из контекста и входных данных;
недостающее перечисли как допущения;
- не более ~25 вариантов использования на диаграмме, при необходимости разбей на
подсистемы.
```

Формат вывода: (1) блок ```plantuml```; (2) эквивалент на Mermaid (flowchart) в блоке ```mermaid```;
(3) описание диаграммы 3–5 предложений: что показывает, на чём основана, кому полезна;
(4) сценарии - по таблице на каждый вариант использования; (5) список допущений.

2.4. Промпт: диаграмма деятельности

```
Роль: ты - системный аналитик, моделируешь бизнес-процессы в UML на языке PlantUML.

Контекст:
{{КОНТЕКСТ}}

Входные данные: процесс «{{название}}»; связанные User Stories {{US-…}}; срезы Event Modeling {{S…}};
противоречия Event Storming {{H…}}.

Задача: построй диаграмму деятельности процесса с дорожками (swimlanes) по участникам: роли и «Система».
Покажи ветвления с подписанными условиями, циклы (repeat/while), точки завершения с ошибкой, уведомления участникам.
Используй новый синтаксис activity (:действие;, if/else/endif, repeat).

Ограничения:
- синтаксис PlantUML, блок @startuml act_<имя> … @enduml;
- подписи по-русски, действия - глагол + объект;
- каждое действие стоит в дорожке того, кто его выполняет; соблюдай права ролей из бизнес-правил;
- используй только шаги, следующие из входных данных; недостающее перечисли как допущения.

Формат вывода: (1) блок ```plantuml```; (2) эквивалент на Mermaid (flowchart) в блоке ```mermaid```;
(3) описание 3–5 предложений: что показывает диаграмма, на чём основана, кому полезна; (4) список допущений.
Самопроверка: перечисли, какие бизнес-правила отражены на диаграмме.
```

2.5. Промпт: диаграмма классов

```
Роль: ты - системный аналитик и архитектор, моделируешь структуру системы в UML на языке PlantUML.

Контекст:
{{КОНТЕКСТ}}

Входные данные: {{доменная модель | модель ролей и прав | слой сервера (Flask) | модуль мобильного приложения (Flutter)}};
объектная модель и таблица связей из ЛР2; агрегаты Event Storming; существующая диаграмма классов.

Задача: построй диаграмму классов: атрибуты с типами, ключевые операции, перечисления (enum),
ассоциации с кратностями и ролями, агрегация/композиция, наследование и реализация интерфейсов.
Для технических диаграмм используй имена из Flask (Blueprint, Service, Repository) или Flutter (Screen, Controller, Repository).

Ограничения:
- синтаксис PlantUML, блок @startuml cls_<имя> … @enduml;
- в доменной модели подписи по-русски, в технических диаграммах имена классов и атрибутов - латиницей;
```

- используй только сущности из контекста и входных данных; не добавляй библиотеки и технологии, которых нет в контексте;
- не более ~25 классов, при необходимости разбей на несколько диаграмм.

Формат вывода: (1) блок ```plantuml```; (2) эквивалент на Mermaid (classDiagram) в блоке ```mermaid```;
(3) описание 3–5 предложений: что показывает диаграмма, на чём основана, кому полезна; (4) список допущений.
Самопроверка: для каждого бизнес-правила укажи, каким элементом модели оно обеспечивается.

2.6. Промпт: объектная диаграмма

Роль: ты - системный аналитик, иллюстрируешь модель данных объектной диаграммой UML на языке PlantUML.

Контекст:
{{КОНТЕКСТ}}

Входные данные: доменная модель (диаграмма классов); персонажи из ЛР2; ситуация «{{описание момента}}».

Задача: построй объектную диаграмму - снимок состояния системы в описанный момент: конкретные экземпляры
«имя : Класс» со значениями атрибутов и связи между ними.

Ограничения:
- синтаксис PlantUML, блок @startuml obj_<имя> … @enduml;
- подписи по-русски, значения - реалистичные, на русском, без реальных персональных данных;
- значения должны соответствовать бизнес-правилам (например, исполнитель - участник проекта, версия задачи
  совпадает с числом изменений);
- используй только классы и атрибуты доменной модели.

Формат вывода: (1) блок ```plantuml```; (2) эквивалент на Mermaid (classDiagram с подписями «имя : Класс»)
в блоке ```mermaid```; (3) описание 3–5 предложений: что показывает снимок и для чего его использовать.

2.7. Промпт: диаграмма последовательности

Роль: ты - системный аналитик, моделируешь взаимодействие компонентов в UML на языке PlantUML.

Контекст:
{{КОНТЕКСТ}}

Входные данные: сценарий «{{название}}»; связанные User Stories {{US-…}} и сценарий варианта использования {{UC-…}};
срезы Event Modeling {{S…}} с маршрутами API; классы сервера и мобильного приложения.

Задача: построй диаграмму последовательности для сценария: актёр, клиент (веб или Android), Blueprint REST API,
сервис, репозиторий/БД, внешние системы. Укажи HTTP-методы и маршруты из срезов Event Modeling, коды ответов,
alt/opt/loop для ветвлений и ошибок, autonumber. Асинхронные уведомления покажи отдельными сообщениями после ответа клиенту.

Ограничения:
- синтаксис PlantUML, блок @startuml seq_<имя> … @enduml;
- подписи сообщений по-русски, маршруты и имена методов - латиницей;

- используй только маршруты и компоненты из входных данных; новые маршруты перечисли как допущения;
- покажи все ветки ошибок, которые следуют из критериев приёмки.

Формат вывода: (1) блок ```plantuml```; (2) эквивалент на Mermaid (sequenceDiagram) в блоке ```mermaid```;
(3) описание 3–5 предложений: что показывает диаграмма, на чём основана, кому полезна; (4) список допущений.

2.8. Промпт: диаграмма компонентов

Роль: ты - архитектор, описываешь компонентную структуру системы в UML на языке PlantUML.

Контекст:
{{КОНТЕКСТ}}

Входные данные: ограниченные контексты Event Storming; классы сервера; классы мобильного приложения.

Задача: построй диаграмму компонентов: клиентские приложения, компоненты сервера (Blueprints, сервисы,
планировщик, шлюзы FCM/SMTP), БД, внешние системы; предоставляемые и требуемые интерфейсы (REST API, SQL, FCM API, SMTP).
Покажи, как компоненты обмениваются доменными событиями.

Ограничения:
- синтаксис PlantUML, блок @startuml cmp_<имя> … @enduml;
- компоненты сервера соответствуют ограниченным контекстам;
- не добавляй технологии, которых нет в контексте;
- подписи интерфейсов не должны перекрывать элементы: при необходимости подписывай связи, а не интерфейсы.

Формат вывода: (1) блок ```plantuml```; (2) эквивалент на Mermaid (flowchart) в блоке ```mermaid```;
(3) описание 3–5 предложений; (4) список допущений.

2.9. Промпт: диаграмма пакетов

Роль: ты - архитектор, описываешь структуру кода в UML на языке PlantUML.

Контекст:
{{КОНТЕКСТ}}

Входные данные: ограниченные контексты Event Storming; диаграммы классов сервера и мобильного приложения.

Задача: построй диаграмму пакетов для кода сервера, мобильного и веб-клиента: пакеты верхнего уровня по ограниченным
контекстам Event Storming и слоям (api, services, repositories, models, events, integrations), зависимости
<<import>>/<<use>> без циклов, связь клиентов с сервером через HTTP.

Ограничения:
- синтаксис PlantUML, блок @startuml pkg_<имя> … @enduml;
- зависимости направлены от верхних слоёв к нижним, циклов нет;
- имена пакетов - как в коде (латиницей).

Формат вывода: (1) блок ```plantuml```; (2) эквивалент на Mermaid (flowchart) в блоке ```mermaid```;
(3) описание 3–5 предложений; (4) список допущений.
Самопроверка: подтверди, что граф зависимостей не содержит циклов.

2.10. Промпт: диаграмма развёртывания

```
Роль: ты - архитектор, описываешь физическое размещение системы в UML на языке
PlantUML.

Контекст:
{{КОНТЕКСТ}}

Входные данные: диаграмма компонентов; требования к офлайн-режиму мобильного
приложения; внешние сервисы.

Задача: построй диаграмму развёртывания: устройства (ПК с браузером, смартфон
Android), серверные узлы и среды
выполнения (веб-сервер, Flask-приложение, фоновый процесс планировщика, СУБД),
облачные сервисы (FCM, SMTP),
артефакты на узлах и протоколы связей (HTTPS, SQL, SMTP).

Ограничения:
- синтаксис PlantUML, блок @startuml dep_<имя> … @enduml;
- узлы называй по роли; конкретные продукты указывай, только если они есть в
контексте;
- покажи локальное хранилище на смартфоне, если в требованиях есть офлайн-режим.

Формат вывода: (1) блок ```plantuml```; (2) эквивалент на Mermaid (flowchart) в
блоке ```mermaid```;
(3) описание 3–5 предложений; (4) список допущений.
```

2.11. Промпт: ERD и ORM

```
Роль: ты - системный аналитик, проектируешь реляционную модель данных и описываешь
её на языке PlantUML.

Контекст:
{{КОНТЕКСТ}}

Входные данные: доменная модель (диаграмма классов); агрегаты и противоречия Event
Storming; User Stories US-1…US-14.

Задача:
1. Построй ERD физической модели: таблицы (snake_case, множественное число),
столбцы с типами PostgreSQL,
   PK/FK/UNIQUE/NOT NULL, связи в нотации «вороньей лапки», таблицы-связки для
M:N, справочники статусов и типов.
2. Отдельно построй диаграмму ORM-отображения: классы моделей сервера ↔ таблицы,
связи между моделями.

Ограничения:
- синтаксис PlantUML, блоки @startuml erd_<имя> и @startuml orm_<имя>;
- каждое бизнес-правило, которое можно обеспечить ограничением БД, обеспечь
ограничением;
- не храни вычисляемые значения без необходимости.

Формат вывода: (1) блоки ```plantuml```; (2) эквивалент ERD на Mermaid (erDiagram)
в блоке ```mermaid```;
(3) описание ключевых связей и ограничений; (4) список допущений.
```

2.12. Промпт: техническая спецификация (SRS)

```
Роль: ты - ведущий системный аналитик, готовишь спецификацию требований к ПО (SRS)
по структуре IEEE 830 / ГОСТ 34.602.

Контекст:
{{КОНТЕКСТ}}
```

```
Входные данные: User Stories (US-1…US-N) с критериями приёмки; сценарии вариантов
использования;
UML-диаграммы (use case, activity, class, sequence, component, package,
deployment); ERD;
Event Storming и Event Modeling; таблица платформенных особенностей.

Задача: собери SRS со структурой:
1. Введение (назначение, область действия, определения и сокращения, ссылки).
2. Общее описание (перспектива продукта, функции, классы пользователей, среда
эксплуатации, ограничения, допущения и зависимости).
3. Функциональные требования - сгруппировать по ограниченным контекстам; у каждого
требования ID (FR-xx), формулировка
   «Система должна…», источник (US/UC), приоритет, критерий проверки.
4. Требования к внешним интерфейсам (пользовательский интерфейс веб/Android,
программные интерфейсы REST API, FCM, SMTP).
5. Нефункциональные требования (производительность, безопасность, надёжность,
доступность WCAG, совместимость, офлайн-режим) - измеримые.
6. Модели системы (ссылки на диаграммы).
7. Матрица трассируемости US → UC → FR → диаграммы.
Ограничения: каждое требование атомарно, проверяемо и однозначно; не добавляй
функций, которых нет во входных данных;
противоречия и пробелы вынеси в отдельный раздел «Открытые вопросы».
Формат вывода: Markdown с нумерованными заголовками и таблицами.
```

3. Применение навыков ИИ-агентов

Навыки взяты из папки материалов: `getanalyst-database-designer` и `getanalyst-restapi-designer-skill`. Каждый навык - это файл `SKILL.md` с рабочим процессом и справочники в папке `references/`. Оба навыка запускались в ИИ-агенте Claude Code: агент загружал `SKILL.md` и справочники и выполнял рабочий процесс навыка. Ниже для каждого навыка приведены входные материалы, промпт запуска, результат и то, как результат использован.

3.1. Навык `getanalyst-database-designer`

Назначение навыка: проектировать и проверять реляционные БД. Навык анализирует предметную область и бизнес-правила, строит логическую и физическую модели, ER-диаграмму, PostgreSQL DDL, тестовые данные и SQL-запросы, прорабатывает транзакции, конкурентный доступ и миграции. Справочник: `references/database-design-standards.md`.

Входные материалы:

- Home.md: описание Pinkmine;
- User-Stories.md и новые US-8…US-14;
- Event-Storming.md: агрегаты, ограниченные контексты, противоречия H1–H5;
- Event-Modeling.md: срезы S1–S17 и маршруты API;
- доменная модель `cls_domain` и черновая ERD, полученная по промпту 2.11.

Промпт запуска:

```
Используй $getanalyst-database-designer, чтобы спроектировать production-ready
базу данных для проекта Pinkmine.
Материалы: описание проекта (Home.md), пользовательские истории US-1…US-14, Event
Storming (агрегаты,
контексты, противоречия H1–H5), Event Modeling (срезы и маршруты API), доменная
модель и черновая ERD.
Сначала проведи ревью черновой ERD, затем выполни полное проектирование. СУБД - по
умолчанию.
```

Результат навыка

Ревью черновой ERD

Навык требует при наличии модели начинать с её ревью. Найденные проблемы:

№	Проблема	Последствие	Исправление
R1	`tasks.assignee_member_id` → `project_members.id` не проверяет, что исполнитель из того же проекта	Задачу можно назначить участнику чужого проекта, правило US-2 не выполняется	Составной FK `(project_id, assignee_user_id)` → `project_members (project_id, user_id)`
R2	`task_tags` позволяет привязать тег из другого проекта	Теги «протекают» между проектами	Составные FK `(project_id, task_id)` и `(project_id, tag_id)`
R3	`UNIQUE(email)` чувствителен к регистру	`Ivan@…` и `ivan@…` - два аккаунта (нарушение US-1.2)	Уникальный индекс по `lower(email)`, то же для `login`
R4	`tasks.is_overdue` хранит вычисляемое значение	Нарушение 3НФ; рассинхрон при смене статуса или срока	Просрочка вычисляется; хранится только факт уведомления `overdue_notified_at`

R5	Токен приглашения хранится в открытом виде; можно создать несколько активных приглашений на один email	Утечка БД даёт рабочие ссылки; дубли приглашений (US-9.6)	`token_hash`; частичный уникальный индекс по активным приглашениям
R6	Нет хранения токенов подтверждения email и сброса пароля	US-1.8 и US-13 нельзя реализовать	Таблица `user_tokens`
R7	Нет статуса доставки уведомлений, хотя он есть в агрегате «Уведомление» Event Storming	Нельзя повторить отправку при сбое FCM	`delivery_status`, `attempts`, `last_error`, `sent_at`
R8	`VARCHAR(n)` без бизнес-правил длины	Произвольные ограничения, ошибки на длинных данных	`text` + `CHECK` там, где есть правило
R9	Не заданы правила `ON DELETE` и индексы на FK	Неявное поведение при удалении; медленные проверки FK	Явные правила для каждого FK, индексы по сценариям
R10	Флаги `is_read`, `is_archived`, `email_confirmed` теряют момент события	Нельзя показать, когда прочитано или архивировано	`read_at`, `archived_at`, `email_confirmed_at`
R11	Глобальная роль пользователя заменена флагом `is_admin`, хотя в ЛР2 «Пользователь имеет роль», а US-7 настраивает права ролей	Нельзя ограничить создание проектов ролью	`users.role_id`

R12	История не связана с версией задачи	Повтор запроса из офлайн-очереди (H3) создаёт дубль истории	`task_version` + `UNIQUE (task_id, task_version, field)`

1. Краткое понимание проекта

Pinkmine хранит пользователей и их роли, рабочие пространства, проекты с командами и приглашениями, задачи с жизненным циклом статусов, тегами, комментариями, вложениями и историей изменений, а также уведомления (push, email, в приложении). Критичные операции:

- смена статуса задачи с офлайн-синхронизацией (H3);
- назначение исполнителя только из участников проекта;
- принятие приглашения (H1);
- ежедневная фиксация просрочек (H5).

2. Критичные вопросы

Ни один из вопросов не блокирует проектирование, по каждому принято предположение (п. 3).

1. Роль пользователя глобальная (ЛР2) или назначается внутри проекта (US-3)?
2. В каком часовом поясе определяется наступление срока?
3. Где хранятся файлы вложений?
4. Удаляются ли пользователи физически?
5. Каковы ожидаемые объёмы данных?

3. Принятые предположения и их влияние

№	Предположение	Влияние на модель
A1	Роль действует на двух уровнях: глобальная (`users.role_id`: admin, manager, member) и роль в проекте (`project_members.role_id`)	Две ссылки на `roles`; права обеих ролей задаются `role_permissions`
A2	Срок - календарная дата; просрочка определяется по дате в одном часовом поясе системы, который приложение передаёт параметром `today`	Тип `date`; запросы получают `$today`, а не `current_date`

A 3	Файлы хранятся во внешнем файловом хранилище, в БД - только метаданные и ключ	`attachments.storage_key`; нужна очистка «осиротевших» файлов
A 4	Пользователи физически не удаляются	FK на `users` - `RESTRICT`
A 5	Небольшие команды: сотни пользователей, тысячи задач на проект	Партиционирование и денормализация не нужны; для списка задач проекта допустим `OFFSET`

4. Бизнес-правила

Правило	**Источник**	**Как обеспечено**
Исполнитель - только участник проекта	US-2.2	Составной FK
Один человек - один раз в команде проекта	US-3.3	PK `(project_id, user_id)`
Email уникален без учёта регистра	US-1.2	Уникальный индекс `lower(email)`
Шесть статусов, «Готова» и «Закрыта» - завершённые	US-4.1, US-10.1	Справочник `task_statuses.is_completed`
Закрывает задачу только менеджер	H2, US-4.7	Право `task.close` в `role_permissions`; проверка в приложении
Приглашение действует 7 дней, одно активное на email	H1, US-9	`expires_at`; частичный уникальный индекс
При офлайн-конфликте побеждает сервер	H3	`tasks.version`, оптимистическая блокировка
Проект архивируется, физически удаляет только администратор	H4	`archived_at`; каскад от `projects`

О просрочке - push исполнителю, email-дайджест менеджеру, без повторов	H5, US-10	`overdue_notified_at`, `notifications`
Возврат на доработку - с комментарием	US-8.4	`task_history.reason_comment_id`

5. Сущности, связи и жизненный цикл

- **Задача:** `new → in_progress → review → done → closed`. Переход `review → in_progress` - возврат на доработку. Статус `postponed` доступен из незавершённых статусов. Каждое изменение увеличивает `version` и пишется в `task_history`.
- **Приглашение:** создано → принято (`accepted_at`, `accepted_by`) / отозвано (`revoked_at`) / истекло (`expires_at < now()`). Принятое и отозванное одновременно быть не может (`CHECK`).
- **Уведомление:** `pending → sent | failed`, затем прочитано (`read_at`).
- **Проект:** активен → в архиве (`archived_at`) → физически удалён администратором.

6. Логическая модель

Сущность	**Назначение**	**Ключевые атрибуты**	**Связи**	**Ограничения**
Роль	Набор прав	код, название	M:N с правом; 1:N с пользователем и участником	код уникален
Право	Атомарное действие	код	M:N с ролью	код уникален
Статус задачи	Справочник жизненного цикла	код, порядок, завершённый	1:N с задачей	код и порядок уникальны
Тип задачи	Справочник	код	1:N с задачей	код уникален
Пользователь	Учётная запись	логин, email, хеш пароля, роль	1:N с токенами, проектами, задачами	логин и email уникальны без регистра

Токен пользователя	Подтверждение email, сброс пароля	назначение, хеш, срок	N:1 с пользователем	хеш уникален
Токен устройства	Адрес push	платформа, токен	N:1 с пользователем	токен уникален
Рабочее пространство	Группа проектов	название, владелец	1:N с проектом	-
Проект	Контейнер задач и команды	название, архив	1:N с участниками, приглашениями, задачами, тегами	-
Участник проекта	Связь пользователь–проект с атрибутом роли	роль, дата	N:1 с проектом, пользователем, ролью	пара уникальна
Приглашение	Приглашение по email	email, роль, хеш токена, срок	N:1 с проектом	одно активное на email
Задача	Единица работы	название, статус, тип, исполнитель, срок, версия	N:1 с проектом, статусом, типом; 1:N с комментариями, вложениями, историей	исполнитель из проекта
Тег	Метка в проекте	название, цвет	M:N с задачей	название уникально в проекте
Комментарий	Обсуждение	текст, автор	N:1 с задачей	не пустой
Вложение	Метаданные файла	имя, тип, размер, ключ хранилища	N:1 с задачей	размер > 0
Запись истории	Изменение поля задачи	поле, старое и новое значение, версия	N:1 с задачей	одна запись на поле и версию

Уведомление	Сообщение пользователю	событие, канал, статус доставки	N:1 с пользователем, задачей	значения из перечня

7. Физическая модель (ключевые таблицы)

Остальные таблицы полностью определены в DDL (п. 10).

Таблица	**Столбец**	**Тип**	**NULL**	**Ограничения**	**Назначение**
tasks	id	bigint identity	нет	PK	Технический ключ
tasks	project_id	bigint	нет	FK projects, CASCADE	Проект
tasks	title	text	нет	не пустой	Название
tasks	status_id	smallint	нет	FK task_statuses, RESTRICT	Статус
tasks	type_id	smallint	нет	FK task_types, RESTRICT	Тип
tasks	author_id	bigint	нет	FK users, RESTRICT	Автор
tasks	assignee_user_id	bigint	да	FK (project_id, assignee_user_id) → project_members	Исполнитель
tasks	due_date	date	да	-	Срок
tasks	overdue_notified_at	timestamptz	да	-	Когда отправлено напоминание о просрочке
tasks	version	integer	нет	> 0, default 1	Оптимистическая блокировка
project_members	project_id, user_id	bigint	нет	PK, FK	Участник

project_members	role_id	smallint	нет	FK roles	Роль в проекте
invitations	token_hash	text	нет	UNIQUE	Хеш токена ссылки
invitations	expires_at	timestamptz	нет	> created_at	Срок действия (7 дней, H1)
invitations	accepted_at / accepted_by	timestamptz / bigint	да	заполняются вместе	Факт принятия
notifications	delivery_status	text	нет	pending / sent / failed	Статус доставки
notifications	attempts	smallint	нет	>= 0	Число попыток отправки

8. Архитектурные решения

- **Идентификаторы:** `bigint GENERATED ALWAYS AS IDENTITY`. Маршруты Event Modeling используют числовые `{id}`, а требований к публичным непредсказуемым идентификаторам нет. От перебора чужих задач защищает проверка доступа к объекту, а не тип ключа.
- **Справочники вместо `ENUM`:** статусы и типы - таблицы. У статуса есть дополнительные атрибуты (порядок, признак завершения), а в объектной модели ЛР2 статус и тип - редактируемые объекты. Короткие неизменяемые перечни (`channel`, `delivery_status`, `purpose`, `field`) заданы `CHECK`.
- **История:** текущее состояние хранится в `tasks`, история - в отдельной `task_history`. Основная таблица не превращается в журнал.
- **Удаление:** мягкое удаление не используется. Проект архивируется (`archived_at`), физически его удаляет администратор каскадом. Задачи, комментарии и вложения удаляются только вместе с проектом или задачей.
- **Исполнитель:** FK `tasks → project_members` объявлен как `NO ACTION`, а не `RESTRICT`. Проверка `NO ACTION` выполняется в конце оператора, поэтому каскадное удаление проекта, которое удаляет и участников, и задачи, не блокируется. Удаление отдельного участника с назначенными задачами при этом запрещено.
- **`JSONB`, партиционирование, денормализация:** не используются, сценариев для них нет (A5).

- **Секреты:** токены приглашений и сброса пароля хранятся только как хеш, пароль - только как хеш.

9. ER-диаграмма

Навык построил ER-диаграмму с атрибутами всех таблиц. Ниже - связи между таблицами.

Ключевые связи:

- задача → участник проекта (составной ключ гарантирует один проект);
- задача ↔ тег через `task_tags` в пределах одного проекта;
- запись истории → комментарий с причиной возврата (US-8).

10. SQL DDL

Навык сформировал полный исполнимый DDL-скрипт: схема `pinkmine`, 19 таблиц, ограничения, индексы и комментарии к неочевидным объектам. Ниже фрагмент - таблица задач. Составной внешний ключ `(project_id, assignee_user_id)` не даёт назначить исполнителем пользователя из другого проекта (находка R1), а поле `version` используется для оптимистической блокировки (H3).

```
CREATE TABLE pinkmine.tasks (
    id                  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    project_id          bigint NOT NULL REFERENCES pinkmine.projects (id) ON
DELETE CASCADE,
    title               text NOT NULL CHECK (btrim(title) <> ''),
    description         text,
    status_id           smallint NOT NULL REFERENCES pinkmine.task_statuses (id)
ON DELETE RESTRICT,
    type_id             smallint NOT NULL REFERENCES pinkmine.task_types (id) ON
DELETE RESTRICT,
    author_id           bigint NOT NULL REFERENCES pinkmine.users (id) ON DELETE
RESTRICT,
    assignee_user_id    bigint,
    due_date            date,
    overdue_notified_at timestamptz,
    version             integer NOT NULL DEFAULT 1 CHECK (version > 0),
    created_at          timestamptz NOT NULL DEFAULT now(),
    updated_at          timestamptz NOT NULL DEFAULT now(),
    UNIQUE (project_id, id),
    -- NO ACTION: проверка в конце оператора, каскадное удаление проекта не
блокируется
    FOREIGN KEY (project_id, assignee_user_id)
        REFERENCES pinkmine.project_members (project_id, user_id) ON DELETE NO
ACTION
);
```

11. Тестовые данные

Навык подготовил отдельный скрипт тестовых данных: справочники ролей, прав, статусов и типов, а также пользователей, проекты, задачи, комментарии, историю и уведомления на основе персонажей ЛР2. Набор включает граничные случаи: архивный проект, отозванное приглашение, просроченную задачу, уведомление с ошибкой доставки.

12. Основные SQL-запросы

Навык подготовил пять ключевых запросов: список задач проекта с фильтрами, смену статуса, фиксацию просрочек, принятие приглашения и ленту уведомлений. Ниже - запрос смены статуса задачи с оптимистической блокировкой (US-4, US-8, H3), выполняемый в одной транзакции.

```
BEGIN;

-- 1. Заблокировать задачу и прочитать состояние для проверок приложения
SELECT t.project_id, t.version, s.code AS current_status, p.archived_at,
pm.role_id AS project_role_id
FROM pinkmine.tasks AS t
JOIN pinkmine.task_statuses AS s ON s.id = t.status_id
JOIN pinkmine.projects AS p ON p.id = t.project_id
LEFT JOIN pinkmine.project_members AS pm ON pm.project_id = t.project_id AND
pm.user_id = $2::bigint
WHERE t.id = $1::bigint
FOR UPDATE OF t;

-- 2. Приложение проверяет: доступ, архив, версию, право task.close, комментарий
причины

-- 3. Изменить статус только при совпадении версии
UPDATE pinkmine.tasks
SET status_id = (SELECT s.id FROM pinkmine.task_statuses AS s WHERE s.code =
$3::text),
    version = version + 1,
    updated_at = now()
WHERE id = $1::bigint
  AND version =
4:: integer RETURNING version, updated_at t; -- 4. Записать историю новой версией INSERT INTO pinkmine. task_history (ta
1::bigint, $5::integer, $2::bigint, 'status', $6::text, $3::text, $7::bigint);

COMMIT;
```

- **Результат:** новая версия задачи и запись истории.
- **Граничные случаи:**
  - если `UPDATE` вернул 0 строк, версия устарела: транзакция откатывается, сервер отвечает 409;
  - повторная вставка истории для той же версии отклоняется уникальным ограничением.

13. Индексы и производительность

Индекс	Таблица	Столбцы	Сценарий	Обоснование	Риски
users_email_lower_uq	users	lower(email)	Регистрация, вход	Уникальность без учёта регистра	Запросы должны использовать `lower(email)`
invitations_active_email_uq	invitations	project_id, lower(emai	Приглашение	Одно активное	Истёкшее, но не отозванное приглашение нужно

		l), частичный		приглашение на email	отзывать перед новым
tasks_project_status_idx	tasks	project_id, status_id, due_date, id	Доска и список проекта	Фильтр по проекту и статусу, сортировка по сроку	Замедляет запись при частых изменениях статуса (приемлемо)
tasks_assignee_idx	tasks	assignee_user_id, project_id, частичный	«Мои задачи», проверка составного FK	Одна структура для двух сценариев	-
tasks_overdue_check_idx	tasks	due_date, частичный	Ежедневная проверка просрочек	Индексирует только задачи без напоминания	-
task_tags_tag_idx	task_tags	project_id, tag_id	Фильтр по тегу, удаление тега	PK начинается с task_id и не покрывает поиск по тегу	-
comments_task_idx	comments	task_id, created_at, id	Обсуждение в карточке	Хронологический порядок	-
notifications_recipient_idx	notifications	recipient_id, created_at DESC, id DESC	Лента уведомлений	Пагинация по ключу	Рост таблицы, см. риски
notifications_unread_idx	notifications	recipient_id, частичный	Счётчик непрочитанных	Индексирует только непрочитанные	-
notifications_pending_idx	notifications	created_at, частичный	Отправка push и email	Очередь доставки	-

FK `tasks.status_id` и `tasks.type_id` без отдельных индексов: справочники маленькие, удаление статусов редкое. Индексы на остальные FK созданы. Для Q1 и Q3 перед вводом в эксплуатацию нужно выполнить `EXPLAIN (ANALYZE, BUFFERS)` на репрезентативных данных.

14. Транзакции и конкурентный доступ

Операция	**Атомарные действия**	**Защита**
Смена статуса (Q2)	Блокировка задачи, обновление, запись истории	`READ COMMITTED` + `SELECT … FOR UPDATE` + условие `version = $4`. Повтор из офлайн-очереди блокирует `UNIQUE (task_id, task_version, field)`
Принятие приглашения (Q4)	Проверка приглашения, добавление участника, отметка принятия	`FOR UPDATE` на приглашении, `ON CONFLICT DO NOTHING`
Создание приглашения	Вставка приглашения	Частичный уникальный индекс. При нарушении уникальности существующее приглашение обновляется (новый токен и срок, US-9.6)
Фиксация просрочек (Q3)	Отметка задач и постановка уведомлений	Один оператор `UPDATE … RETURNING` и вставка в той же транзакции
Отправка уведомлений	Выбор `pending`, вызов FCM или SMTP, обновление статуса	Выборка `FOR UPDATE SKIP LOCKED` пачкой. Внешний вызов выполняется вне длинной транзакции, затем отдельно `UPDATE delivery_status, attempts`

Таблица `notifications` одновременно служит очередью отправки. Уведомление записывается в той же транзакции, что и изменение задачи, поэтому событие не теряется при сбое FCM.

15. План миграции

1. Миграция 1: DDL из п. 10 (схема, таблицы, индексы). Черновая ERD не была развёрнута, поэтому данных для переноса нет.

2. Миграция 2: справочные данные - `roles`, `permissions`, `role_permissions`, `task_statuses`, `task_types`(первая часть скрипта п. 11).
3. Тестовые пользователи и задачи из п. 11 загружаются только в тестовую среду.
4. Будущие несовместимые изменения (например, переименование кода статуса) выполнять по схеме expand-and-contract: добавить новое значение → перевести приложение → перенести данные → удалить старое значение отдельной миграцией.

16. Риски и открытые вопросы

Риск	**Последствие**	**Снижение**
Модель ролей A1 не согласована с командой	Переделка `users.role_id` и проверок прав	Подтвердить на ревью до реализации US-7
Часовой пояс просрочки (A2)	Напоминание приходит на день раньше или позже	Зафиксировать часовой пояс системы в спецификации
Каскадное удаление вложений оставляет файлы в хранилище (A3)	Рост хранилища, утечка данных	Фоновая очистка по отсутствующим `storage_key`
Рост `notifications`	Замедление ленты	Срок хранения прочитанных уведомлений, удаление по расписанию
`reason_comment_id` не проверяет, что комментарий относится к той же задаче	Ссылка на чужой комментарий	Проверка в приложении (постановка API, раздел 3.2)
Правило «срок не в прошлом» (US-2.8) не выражается `CHECK`	Некорректный срок через обход API	Проверка в приложении

17. Итоговая проверка production-готовности

Пункт	**Статус**
Полнота сущностей и связей	готово

PK, FK, правила удаления, уникальность, CHECK	готово
Невозможность некорректных данных (исполнитель и теги из другого проекта)	готово
Порядок DDL, отсутствие дублирующих индексов	готово
Исполнимость DDL и тестовых данных	требует уточнения: скрипты не выполнялись
Конкурентный доступ и идемпотентность	готово
Модель ролей	требует уточнения (A1)
Безопасность (хеши токенов и паролей)	готово
Производительность	требует уточнения: нужны `EXPLAIN` и реальные объёмы
Миграции	готово
Партиционирование, `JSONB`, денормализация	неприменимо

Как использован результат

- Черновая ERD, полученная по промпту 2.11, заменена моделью навыка. На странице Схема базы данных опубликованы ER-диаграмма, ORM-отображение и полный DDL.
- По находкам ревью исправлены другие диаграммы задания 4:
  - `cls_domain` и `cls_rbac`: глобальная роль пользователя (R11), вычисляемая просрочка (R4);
  - `seq_overdue`: `overdue_notified_at` вместо хранимого признака просрочки;
  - `obj_project_review`: глобальная роль вместо признака администратора.
- Модель БД стала входным материалом для второго навыка (раздел 3.2).

3.2. Навык `getanalyst-restapi-designer-skill`

Назначение навыка: готовить полную постановку задачи ровно на один REST API-метод: контракт, доступы, валидации, ответы и ошибки, алгоритм, маппинг, логирование и мониторинг. Справочники:

- `task-template.md` - шаблон постановки;
- `design-standards.md` - правила проектирования;
- `method-checklists.md`, раздел PATCH;
- `operational-requirements.md` - логирование, метрики, аудит;
- `quality-checklist.md` - финальная проверка.

Выбор метода. Навык работает с одним методом за раз. Выбран `PATCH /api/tasks/{id}/status` (срезы S9, S11, S12, S17 Event Modeling). Это самый нагруженный правилами метод: права (H2), офлайн-конфликты (H3), возврат на доработку (US-8), архивные проекты (US-14).

Промпт запуска:

```
Используй $getanalyst-restapi-designer-skill, чтобы подготовить полную постановку задачи на один REST API-метод:
PATCH /api/tasks/{id}/status - изменение статуса задачи в Pinkmine (срезы S9, S11, S12, S17 Event Modeling).
```

Шаг 1. Запрос материалов навыком

По правилам навыка он один раз запрашивает доступные материалы:

Запрошено навыком	**Передано**
Base URL, имя API, версия	Нет. Есть только относительный путь `/api/...` из Event Modeling
Схема архитектуры	Ограниченные контексты Event Storming: Flask Blueprints `auth`, `projects`, `tasks`, `notifications`; FCM; SMTP
Дизайн экрана	`wf2_my_tasks_mobile` (Event Modeling); смена статуса в два действия (US-4.2)
Модель БД	Результат навыка 3.1: таблицы `tasks`, `task_statuses`, `task_history`, `project_members`, `projects`, `role_permissions`, `comments`
Особенности алгоритма	US-4, US-8, US-12, US-14; правила H2 и H3; сценарии UC-T3 и UC-T4

Шаг 2. Блокирующие вопросы и ответы

Навык задал вопросы, без ответов на которые контракт нельзя сделать однозначным. Ответы дал исполнитель задания.

Вопрос навыка	**Ответ**
Схема аутентификации	Bearer-токен
Единый формат ошибки	Проектный объект `{ errorCode, message, details[], traceId }`
Контроль конкурентных изменений	`version` в теле запроса, при расхождении - 409
Ответ при успехе	200 с обновлённым представлением задачи

Шаг 3. Результат навыка

Нюансы и рекомендации

1. **Имя path-параметра.** В Event Modeling маршрут записан как `/api/tasks/{id}/status`. Рекомендация - предметное имя `{taskId}`. Риск: нет, URI не меняется.
2. **Регистр значений статуса.** В проекте принят формат `in_progress` (критерий US-5.6: `?status=in_progress`). Общая рекомендация - `IN_PROGRESS`. Риск: нет, если формат единый во всём API.
3. **Матрица переходов статусов** в требованиях полностью не задана. Подтверждены три правила:
   - статус `closed` выставляет только пользователь с правом `task.close` (H2, US-4.7);
   - переход `review → in_progress` (возврат на доработку) требует комментария с причиной (US-8.4);
   - задачи архивного проекта не изменяются (US-14.3). Остальные переходы между шестью статусами разрешены (US-4.1). Риск: если команда введёт строгую матрицу, понадобится новый код ошибки.
4. **Неизвестные поля в теле запроса.** Политика проекта не задана. Рекомендация - отклонять, чтобы опечатки клиента не проходили молча.
5. **Существование задачи для постороннего пользователя.** Рекомендация - отвечать 404, а не 403, чтобы не раскрывать существование задач чужих проектов (защита от IDOR).

В постановке ниже применено: `{taskId}`; значения статуса в нижнем регистре; только подтверждённые правила переходов; неизвестные поля отклоняются; задачи чужих проектов - 404.

[PATCH /api/tasks/{taskId}/status] Изменить статус задачи

Связанные документы

Документ	**Идентификатор**
Пользовательские истории	US-4, US-8, US-12, US-14
Сценарии вариантов использования	UC-T3 «Изменить статус задачи», UC-T4 «Проверить выполненную задачу»
Event Modeling	Срезы S9, S11, S12, S17
Event Storming	Противоречия H2, H3; интеграционное событие «Статус задачи изменён»
Схема БД	Database-Schema, схема `pinkmine`
Диаграммы последовательности	`seq_status_mobile`, `seq_task_review`

Общее описание

Метод изменяет статус задачи. Инициатор - авторизованный участник проекта задачи: исполнитель с Android или менеджер проекта с веб-клиента. Метод используется, чтобы:

- взять задачу в работу;
- отправить её на проверку;
- принять или вернуть на доработку;
- закрыть;
- отложить.

Мобильное приложение вызывает метод и при отправке изменений из офлайн-очереди. На вход передаются новый статус, версия задачи, на которой основано изменение, и, при возврате на доработку, идентификатор комментария с причиной. При успехе статус сохраняется, версия задачи увеличивается, изменение записывается в историю, а контекст уведомлений получает событие «Статус задачи изменён».

Доступы

- Метод защищённый. Аутентификация - `Authorization: Bearer <token>`, credentials обязательны.

- Доступ к задаче имеет пользователь, который состоит в проекте задачи (`project_members`), а также пользователь с глобальной ролью `admin`.
- Перевод в статус `closed` разрешён, только если глобальная роль пользователя или его роль в проекте содержит право `task.close`.
- Изменение статуса разрешено, если роль пользователя в проекте содержит право `task.change_status`.

Нефункциональные требования

- **Идемпотентность.** Повторная отправка запроса, который уже применён (текущий статус совпадает с запрошенным), не изменяет задачу, не создаёт запись истории и уведомление и возвращает `200`.
- **Конкурентность.** Изменение применяется, только если версия задачи совпадает с переданной. Одновременные изменения одной задачи выполняются последовательно.

Ограничения

- Создание комментария с причиной возврата выполняется отдельным методом `POST /api/tasks/{taskId}/comments` и в эту задачу не входит.
- Формирование и доставка уведомлений (контекст «Уведомления», FCM, SMTP) не разрабатываются в рамках этой задачи.
- Офлайн-очередь мобильного клиента не разрабатывается в рамках этой задачи.

Контракт запроса

```
Method type: PATCH
Method name: /api/tasks/{taskId}/status
Authorization: Bearer token
```

Headers

Header	Обяз.	Формат	Default	Описание	Ошибка при нарушении
Authorization	да	`Bearer <token>`	-	Токен доступа пользователя	401 `UNAUTHORIZED`
Content-Type	да	`application/json`	-	Формат тела запроса	415 `UNSUPPORTED_MEDIA_TYPE`

Path-параметры

Параметр	Обяз.	Тип/формат	Описание	Ограничения	Пример

taskId	да	integer, int64	Идентификатор задачи	Целое число ≥ 1	42

Пример запроса

```
PATCH /api/tasks/42/status
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.test.signature
Content-Type: application/json
```

Тело запроса (Body)

```
{
  "status": "in_progress",
  "version": 3,
  "reasonCommentId": 118
}
```

JSON path	Обяз.	Nullable	Тип/формат	Описание	Ограничения/валидация	Ошибка
status	да	нет	string, enum	Новый статус	`new`, `in_progress`, `review`, `postponed`, `done`, `closed`	400 `VALIDATION_ERROR`
version	да	нет	integer, int32	Версия задачи, на которой основано изменение	Целое ≥ 1	400 `VALIDATION_ERROR`
reasonCommentId	нет	да	integer, int64	Комментарий с причиной возврата на доработку	Обязателен и не `null` при переходе `review → in_progress`; комментарий должен принадлежать этой задаче. В остальных случаях игнорируется	422 `REASON_COMMENT_REQUIRED`, 422 `REASON_COMMENT_INVALID`

(другие поля)	-	-	-	-	Не допускаются	400 `VALIDATION_ERROR`

Пример ответа - Успех

Условие: статус изменён, или текущий статус уже совпадает с запрошенным.

`200 OK`, `Content-Type: application/json`

```
{
  "id": 42,
  "projectId": 7,
  "status": "in_progress",
  "version": 4,
  "updatedAt": "2026-10-14T07:12:03Z"
}
```

JSON path	Обяз.	Nullable	Тип/формат	Описание
id	да	нет	integer, int64	Идентификатор задачи
projectId	да	нет	integer, int64	Идентификатор проекта
status	да	нет	string, enum	Текущий статус задачи
version	да	нет	integer, int32	Текущая версия задачи
updatedAt	да	нет	string, date-time RFC 3339, UTC	Момент последнего изменения задачи

Ошибочные ответы

Условие	Этап проверки	HTTP	errorCode	message	Response headers
Нет токена, токен истёк или недействителен	3	401	`UNAUTHORIZED`	Требуется авторизация	`WWW-Authenticate: Bearer`
Content-Type не `application/json`	1	415	`UNSUPPORTED_MEDIA_TYPE`	Неподдерживаемый формат запроса	-

Некорректный JSON, тип, значение `status`, `taskId` или лишнее поле	1, 4	400	`VALIDATION_ERROR`	Запрос содержит некорректные данные	-
Задача не найдена или пользователь не состоит в её проекте	5	404	`TASK_NOT_FOUND`	Задача не найдена	-
Перевод в `closed` без права `task.close`	6	403	`TASK_CLOSE_FORBIDDEN`	Закрыть задачу может менеджер проекта	-
Нет права `task.change_status`	6	403	`STATUS_CHANGE_FORBIDDEN`	Недостаточно прав для изменения статуса	-
Проект задачи в архиве	7	409	`PROJECT_ARCHIVED`	Проект в архиве, изменение задач недоступно	-
Версия не совпадает с текущей	9	409	`TASK_VERSION_CONFLICT`	Задача изменена другим пользователем	-
Возврат на доработку без комментария	10	422	`REASON_COMMENT_REQUIRED`	Укажите комментарий с причиной возврата	-
Комментарий не найден или относится к другой задаче	10	422	`REASON_COMMENT_INVALID`	Комментарий с причиной не найден в этой задаче	-
Непредвиденная ошибка	любой	500	`INTERNAL_ERROR`	Внутренняя ошибка сервера	-

Пример ответа - Ошибка авторизации

401 Unauthorized, WWW-Authenticate: Bearer

```
{
  "errorCode": "UNAUTHORIZED",
  "message": "Требуется авторизация",
  "details": [],
  "traceId": "b2d6949c-60fa-4ef8-8a18-1bde451f7e7d"
}
```

403 Forbidden

```
{
  "errorCode": "TASK_CLOSE_FORBIDDEN",
  "message": "Закрыть задачу может менеджер проекта",
  "details": [],
  "traceId": "0c4d1e2a-5b7f-4d1c-9a3e-6f2b8c9d0e1f"
}
```

Пример ответа - Запись не найдена

404 Not Found

```
{
  "errorCode": "TASK_NOT_FOUND",
  "message": "Задача не найдена",
  "details": [],
  "traceId": "4e7a2b1c-8d3f-4a6e-b5c9-1d2e3f4a5b6c"
}
```

Пример ответа - Конфликт версии

409 Conflict

```
{
  "errorCode": "TASK_VERSION_CONFLICT",
  "message": "Задача изменена другим пользователем",
  "details": [
    {
      "field": "version",
      "reason": "STALE_VERSION"
    }
  ],
  "traceId": "9a8b7c6d-5e4f-4a3b-8c2d-1e0f9a8b7c6d"
}
```

Пример ответа - Проект в архиве

409 Conflict

```
{
  "errorCode": "PROJECT_ARCHIVED",
  "message": "Проект в архиве, изменение задач недоступно",
  "details": [],
  "traceId": "1f2e3d4c-5b6a-4978-8695-a4b3c2d1e0f9"
}
```

Пример ответа - Семантическая ошибка

422 Unprocessable Content

```
{
  "errorCode": "REASON_COMMENT_REQUIRED",
  "message": "Укажите комментарий с причиной возврата",
  "details": [
    {
      "field": "reasonCommentId",
```

```
      "reason": "REQUIRED_FOR_RETURN_TO_WORK"
    }
  ],
  "traceId": "7d6c5b4a-3f2e-4d1c-8b9a-0f1e2d3c4b5a"
}
```

Пример ответа - Внутренняя ошибка

```
500 Internal Server Error
{
  "errorCode": "INTERNAL_ERROR",
  "message": "Внутренняя ошибка сервера",
  "details": [],
  "traceId": "3c2b1a09-8f7e-4d6c-9b5a-4e3d2c1b0a9f"
}
```

Требования к реализации

1. Blueprint `tasks` принимает запрос. Если `Content-Type` не `application/json`, вернуть 415 `UNSUPPORTED_MEDIA_TYPE`. Если тело - некорректный JSON или `taskId` не целое число ≥ 1, вернуть 400 `VALIDATION_ERROR`.
2. Rate limit для метода не применяется.
3. Проверить Bearer-токен и определить пользователя. При отсутствии, истечении или недействительности токена вернуть 401 `UNAUTHORIZED` с `WWW-Authenticate: Bearer`.
4. Проверить тело:
   - `status` из перечня;
   - `version` - целое ≥ 1;
   - `reasonCommentId`, если передан, - целое ≥ 1 или `null`;
   - других полей нет. При нарушении вернуть 400 `VALIDATION_ERROR` и перечислить поля в `details`.
5. Открыть транзакцию. Прочитать задачу с блокировкой строки `SELECT … FOR UPDATE` (таблицы `tasks`, `task_statuses`, `projects`, `project_members`). Если задачи нет, или пользователь не состоит в её проекте и не имеет глобальной роли `admin`, откатить транзакцию и вернуть 404 `TASK_NOT_FOUND`.
6. Проверить права по `role_permissions`:
   - если запрошен `closed`, а у пользователя нет `task.close`, - 403 `TASK_CLOSE_FORBIDDEN`;
   - если нет `task.change_status`, - 403 `STATUS_CHANGE_FORBIDDEN`. При ошибке откатить транзакцию.
7. Если `projects.archived_at` заполнено, откатить транзакцию и вернуть 409 `PROJECT_ARCHIVED`.
8. Если текущий статус совпадает с запрошенным, завершить транзакцию без изменений и вернуть 200 с текущим представлением задачи (повторная отправка).

9. Если `version` из запроса не равна `tasks.version`, откатить транзакцию и вернуть 409 `TASK_VERSION_CONFLICT`.
10. Если текущий статус `review`, а запрошен `in_progress`:
    - `reasonCommentId` отсутствует или `null` - вернуть 422 `REASON_COMMENT_REQUIRED`;
    - комментарий не найден в `comments` с `task_id = taskId` - вернуть 422 `REASON_COMMENT_INVALID`. При ошибке откатить транзакцию.
11. Обновить задачу: `status_id` нового статуса, `version = version + 1`, `updated_at = now()`, с условием `version` = переданной версии.
12. В той же транзакции вставить запись в `task_history`:
    - `field = 'status'`;
    - старый и новый коды статуса;
    - новая версия;
    - автор - текущий пользователь;
    - `reason_comment_id` - при возврате на доработку. Нарушение `UNIQUE (task_id, task_version, field)` означает параллельное применение того же изменения: откатить транзакцию и вернуть 409 `TASK_VERSION_CONFLICT`.
13. Зафиксировать транзакцию.
14. После фиксации опубликовать интеграционное событие «Статус задачи изменён»: идентификаторы задачи и проекта, старый и новый статус, автор. Ошибка публикации не меняет ответ клиенту, она логируется и учитывается метрикой.
15. Вернуть 200 с представлением задачи. При непредвиденной ошибке на любом шаге откатить транзакцию и вернуть 500 `INTERNAL_ERROR` без внутренних деталей.

Маппинг данных

Request → БД

Источник	**Поле**	**Преобразование/правило**	**Назначение**	**Комментарий**
path	taskId	-	`tasks.id`	Поиск задачи
body	status	Код → `task_statuses.id` по `task_statuses.code`	`tasks.status_id`	-
body	version	Сравнение с текущей	`tasks.version`	Оптимистическая

				блокировка
body	reasonCommentId	Проверка `comments.task_id = taskId`	`task_history.reason_comment_id`	Только для `review → in_progress`
токен	пользователь	-	`task_history.author_id`	Субъект определяется только по токену
вычисление	-	`version + 1`	`tasks.version`, `task_history.task_version`	-
текущее время	-	`now()`	`tasks.updated_at`, `task_history.created_at`	-

БД → Response

JSON path	Тип	Обяз.	Nullable	Источник	Преобразование/правило
id	integer	да	нет	`tasks.id`	-
projectId	integer	да	нет	`tasks.project_id`	-
status	string	да	нет	`task_statuses.code` по `tasks.status_id`	-
version	integer	да	нет	`tasks.version`	-

updatedAt	string	да	нет	`tasks.updated_at`	RFC 3339, UTC

Логирование и мониторинг

Логи

- **Поля структурированного лога:** timestamp, компонент `tasks`, метод `PATCH`, шаблон маршрута `/api/tasks/{taskId}/status`, traceId, технический идентификатор пользователя, taskId, HTTP-статус, errorCode, длительность, старый и новый статус при успехе.
- **Не логировать:** значение токена, тело запроса целиком, тексты комментариев.

Метрики

- **`requests_total`** по маршруту, методу, классу статуса и errorCode.
- **Гистограмма длительности:** p50, p95, p99.
- **Число конфликтов `TASK_VERSION_CONFLICT`:** показатель офлайн-конфликтов (H3).
- **Повторные отправки:** число запросов, где статус уже совпадал.
- **Ошибки публикации** события «Статус задачи изменён».
- **Бизнес-метрика:** число переходов по паре «старый → новый статус».

Алерты

- Рост доли ответов 5xx.
- Превышение целевой задержки, когда команда её утвердит.
- Резкий рост `TASK_VERSION_CONFLICT`.
- Ошибки публикации события «Статус задачи изменён».

Аудит

Каждое успешное изменение фиксируется в `task_history`:

- актор;
- действие «смена статуса»;
- задача;
- статус до и после;
- версия;
- время;
- причина возврата.

Связь с логом - через traceId.

Автор скилла - Екатерина Ананьева Сообщество системных аналитиков **GetAnalyst** · getanalyst.ru · Telegram

Как использован результат

- Постановка задаёт контракт для диаграмм `seq_status_mobile` и `seq_task_review`. В `seq_status_mobile`ответ на конфликт приведён к постановке: 409 `TASK_VERSION_CONFLICT`, затем клиент запрашивает актуальную задачу.
- Решения, принятые на шаге 2 (Bearer-токен, формат ошибки, `version` → 409), используются как стандарт для остальных методов API.
- Постановку можно разместить на странице вики «API-проекта» (сейчас заглушка) как первый описанный метод.

4. Расширение пользовательских историй (примеры)

По промпту 2.2 найдены команды Event Storming без историй. Дополнены критерии US-1, 2, 4, 6, добавлены истории US-8…US-14. Полный текст - на странице Пользовательские истории.

№	История	Приоритет
US-8	Как менеджер проекта, я хочу принимать выполненную задачу или возвращать её на доработку с комментарием…	High
US-9	Как менеджер проекта, я хочу пригласить в проект человека по email…	Medium
US-10	Как менеджер проекта, я хочу видеть просроченные задачи…	Medium
US-11	Как участник команды, я хочу оставлять комментарии и прикреплять файлы…	Medium
US-12	Как участник команды, я хочу менять статус задачи без интернета…	Low
US-13	Как зарегистрированный пользователь, я хочу восстановить забытый пароль…	Medium
US-14	Как менеджер проекта, я хочу архивировать завершённый проект…	Low

Пример критериев приёмки (US-8):

1. Когда исполнитель переводит задачу в статус «На проверке», менеджер получает уведомление.
2. «Вернуть на доработку» требует комментарий с причиной; без комментария кнопка неактивна.
3. После возврата задача получает статус «В работе», а исполнитель - уведомление с текстом комментария.

5. UML-диаграммы (кратко)

Вид	**Требование**	**Выполнено**	**Пример**
Use case + сценарии	Диаграммы и 3–5 детальных сценариев	2 диаграммы (`uc_overview`, `uc_tasks`), 5 сценариев UC-T1…UC-T5	UC-T3 «Изменить статус задачи»
Деятельность	2–3	3 (1 из задания 2 + `act_task_review`, `act_invite`)	Цикл проверки и возврата задачи
Классы и объекты	≥ 5	6 (1 из задания 2 + `cls_domain`, `cls_rbac`, `cls_backend`, `cls_mobile`, `obj_project_review`)	Модель ролей и прав
Последовательность	≥ 5	6 (`seq_auth`, `seq_create_assign`, `seq_status_mobile`, `seq_invite`, `seq_task_review`, `seq_overdue`)	Смена статуса онлайн и офлайн
Компоненты	1	`cmp_system`	Blueprints по ограниченным

			контекстам
Пакеты	1	`pkg_structure`	Слои сервера и клиента
Развёртывание	1	`dep_system`	Сервер приложений, БД, FCM, SMTP
ERD / ORM	1 / при необходимости	`erd_pinkmine`, `orm_pinkmine` (навык 3.1)	19 таблиц

Задание 5. Проектирование системы на основе метода EventStorming

5.1. Инструменты и нотация

Спецификация выполнена на языке PlantUML с использованием рекомендуемой библиотеки plantuml-libs (пакет eventstorming, репозиторий tmorin/plantuml-libs), подключённой локально. В библиотеке нет отдельных элементов для противоречий и поворотных событий, поэтому они добавлены в общий файл es-common.iuml как собственные стереотипы Hotspot и PivotalEvent в той же стилистике. Моделирование проведено на трёх уровнях, Big Picture, Process Modeling и Software Design, что соответствует семи этапам метода из методических указаний.

Элемент	Цвет	Назначение в модели Pinkmine
Событие предметной области (DomainEvent)	оранжевый	факт, уже произошедший в системе, глагол в прошедшем времени: «Задача создана»
Поворотное событие (PivotalEvent)	тёмно-оранжевый, жирная рамка	опорные для бизнеса события: «Проект создан», «Задача создана», «Задача закрыта»
Противоречие (Hotspot)	малиновый	спорный вопрос, требующий решения команды

Элемент	Цвет	Назначение в модели Pinkmine
Команда (Command)	голубой	намерение изменить состояние, повелительное наклонение: «Назначить исполнителя»
Действующее лицо (Person)	жёлтый (фигура)	менеджер проекта, участник команды, приглашённый пользователь
Правило (Policy)	сиреневый	реакция «когда произошло А, выполнить Б»
Модель чтения (ReadModel)	зелёный	информация для принятия решения: «Мои задачи», «Канбан-доска»
Интерфейс (UserInterface)	светло-зелёный	экран веб-сайта или мобильного приложения
Внешняя система (System)	розовый	SMTP-сервис, Firebase Cloud Messaging, Pinkmine API
Таймер (Timer)	серый	запуск по расписанию (ежедневная проверка сроков)
Агрегат (Aggregate)	жёлтый	кластер сущностей, принимающий команды и порождающий события
Интеграционное событие (IntegrationEvent)	светло-оранжевый	событие, пересекающее границу ограниченного контекста

5.2. Уровень Big Picture (этапы 1–2)

Этап 1. Произвольное исследование

На основе объектной модели и профилей задач из лабораторной работы №2 каждый участник команды самостоятельно выписал события предметной области. После удаления дубликатов и приведения формулировок к прошедшему времени получено 32 события, сгруппированные в пять потоков.

Поток	События предметной области
1. Регистрация и доступ	Пользователь зарегистрирован; Email подтверждён; Пользователь вошёл в систему; Пароль сброшен; Пользователь вышел из системы
2. Проекты и команда	Рабочее пространство создано; Проект создан; Участник приглашён; Приглашение принято; Участник добавлен в проект; Роль участника изменена; Проект архивирован
3а. Планирование задачи	Задача создана; Исполнитель назначен; Срок выполнения установлен; Задача отложена
3б. Выполнение и проверка	Задача взята в работу; Вложение прикреплено; Задача отправлена на проверку; Задача возвращена на доработку; Задача принята; Задача закрыта

Поток	События предметной области
4. Коммуникация по задаче	Комментарий добавлен; Комментарий отредактирован; Тег привязан к задаче
5. Контроль и уведомления	Срок задачи истёк; Уведомление сформировано; Push-уведомление доставлено; Email-уведомление отправлено; Уведомление прочитано; Сводка по проекту сформирована

Этап 2. Временные линии

События упорядочены слева направо по мере их наступления, начиная с успешного сценария. Параллельные потоки расположены друг под другом, поток жизненного цикла задачи разделён на планирование и выполнение, так как эти части выполняют разные роли. Событие «Задача отложена» показано как альтернативная ветка после установки срока, событие «Задача возвращена на доработку» входит в успешный сценарий как типичный цикл проверки. Противоречия привязаны к событиям, вызвавшим вопросы, внешние системы привязаны к событиям доставки уведомлений.

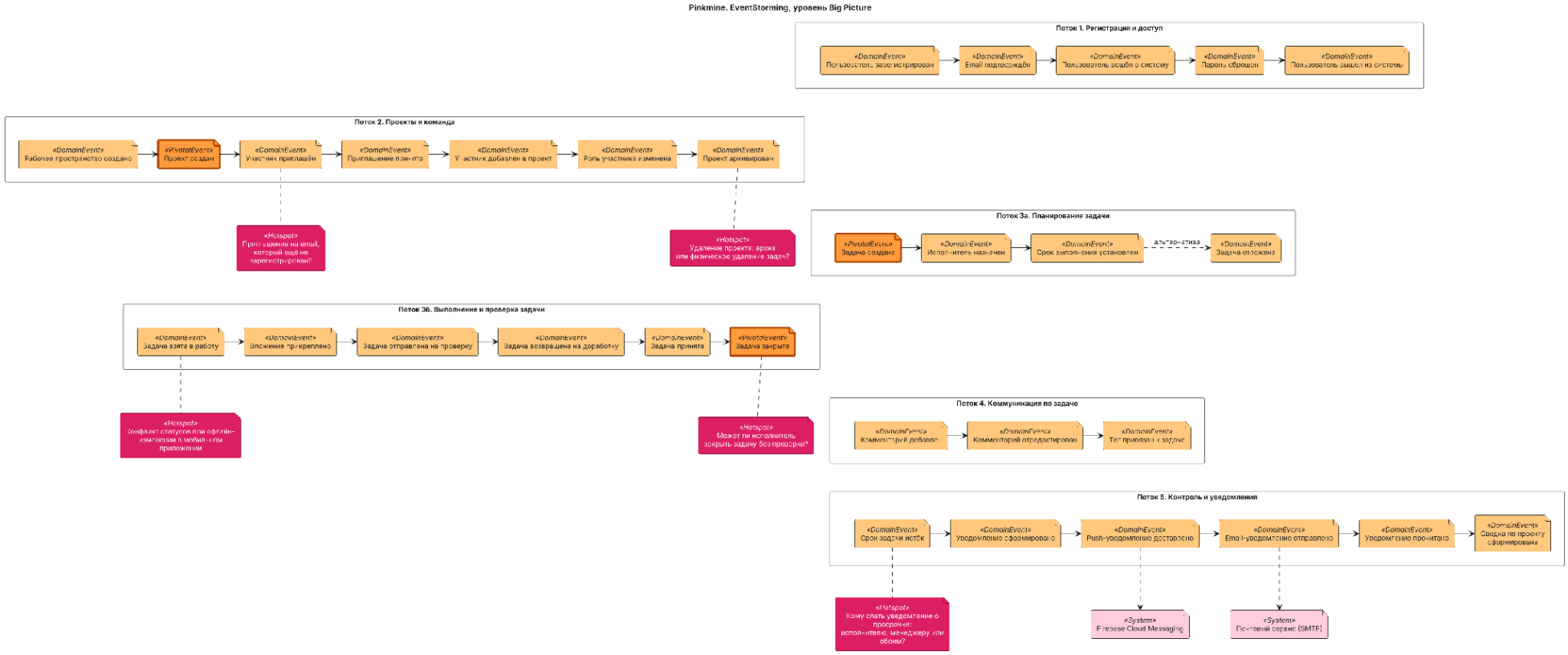


Рисунок 1. EventStorming, уровень Big Picture: временная линия событий Pinkmine

Поворотные события выбраны так, чтобы разделить предметную область на устойчивые этапы: «Проект создан» открывает работу команды, «Задача создана» запускает основной рабочий цикл, «Задача закрыта» фиксирует бизнес-результат, на основе которого считаются метрики успешности из стратегии дизайна (процент завершённых задач).

Противоречия, выявленные при обсуждении, и принятые по ним решения:

№	Противоречие	Принятое решение
H1	Приглашение отправлено на email, который ещё не зарегистрирован в системе	письмо содержит ссылку на регистрацию, приглашение действует 7 дней и привязывается к аккаунту после регистрации
H2	Может ли исполнитель сам закрыть задачу без проверки	нет, исполнитель доводит задачу до статуса «На проверке», принимает и закрывает менеджер проекта
H3	Конфликт статусов при офлайн-изменении в мобильном приложении	изменение ставится в очередь и отправляется при появлении сети, при конфликте сохраняется последнее состояние сервера, пользователь получает сообщение
H4	Удаление проекта: архив или физическое удаление задач	проект архивируется, задачи доступны только для чтения, физическое удаление выполняет только администратор
H5	Кому отправлять уведомление о просрочке	исполнителю приходит push-уведомление, менеджеру email-дайджест просроченных задач

5.3. Уровень Process Modeling (этапы 3–5)

На этом уровне к событиям добавлены команды, действующие лица, правила (политики), модели чтения и внешние системы. Используется грамматика «модель чтения → действующее лицо → команда → система → событие → правило → команда». Смоделированы четыре ключевых процесса, соответствующие ежедневным и эпизодическим сценариям персонажей Джесики (менеджер проекта) и Райана (участник команды) из лабораторной работы №2.

Этап 3. Команды и действующие лица

Команда	Действующее лицо	Платформа	Результирующее событие
Зарегистрировать пользователя	новый пользователь	веб, моб.	Пользователь зарегистрирован
Создать проект	менеджер проекта	веб	Проект создан

Команда	Действующее лицо	Платформа	Результирующее событие
Пригласить участника	менеджер проекта	веб	Участник приглашён
Принять приглашение	приглашённый пользователь	веб, моб.	Приглашение принято
Создать задачу	менеджер проекта	веб	Задача создана
Назначить исполнителя	менеджер проекта	веб	Исполнитель назначен
Взять задачу в работу	участник команды	моб., веб	Задача взята в работу
Прикрепить вложение	участник команды	моб., веб	Вложение прикреплено
Отправить на проверку	участник команды	моб., веб	Задача отправлена на проверку
Принять задачу	менеджер проекта	веб	Задача принята
Вернуть на доработку	менеджер проекта	веб	Задача возвращена на доработку
Закрыть задачу	менеджер проекта	веб	Задача закрыта
Изменить срок выполнения	менеджер проекта	веб	Срок выполнения изменён
Проверить сроки задач	планировщик (таймер)	сервер	Срок задачи истёк

Этап 4. Правила

Правило (политика)	Событие-триггер	Вызываемая команда
Когда задача изменена, записать изменение в историю	Задача создана, Исполнитель назначен, Статус задачи изменён	Записать изменение
Когда исполнитель назначен, уведомить исполнителя	Исполнитель назначен	Отправить уведомление
Когда задача на проверке, уведомить менеджера	Задача отправлена на проверку	Отправить уведомление
Когда задача возвращена, уведомить исполнителя	Задача возвращена на доработку	Отправить уведомление
Каждое утро проверять сроки открытых задач	Таймер «Ежедневно в 09:00»	Проверить сроки задач
Когда срок истёк, уведомить исполнителя и менеджера	Срок задачи истёк	Отправить уведомление о просрочке
Когда участник приглашён, отправить письмо-приглашение	Участник приглашён	Отправить приглашение
Когда приглашение принято, добавить в проект с ролью «Участник команды»	Приглашение принято	Добавить участника в проект

Этап 5. Внешние системы и модели чтения

Внешняя система	Роль	Связанные события
Почтовый сервис (SMTP)	вывод: письма-приглашения, подтверждение email, email-уведомления и дайджест просрочек	Приглашение отправлено, Email-уведомление отправлено
Firebase Cloud Messaging	вывод: push-уведомления в мобильное приложение Flutter	Push-уведомление доставлено
Pinkmine API (Flask)	исследуемая система, принимает команды веб- и мобильного клиентов по REST	все события предметной области

Модель чтения	Кто использует	Содержимое
Канбан-доска проекта	менеджер, участник	задачи проекта по колонкам статусов, фильтры по сроку, типу, исполнителю
Мои задачи	участник команды	задачи, назначенные текущему пользователю, сортировка по сроку
Задачи на проверке	менеджер проекта	задачи в статусе «На проверке» по всем проектам менеджера
Просроченные задачи	менеджер проекта	открытые задачи с истёкшим сроком
Участники проекта и роли	менеджер проекта	состав команды, роли, ожидающие приглашения

Процесс создания задачи и назначения исполнителя (рисунок 2) начинается с Канбан-доски, на которой менеджер видит загрузку команды. Создание задачи фиксируется в истории изменений, назначение исполнителя запускает правило уведомления, которое выбирает канал доставки: push при наличии зарегистрированного мобильного устройства, иначе email.

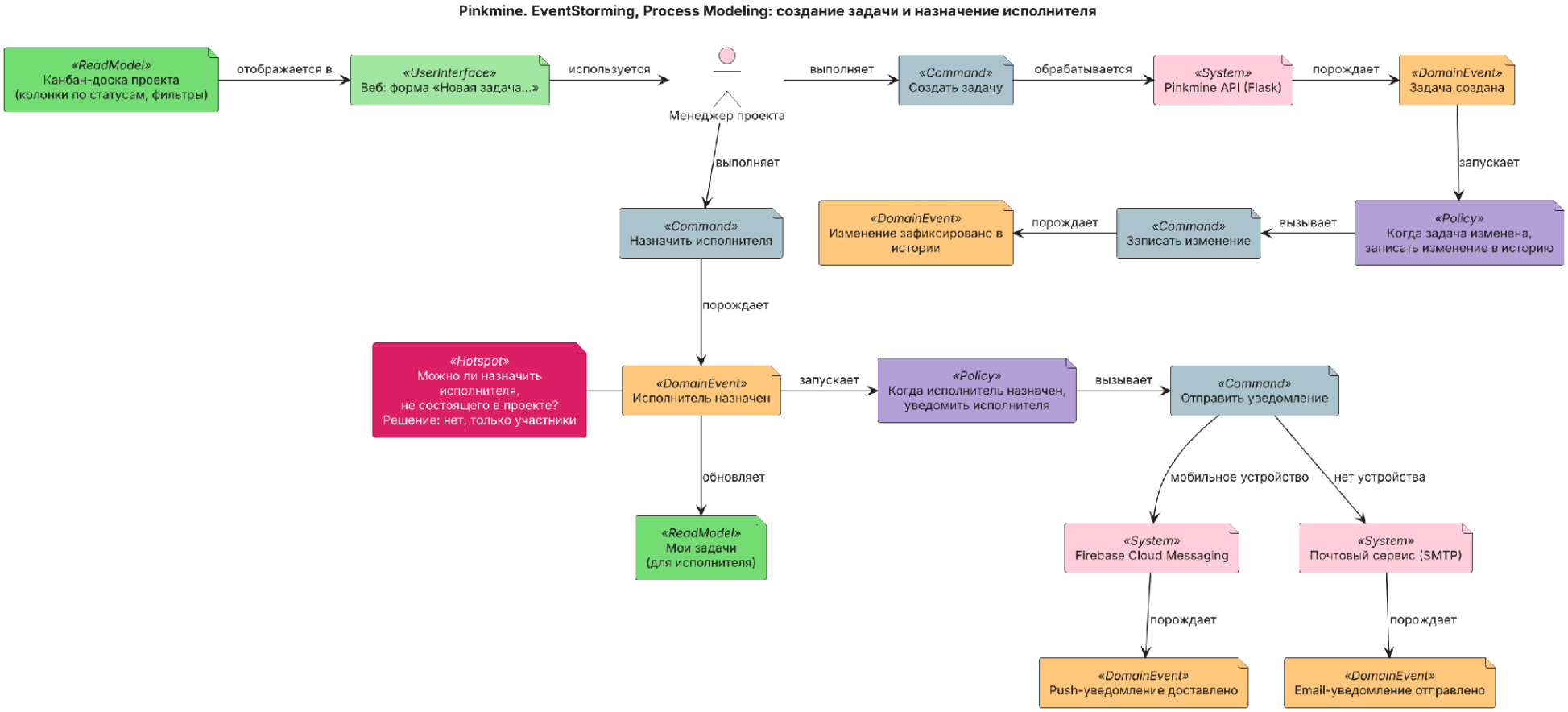


Рисунок 2. Process Modeling: создание задачи и назначение исполнителя

Процесс выполнения и проверки задачи (рисунок 3) отражает ежедневный сценарий участника команды в мобильном приложении и цикл проверки менеджером, включая возврат на доработку. Здесь закреплены решения по противоречиям H2 и H3.

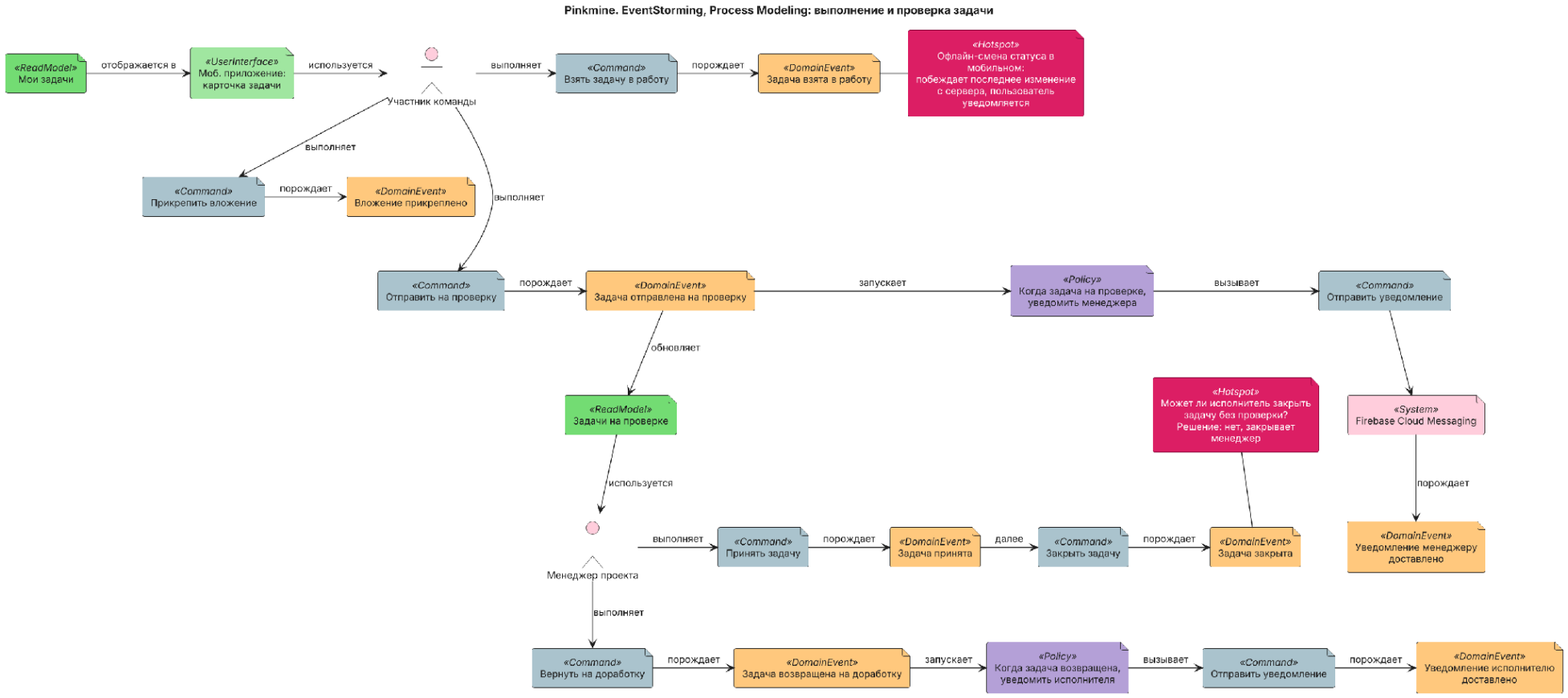


Рисунок 3. Process Modeling: выполнение и проверка задачи

Процесс контроля сроков (рисунок 4) запускается таймером и не имеет действующего лица на первом шаге, это пример автоматизированного правила. Модель чтения «Просроченные задачи» поддерживает эпизодический сценарий менеджера «Разобраться с просроченными задачами».

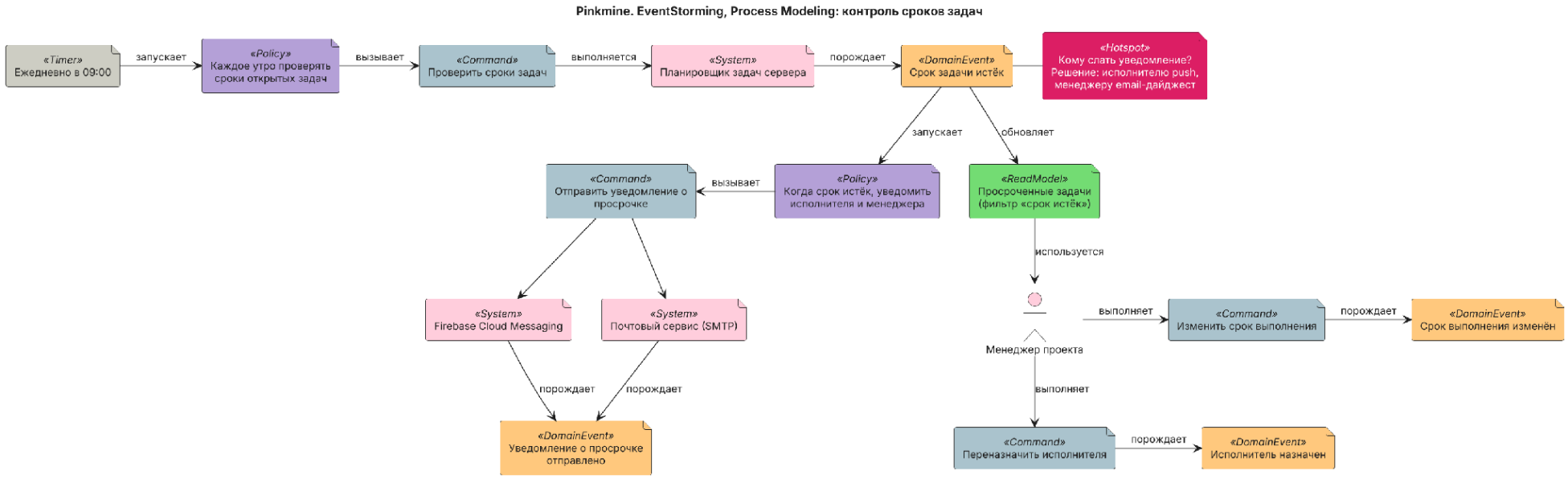


Рисунок 4. Process Modeling: контроль сроков задач

Процесс добавления участника (рисунок 5) связывает два контекста: приглашение создаётся в контексте проектов, а регистрация нового пользователя происходит в контексте идентификации. Решение по противоречию H1 показано на диаграмме.

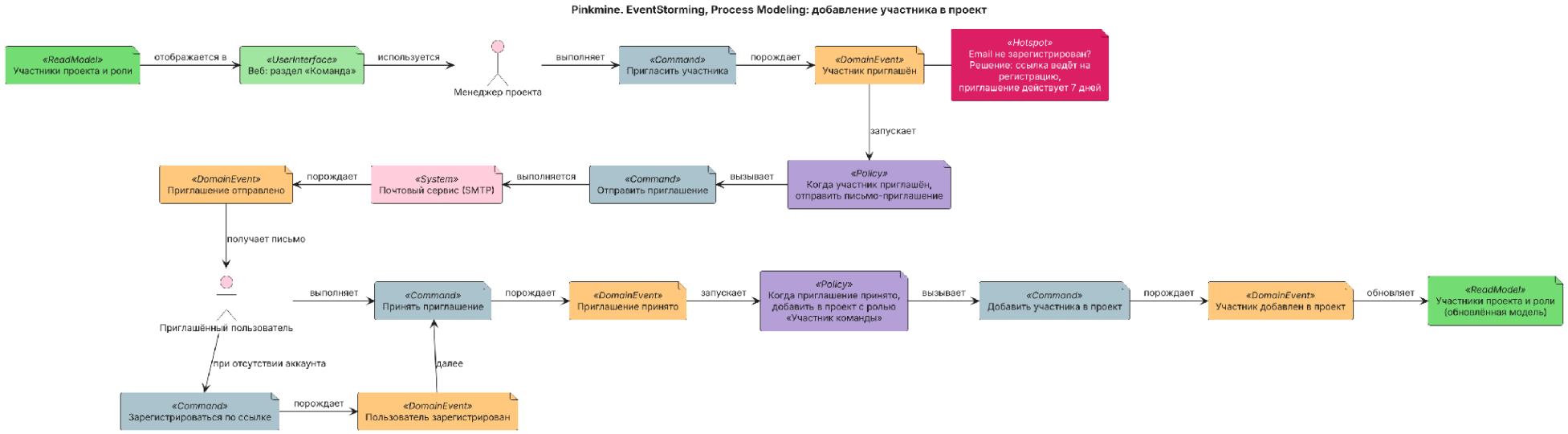


Рисунок 5. Process Modeling: добавление участника в проект

5.4. Уровень Software Design (этапы 6–7)

Этап 6. Агрегаты

Команды и события сгруппированы в пять агрегатов. Комментарий выделен в отдельный агрегат, так как комментарии добавляются часто и параллельно разными участниками, и их изменение не должно блокировать изменение самой задачи. На диаграмме команды расположены слева от агрегата, события справа.

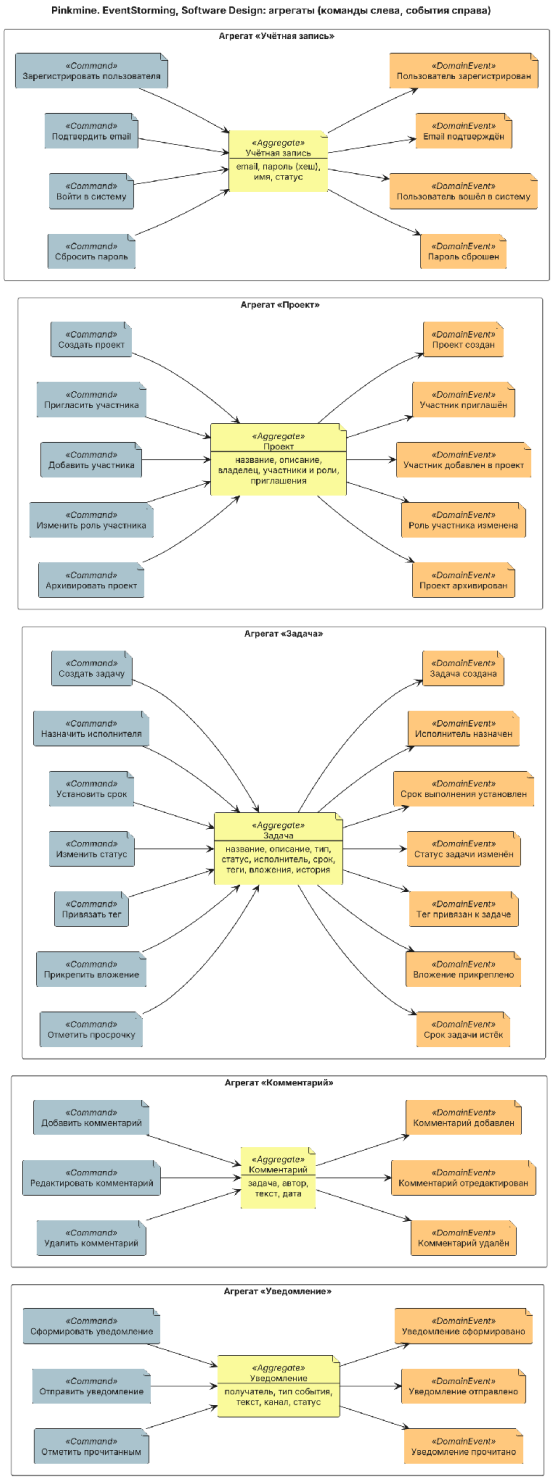


Рисунок 6. Software Design: агрегаты системы Pinkmine

Агрегат	Основные данные	Инварианты (бизнес-правила)
Учётная запись	email, хеш пароля, имя, статус подтверждения	email уникален; вход возможен только после подтверждения email
Проект	название, описание, владелец, участники и роли, приглашения	у проекта ровно один владелец; приглашение действует 7 дней; в архивированный проект нельзя добавлять участников
Задача	название, описание, тип, статус, исполнитель, срок, теги, вложения, история	исполнитель является участником проекта; срок не раньше даты создания; переходы статусов только по таблице ниже; закрывает задачу менеджер
Комментарий	задача, автор, текст, дата	редактировать и удалять комментарий может только автор или администратор
Уведомление	получатель, тип события, текст, канал, статус	одно уведомление на одно событие для одного получателя; прочитанное уведомление не отправляется повторно

Допустимые переходы статусов задачи, проверяемые агрегатом «Задача»:

Из статуса	В статус	Команда	Кто выполняет
Новая	В работе	Взять задачу в работу	исполнитель
В работе	На проверке	Отправить на проверку	исполнитель
На проверке	Готова	Принять задачу	менеджер проекта
На проверке	В работе	Вернуть на доработку	менеджер проекта
Готова	Закрыта	Закрыть задачу	менеджер проекта
Новая, В работе	Отложена	Отложить задачу	менеджер проекта
Отложена	В работе	Возобновить задачу	менеджер проекта

Этап 7. Ограниченные контексты

Агрегаты, связанные общим функционалом и правилами, объединены в четыре ограниченных контекста. Взаимодействие между контекстами выполняется только через интеграционные события и правила, что позволяет реализовать каждый контекст отдельным модулем (Blueprint) Flask-приложения и при необходимости вынести контекст уведомлений в отдельный сервис.

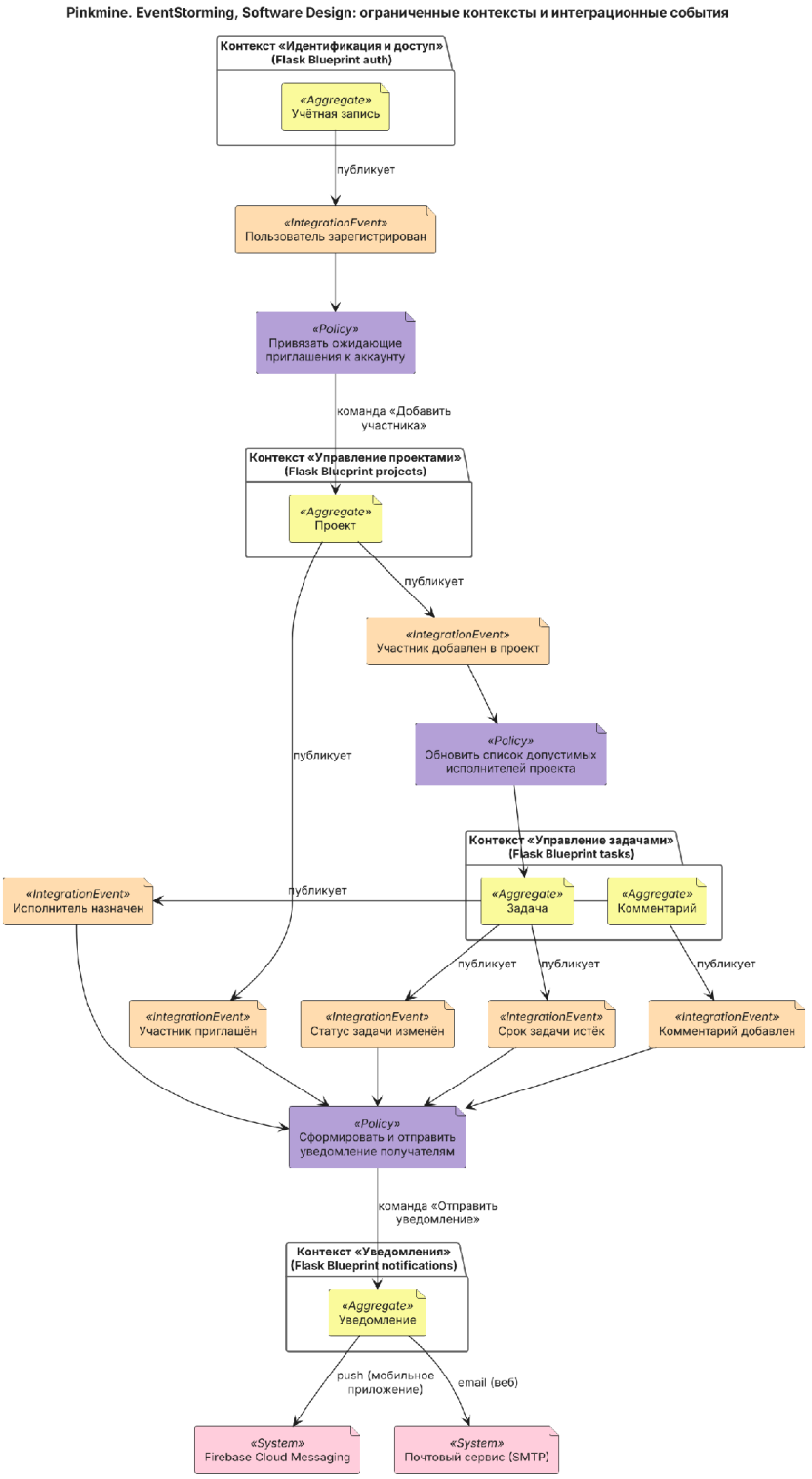


Рисунок 7. Software Design: ограниченные контексты и интеграционные события

Контекст	Агрегаты	Реализация	Публикует	Подписан на
Идентификация и доступ	Учётная запись	Blueprint auth; экраны входа и регистрации (веб, Flutter)	Пользователь зарегистрирован	-
Управление проектами	Проект	Blueprint projects; разделы «Проекты», «Команда»	Участник приглашён,	Пользователь зарегистрирован

Контекст	Агрегаты	Реализация	Публикует	Подписан на
			Участник добавлен в проект	
Управление задачами	Задача, Комментарий	Blueprint tasks; список, Канбан-доска, карточка задачи	Исполнитель назначен, Статус задачи изменён, Срок задачи истёк, Комментарий добавлен	Участник добавлен в проект
Уведомления	Уведомление	Blueprint notifications; FCM, SMTP, колокольчик в header	Уведомление отправлено	все интеграционные события задач и проектов

5.5. Публикация

Исходные файлы спецификации (es-common.iuml, es1_big_picture.puml, es2a–es2d, es3a, es3b) и отрисованные изображения размещаются в вики репозитория проекта на странице Event-Storming и на GitHub Pages.

Задание 6. Проектирование системы с применением метода Event Modeling

6.1. Инструменты и нотация

Модель построена на языке PlantUML с использованием рекомендуемой библиотеки «Event Modeling library for PlantUML» (файл event-modeling-lib.iuml, репозиторий chilit-nl/plantuml-event-modeling). Библиотека автоматически раскладывает элементы по дорожкам и связывает их в порядке объявления, поэтому модель читается слева направо как временная шкала системы.

Элемент	Цвет	Значение
Триггер (wireframe)	белый	экран веб-сайта или мобильного приложения, с которого пользователь инициирует действие
Команда (command)	голубой	намерение изменить состояние системы, соответствует REST-запросу к Pinkmine API
Событие (event)	оранжевый	бизнес-факт, сохранённый в базе данных
Представление (view)	зелёный	запрос, формирующий данные для экрана или автоматизации

Элемент	Цвет	Значение
Автоматизация / передача (extra)	серый	автоматический триггер: планировщик или обработчик уведомлений
Контекст (frame) и понятие (concept)	рамка, жёлтый	ограниченный контекст из EventStorming и главная сущность дорожки событий

Дорожки модели: две дорожки триггеров, «Менеджер проекта (веб)» и «Участник команды (моб.)», что отражает платформенное разделение ролей из персонажей ЛР2; дорожки событий соответствуют ограниченным контекстам из задания 5, «Идентификация и доступ», «Управление проектами», «Управление задачами», «Уведомления».

6.2. Триггеры (макеты низкой точности)

Триггеры описаны макетами низкой точности, выполненными на языке PlantUML Salt. На рисунке 8 показана веб-форма создания задачи, на рисунке 9 мобильный экран «Мои задачи», с которого участник команды меняет статус задачи в одно действие (требование персонажа Райана).

Триггер «Создать задачу»: веб-форма «Новая задача…»

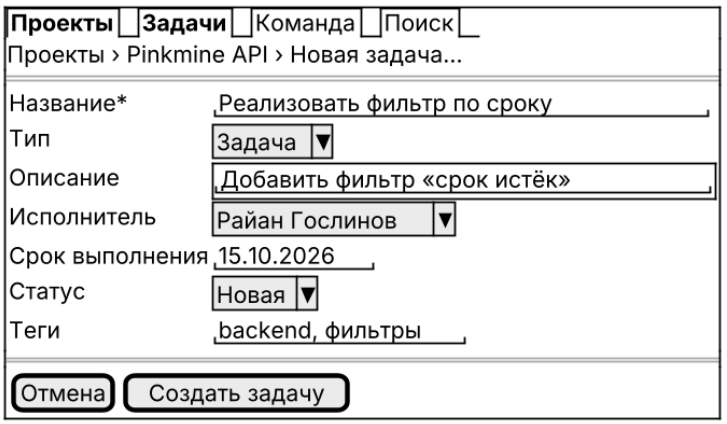


Рисунок 8. Триггер «Создать задачу»: веб-форма «Новая задача…»

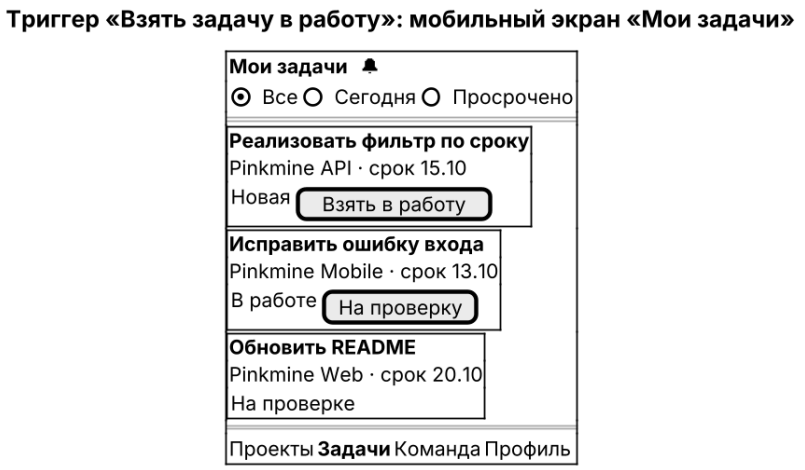


Рисунок 9. Триггер «Взять задачу в работу»: мобильный экран «Мои задачи»

6.3. Модель событий

Модель разделена на три части по ходу жизни системы. Часть 1 описывает подготовку рабочего пространства: регистрацию, создание проекта и формирование команды, включая автоматическую рассылку приглашений.

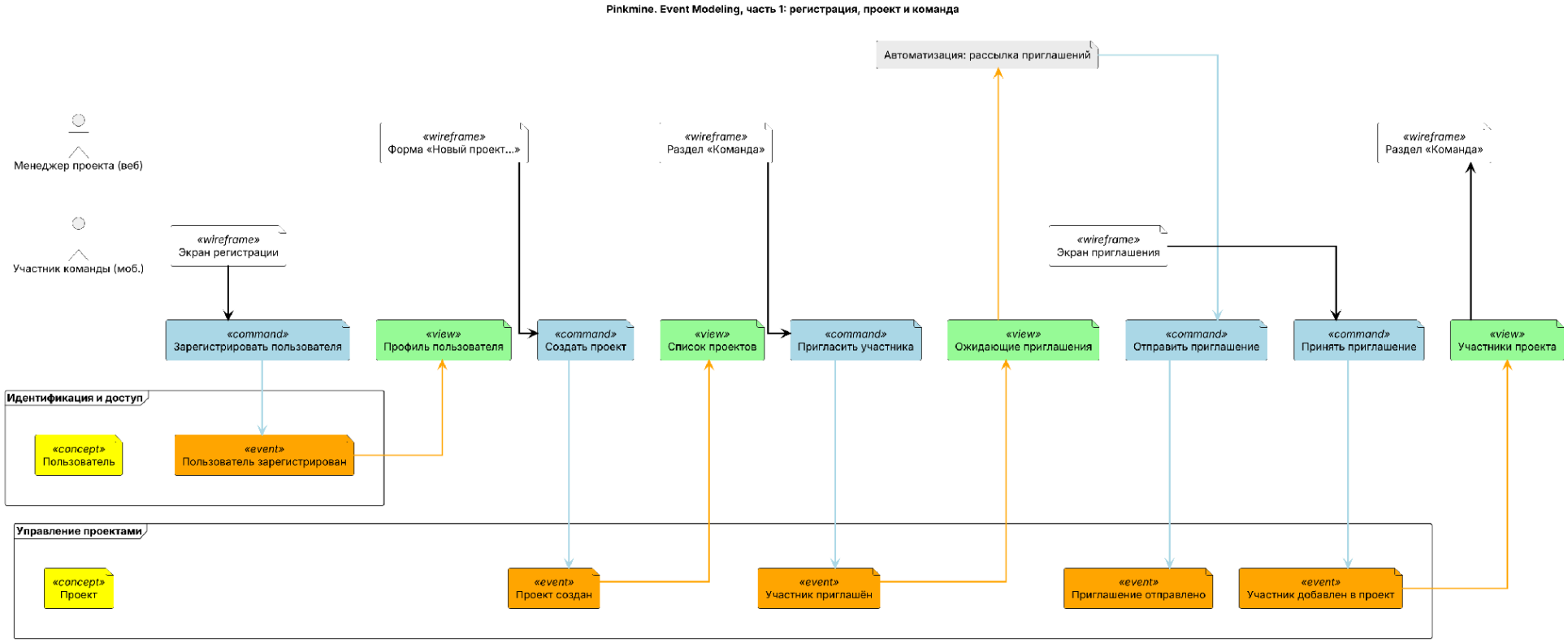


Рисунок 10. Event Modeling, часть 1: регистрация, проект и команда

Часть 2 описывает основной жизненный цикл задачи от создания до закрытия. Назначение исполнителя порождает два потока: представление «Мои задачи» для мобильного приложения и представление «Уведомления к отправке», которое читает автоматизация уведомлений и передаёт данные в контекст уведомлений (шаблон передачи данных).

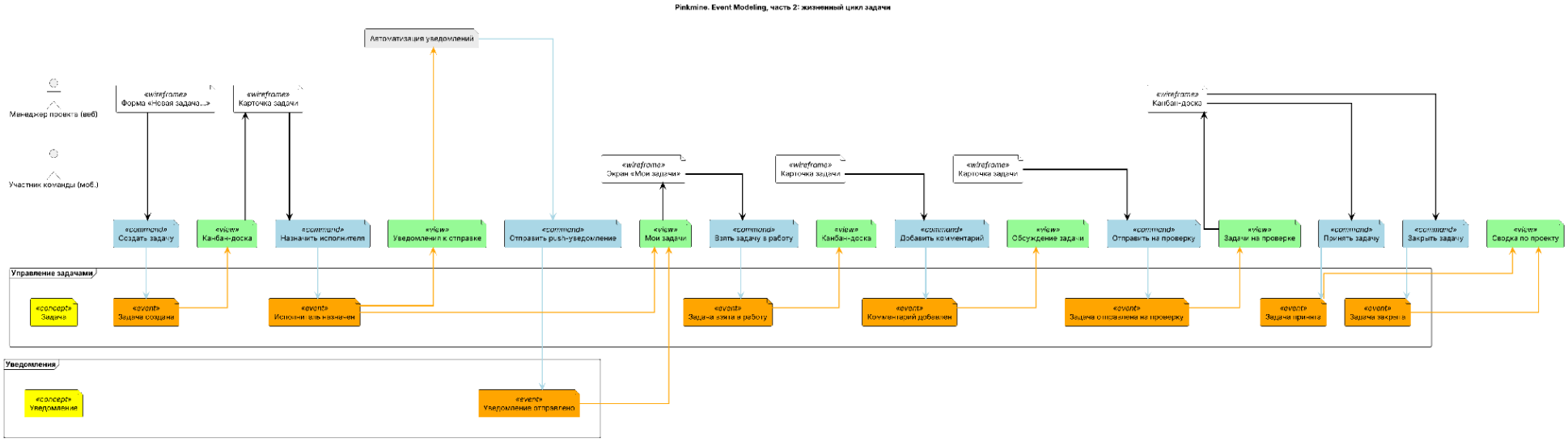


Рисунок 11. Event Modeling, часть 2: жизненный цикл задачи

Часть 3 описывает контроль сроков: планировщик ежедневно читает представление «Открытые задачи со сроком» и фиксирует просрочку (шаблон автоматизации), после чего менеджер переносит срок или возвращает задачу на доработку.

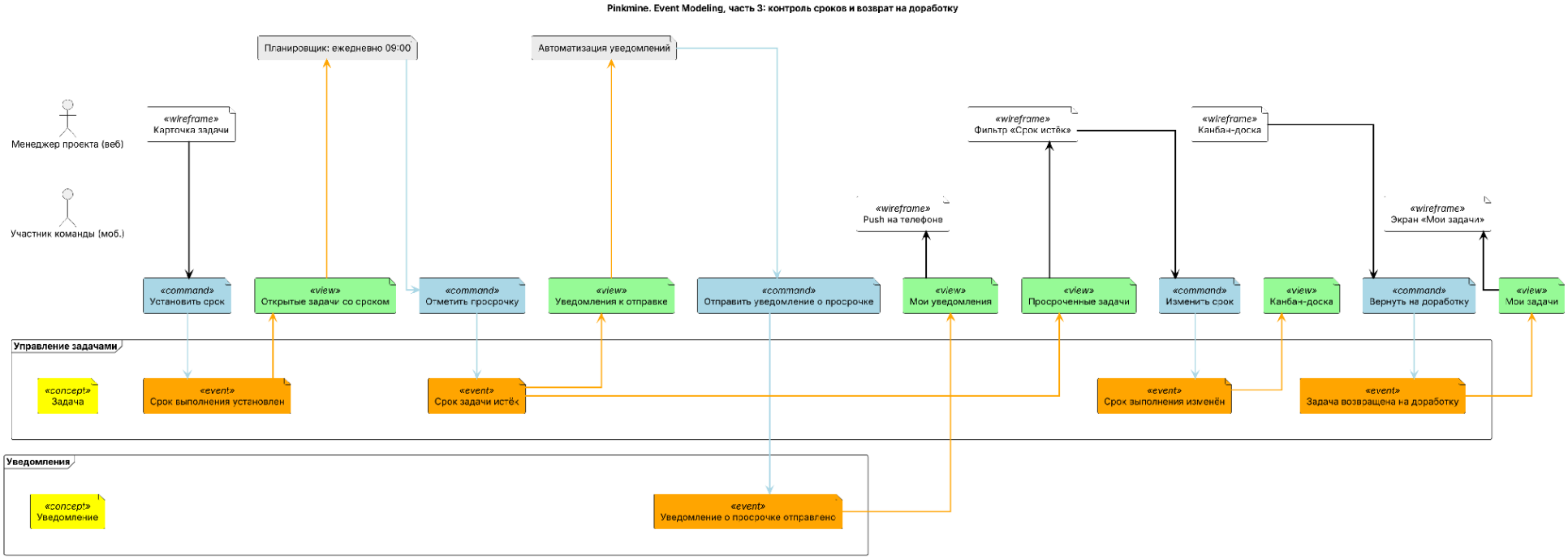


Рисунок 12. Event Modeling, часть 3: контроль сроков и возврат на доработку

6.4. Срезы модели

Каждый шаблон модели рассматривается как срез, минимальная единица работы для разработчика, описывающая путь от экрана до сохранения данных. Для командных срезов указан маршрут REST API, через который триггер вызывает команду.

№	Срез	Шаблон	Триггер → Команда → Событие → Представление	Маршрут API
S1	Регистрация	команда, представление	Экран регистрации → Зарегистрировать пользователя → Пользователь зарегистрирован → Профиль пользователя	POST /api/auth/register
S2	Создание проекта	команда, представление	Форма «Новый проект…» → Создать проект → Проект создан → Список проектов	POST /api/projects
S3	Приглашение участника	команда, представление	Раздел «Команда» → Пригласить участника → Участник приглашён → Ожидающие приглашения	POST /api/projects/{id}/invites
S4	Рассылка приглашений	автоматизация	Ожидающие приглашения → Автоматизация рассылки → Отправить приглашение → Приглашение отправлено	внутренний обработчик, SMTP
S5	Принятие приглашения	команда, представление	Экран приглашения → Принять приглашение → Участник добавлен в проект → Участники проекта → Раздел «Команда»	POST /api/invites/{token}/accept
S6	Создание задачи	команда, представление	Форма «Новая задача…» → Создать задачу → Задача создана → Канбан-доска	POST /api/projects/{id}/tasks
S7	Назначение исполнителя	команда, представление	Карточка задачи → Назначить исполнителя → Исполнитель назначен → Мои задачи → Экран «Мои задачи»	PATCH /api/tasks/{id}/assignee
S8	Уведомление исполнителя	передача данных	Исполнитель назначен → Уведомления к отправке → Автоматизация уведомлений → Отправить push-уведомление → Уведомление отправлено	внутренний обработчик, FCM
S9	Начало работы	команда, представление	Экран «Мои задачи» → Взять задачу в работу → Задача взята в работу → Канбан-доска	PATCH /api/tasks/{id}/status
S10	Комментарий	команда, представление	Карточка задачи → Добавить комментарий → Комментарий добавлен → Обсуждение задачи	POST /api/tasks/{id}/comments

№	Срез	Шаблон	Триггер → Команда → Событие → Представление	Маршрут API
S11	Отправка на проверку	команда, представление	Карточка задачи → Отправить на проверку → Задача отправлена на проверку → Задачи на проверке → Канбан-доска (менеджер)	PATCH /api/tasks/{id}/status
S12	Приёмка и закрытие	команда, представление	Канбан-доска → Принять задачу / Закрыть задачу → Задача принята / Задача закрыта → Сводка по проекту	PATCH /api/tasks/{id}/status
S13	Установка срока	команда, представление	Карточка задачи → Установить срок → Срок выполнения установлен → Открытые задачи со сроком	PATCH /api/tasks/{id}/deadline
S14	Фиксация просрочки	автоматизация	Открытые задачи со сроком → Планировщик (09:00) → Отметить просрочку → Срок задачи истёк	фоновая задача сервера
S15	Уведомление о просрочке	передача данных	Срок задачи истёк → Уведомления к отправке → Автоматизация → Отправить уведомление о просрочке → Уведомление отправлено → Мои уведомления → Push на телефоне	внутренний обработчик, FCM, SMTP
S16	Перенос срока	команда, представление	Просроченные задачи → Фильтр «Срок истёк» → Изменить срок → Срок выполнения изменён → Канбан-доска	PATCH /api/tasks/{id}/deadline
S17	Возврат на доработку	команда, представление	Канбан-доска → Вернуть на доработку → Задача возвращена на доработку → Мои задачи → Экран «Мои задачи»	PATCH /api/tasks/{id}/status

6.5. Данные событий

Для каждого события определён состав сохраняемых данных. Именно эти данные доступны представлениям, поэтому представление может показать только то, что было записано событиями ранее.

Событие	Контекст	Данные события
Пользователь зарегистрирован	Идентификация	userId, email, имя, дата регистрации
Проект создан	Проекты	projectId, workspaceId, название, описание, ownerId, дата
Участник приглашён	Проекты	inviteId, projectId, email, роль, срок действия, invitedBy
Приглашение отправлено	Проекты	inviteId, канал (email), дата отправки

Событие	Контекст	Данные события
Участник добавлен в проект	Проекты	projectId, userId, роль, дата
Задача создана	Задачи	taskId, projectId, название, описание, тип, статус «Новая», authorId, дата
Исполнитель назначен	Задачи	taskId, assigneeId, предыдущий исполнитель, assignedBy, дата
Срок выполнения установлен / изменён	Задачи	taskId, новый срок, прежний срок, автор изменения
Задача взята в работу	Задачи	taskId, статус «Новая» → «В работе», userId, дата
Комментарий добавлен	Задачи	commentId, taskId, authorId, текст, дата
Задача отправлена на проверку	Задачи	taskId, статус «В работе» → «На проверке», userId
Задача принята	Задачи	taskId, статус «На проверке» → «Готова», reviewerId
Задача возвращена на доработку	Задачи	taskId, статус «На проверке» → «В работе», reviewerId, причина
Задача закрыта	Задачи	taskId, статус «Готова» → «Закрыта», closedBy, дата
Срок задачи истёк	Задачи	taskId, срок, assigneeId, managerId, дата фиксации
Уведомление отправлено	Уведомления	notificationId, recipientId, тип события, taskId или projectId, канал (push/email), дата

6.6. Спецификация срезов в формате Given / When / Then

Модель Event Modeling напрямую переводится в приёмочные сценарии BDD: «Дано» соответствует ранее произошедшим событиям, «Когда» команде, «Тогда» новым событиям или содержимому представления. Для ключевых срезов составлены позитивные и негативные сценарии.

Срез	Дано	Когда	Тогда
S1	пользователя с email ivan@mail.by нет в системе	отправлена форма регистрации с этим email и паролем	событие «Пользователь зарегистрирован», на email отправлено письмо подтверждения
S1 (негатив)	пользователь с email ivan@mail.by уже зарегистрирован	отправлена форма регистрации с тем же email	событие не создаётся, показано сообщение «Пользователь с таким email уже существует»

Срез	Дано	Когда	Тогда
S3	Проект создан; текущий пользователь, менеджер проекта	менеджер приглашает dev@mail.by с ролью «Участник команды»	событие «Участник приглашён», приглашение видно в представлении «Ожидающие приглашения»
S5	Участник приглашён; приглашению меньше 7 дней	пользователь открывает ссылку и принимает приглашение	событие «Участник добавлен в проект», пользователь виден в разделе «Команда»
S5 (негатив)	Участник приглашён более 7 дней назад	пользователь принимает приглашение	событие не создаётся, показано сообщение «Срок действия приглашения истёк»
S6	Проект создан; Участник добавлен в проект	менеджер заполняет форму «Новая задача…» и нажимает «Создать задачу»	событие «Задача создана» со статусом «Новая», задача в колонке «Новая» Канбан-доски
S7	Задача создана; Райан добавлен в проект	менеджер назначает Райана исполнителем	событие «Исполнитель назначен», задача появилась в «Мои задачи» у Райана
S7 (негатив)	Задача создана; пользователь не участник проекта	менеджер пытается назначить его исполнителем	событие не создаётся, пользователь отсутствует в списке выбора исполнителя
S8	Исполнитель назначен; у исполнителя зарегистрировано мобильное устройство	автоматизация читает «Уведомления к отправке»	событие «Уведомление отправлено» с каналом push, уведомление содержит название проекта и задачи
S9	Исполнитель назначен; задача в статусе «Новая»	исполнитель нажимает «Взять в работу» в мобильном приложении	событие «Задача взята в работу», карточка в колонке «В работе»
S11	Задача взята в работу	исполнитель нажимает «На проверку»	событие «Задача отправлена на проверку», менеджер получает уведомление
S12	Задача отправлена на проверку	менеджер принимает задачу, затем закрывает её	события «Задача принята» и «Задача закрыта», задача учтена в сводке по проекту
S12 (негатив)	Задача взята в работу	исполнитель пытается закрыть задачу	событие не создаётся, кнопка «Закрыть» недоступна исполнителю
S14	Срок выполнения установлен на 10.10.2026; задача не	планировщик выполняет ежедневную проверку в 09:00	событие «Срок задачи истёк», задача в

Срез	Дано	Когда	Тогда
	закрыта; сегодня 11.10.2026		представлении «Просроченные задачи»
S14 (негатив)	Срок выполнения установлен на 10.10.2026; Задача закрыта	планировщик выполняет ежедневную проверку	событие не создаётся, закрытые задачи не проверяются
S16	Срок задачи истёк	менеджер устанавливает новый срок 20.10.2026	событие «Срок выполнения изменён», задача исчезает из «Просроченные задачи»
S17	Задача отправлена на проверку	менеджер нажимает «Вернуть на доработку» и указывает причину	событие «Задача возвращена на доработку», задача снова в статусе «В работе» у исполнителя

Пример записи срезов S7 и S14 на языке Gherkin для автоматизации приёмочных тестов:

```
# language: ru
Функция: Назначение исполнителя задачи
  Как менеджер проекта
  Чтобы распределить работу в команде
  Я хочу назначать исполнителя задачи из числа участников проекта

  Сценарий: Назначение участника проекта исполнителем
    Дано в проекте "Pinkmine API" создана задача "Реализовать фильтр по сроку"
    И пользователь "Райан" добавлен в проект "Pinkmine API"
    Когда менеджер назначает "Райан" исполнителем задачи
    Тогда задача отображается в списке "Мои задачи" пользователя "Райан"
    И пользователь "Райан" получает push-уведомление о назначении

  Сценарий: Попытка назначить исполнителя не из проекта
    Дано пользователь "Виктор" не состоит в проекте "Pinkmine API"
    Когда менеджер открывает список выбора исполнителя
    Тогда пользователь "Виктор" отсутствует в списке

Функция: Контроль сроков задач
  Сценарий: Фиксация просроченной задачи
    Дано у задачи "Обновить README" срок "10.10.2026"
    И задача находится в статусе "В работе"
    Когда планировщик выполняет проверку сроков "11.10.2026" в 09:00
    Тогда задача отображается в списке "Просроченные задачи"
    И исполнитель получает push-уведомление о просрочке
```

6.7. Связь с моделью EventStorming

Event Modeling использует события и агрегаты, найденные в задании 5, и дополняет их тем, чего нет в EventStorming: конкретными экранами-триггерами, представлениями с перечнем данных и границами срезов для планирования разработки. Правила EventStorming («Когда исполнитель назначен, уведомить исполнителя», «Каждое утро проверять сроки») перешли в шаблоны автоматизации и передачи данных, а модели чтения стали представлениями. Противоречия H1–H5 закреплены в негативных сценариях Given / When / Then.

6.8. Публикация

Исходные файлы (em1_projects_team.puml, em2_task_lifecycle.puml, em3_deadlines.puml, wf1_new_task_web.puml, wf2_my_tasks_mobile.puml) и изображения размещаются в вики репозитория проекта на странице Event-Modeling и на GitHub Pages.

Задание 7. Описание архитектуры системы с использованием подхода C4 Model

Описание архитектуры сделано в wiki проекта, C4-Model - https://github.com/fpmi-hci-2026/hci-10-project-group-3/wiki/C4-Model.

Задание 8. Разработать техническое задание на разработку приложения

Спецификация требований к программному обеспечению

«Pinkmine - система управления проектами»

Документ составлен по шаблону SRS К. Вигерса и Д. Битти. Он объединяет и структурирует результаты заданий 1–7 лабораторной работы №3: пользовательские истории, UML-диаграммы, Event Storming, Event Modeling и описание архитектуры по модели C4.

1. Введение

1.1. Назначение

Документ описывает функциональные и нефункциональные требования к системе управления проектами Pinkmine версии 1.0. В состав системы входят веб-клиент, мобильное приложение для Android и сервер с REST API. Документ предназначен для разработчиков, тестировщиков и участников команды проекта и служит основой для реализации и приёмки системы.

1.2. Соглашения, принятые в документах

Таблица 1 - Обозначения

Обозначение	Значение

US-N	Пользовательская история (User Story) с номером N
UC-TN	Вариант использования подсистемы «Работа с задачами»
FR-NN	Функциональное требование
H1–H5	Противоречия, выявленные методом Event Storming, и принятые по ним решения
S1–S17	Срезы модели Event Modeling
[Web], [Android]	Требование относится только к веб-клиенту или только к Android-приложению
High / Medium / Low	Приоритет требования: обязательное в первой версии / важное / желательное

Требования сформулированы в виде «Система должна…». Если платформа не указана, требование относится к обоим клиентам.

1.3. Границы проекта

Pinkmine предназначен для небольших команд и решает задачу прозрачности командной работы: менеджер проекта ставит задачи и видит их актуальное состояние без совещаний и переписки в чатах, а участники команды получают задачи и отмечают прогресс в 1–2 действия.

В границы проекта входят:

- регистрация, вход, подтверждение email и восстановление пароля;
- проекты, рабочие пространства, состав команды, приглашения по email, архивирование проектов;
- задачи: создание, назначение исполнителя, типы, сроки, статусы, теги, комментарии, вложения, история изменений;
- представление задач списком и Канбан-доской, поиск и фильтрация;
- проверка выполненных задач и возврат на доработку;
- уведомления: push на Android, уведомления в веб-клиенте, письма;
- контроль просроченных задач;
- глобальные права ролей;
- офлайн-режим смены статуса в Android-приложении.

В границы проекта не входят:

- мобильное приложение для iOS;

- сервис доставки push-уведомлений (используется Firebase Cloud Messaging) и почтовый сервис (используется внешний SMTP-сервис);
- хранилище файлов вложений: система хранит только метаданные файлов.

1.4. Ссылки

Таблица 2 - Источники требований

Документ	**Содержание**
Вики проекта: «Пользовательские истории»	US-1…US-14, критерии приёмки, приоритеты, зависимости, платформенные особенности (задания 1, 4)
Вики проекта: «Функциональные требования»	Диаграммы вариантов использования и сценарии UC-T1…UC-T5 (задание 4)
Вики проекта: «Спецификация»	UML-диаграммы на PlantUML и Mermaid (задания 2–4)
Вики проекта: «Схема базы данных»	ERD, ORM-отображение, SQL DDL (задание 4)
Вики проекта: «Event Storming», «Event Modeling»	События, агрегаты, ограниченные контексты, противоречия H1–H5, срезы S1–S17 (задания 5, 6)
Вики проекта: «Архитектура C4»	Контекст, контейнеры, компоненты, развёртывание (задание 7)
Отчёт по ЛР2	Персонажи, сценарии, таблица ролей и задач, объектная модель
Вигерс К., Битти Д. Разработка требований к программному обеспечению	Шаблон SRS

2. Общее описание

2.1. Общий взгляд на продукт

Pinkmine - новый продукт. Система состоит из веб-клиента, Android-приложения на Flutter и единого сервера с REST API на Python (Flask). Данные хранятся в PostgreSQL. Push-уведомления доставляются через Firebase Cloud Messaging, письма - через внешний почтовый сервис. Окружение системы показано на рисунке 13.

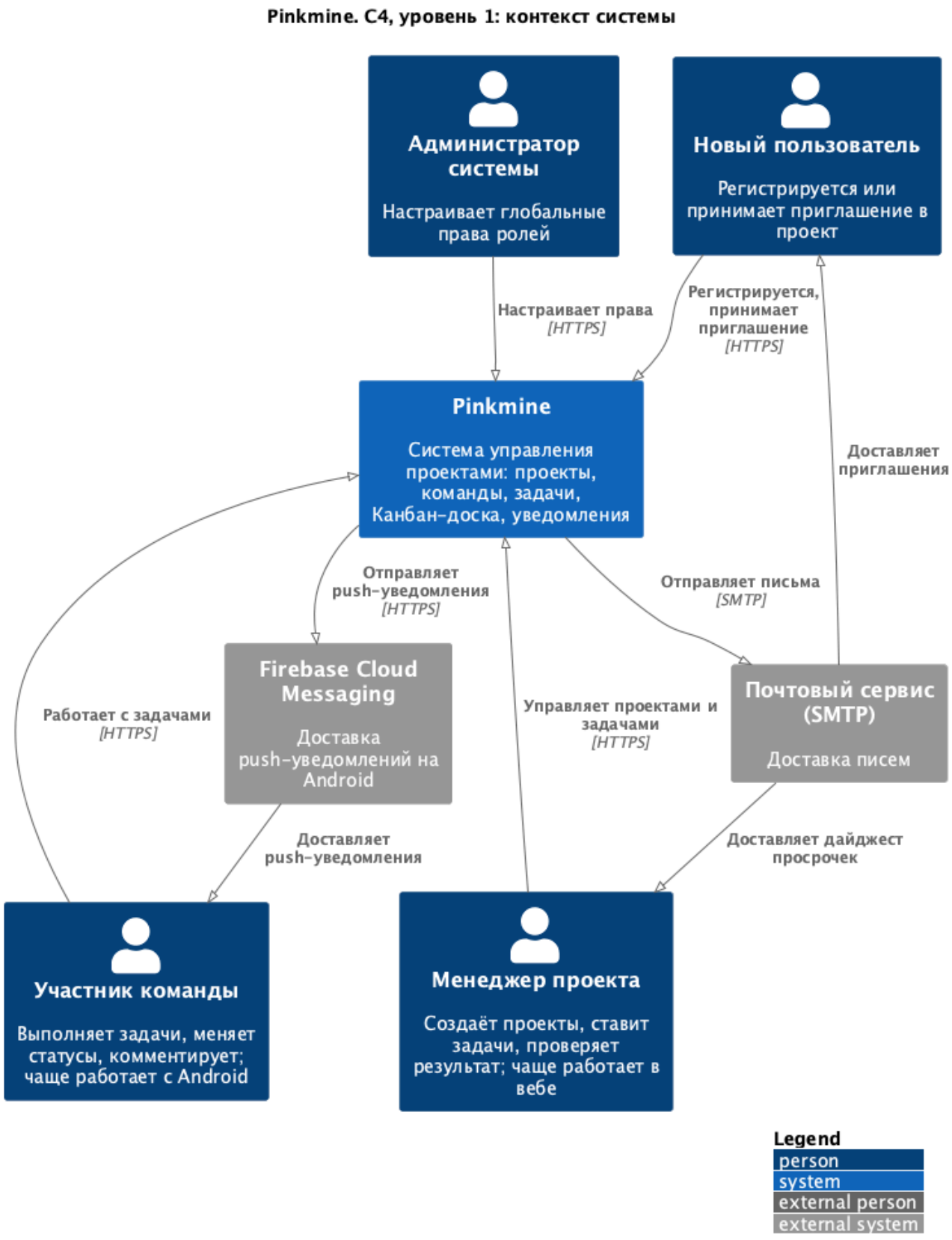


Рисунок 13 - Контекст системы (C4, уровень 1)

Внутри системы выделены четыре ограниченных контекста, определённые методом Event Storming:

- «Идентификация и доступ» - учётные записи, вход, права;
- «Управление проектами» - проекты, команда, приглашения;
- «Управление задачами» - задачи, комментарии, история;
- «Уведомления» - формирование и доставка уведомлений.

Контексты взаимодействуют через доменные события. Состав контейнеров показан на рисунке 14.

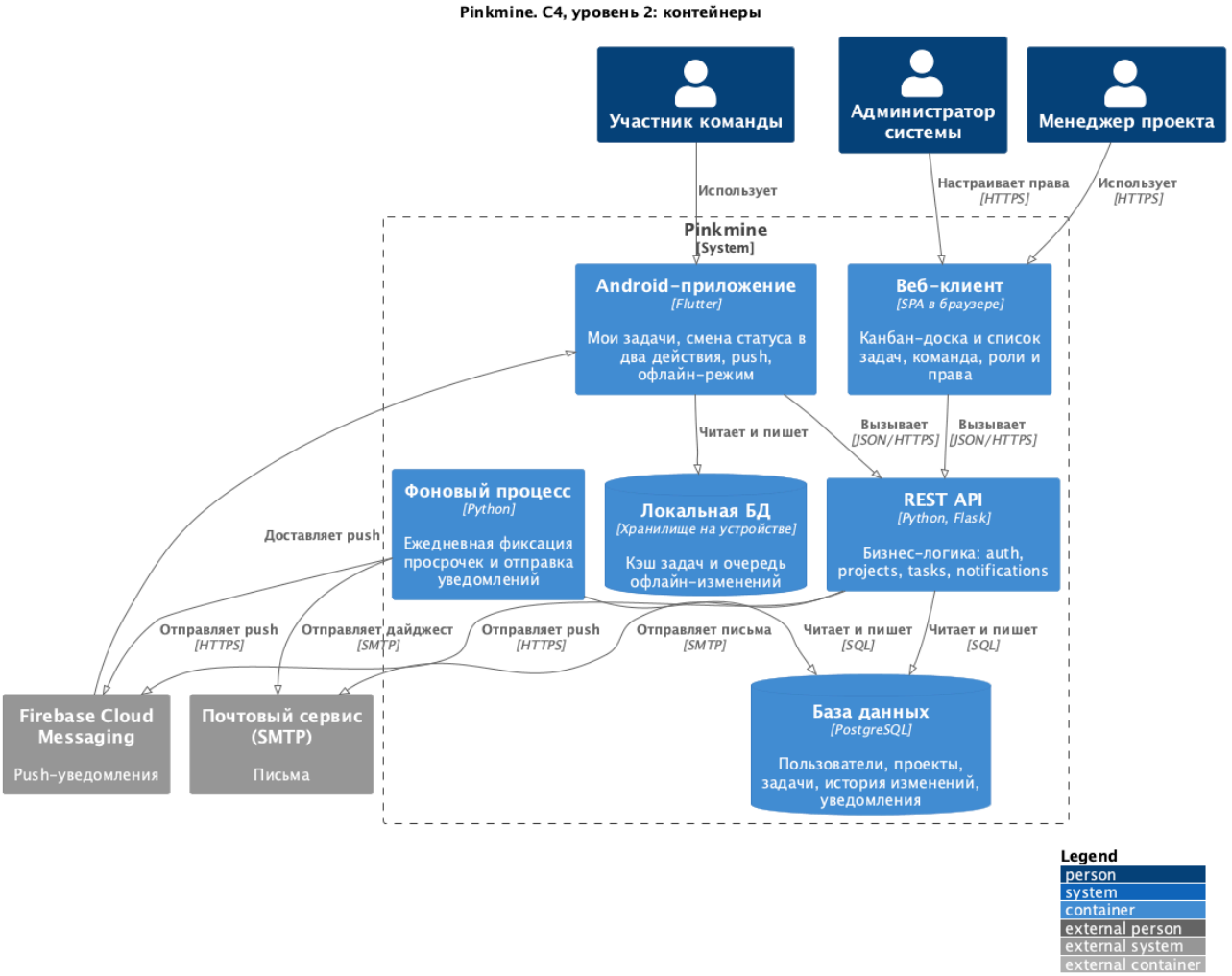


Рисунок 14 - Контейнеры системы (C4, уровень 2)

2.2. Классы и характеристики пользователей

Таблица 3 - Классы пользователей

Класс	Характеристика	Основная платформа	Ключевые функции

Новый пользователь	Ещё не зарегистрирован или получил приглашение по email	Веб, Android	Регистрация, принятие приглашения
Участник команды (исполнитель)	Выполняет задачи, часто работает вне офиса (персонаж ЛР2 - Райан)	Android	Мои задачи, смена статуса, комментарии, вложения, уведомления, офлайн-режим
Менеджер проекта (тимлид)	Создаёт проекты, ставит задачи, проверяет результат, готовит отчёты для заказчика (персонаж ЛР2 - Джесика). Наследует права участника	Веб	Проекты, команда, приглашения, постановка и проверка задач, фильтры, контроль сроков, архив
Администратор системы	Немногочисленная роль, наследует права менеджера	Веб	Настройка глобальных прав ролей, физическое удаление проектов

2.3. Операционная среда

- **Веб-клиент:** современный браузер на ПК (обязательная поддержка Chrome). Макет рассчитан на ширину 1440 px, планшетный вариант - 980 px, сетка 12 колонок.
- **Android-приложение:** смартфон на Android. Макет рассчитан на экран 375×670 px, сетка 8 pt. Приложение хранит локальный кэш задач и очередь офлайн-изменений.
- **Сервер приложений:** веб-сервер раздаёт статику веб-клиента и передаёт запросы `/api` во Flask-приложение. Ежедневная проверка сроков выполняется отдельным фоновым процессом.
- **Сервер БД:** PostgreSQL.
- **Внешние сервисы:** Firebase Cloud Messaging, почтовый сервис (SMTP).

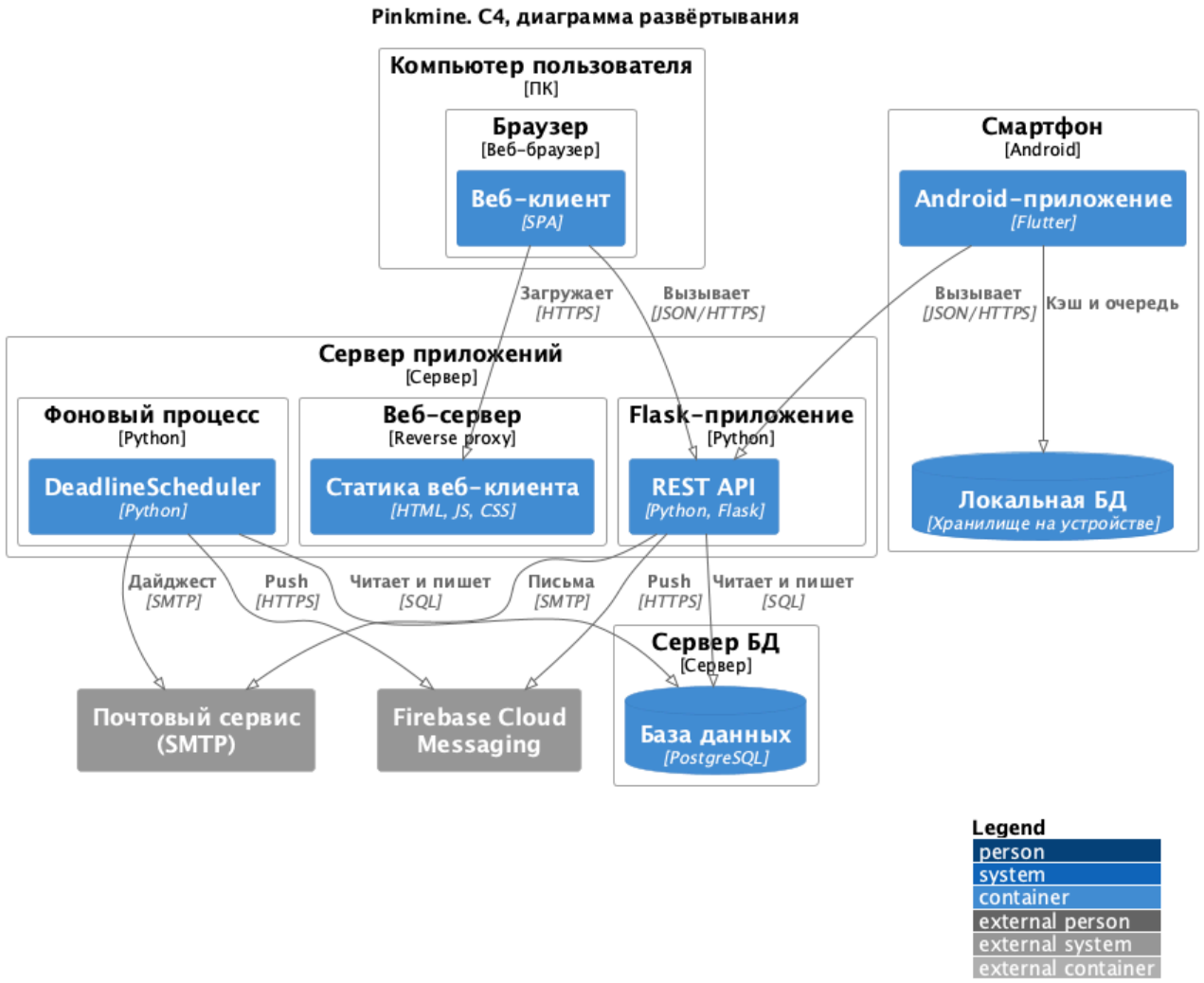


Рисунок 15 - Развёртывание системы (C4)

2.4. Ограничения дизайна и реализации

- Сервер реализуется на Python с фреймворком Flask, модули API - Flask Blueprint по ограниченным контекстам: `auth`, `projects`, `tasks`, `notifications`.
- Мобильное приложение реализуется на Flutter, только для Android.
- СУБД - PostgreSQL, все таблицы в схеме `pinkmine`.
- Клиенты взаимодействуют с сервером только через REST API (JSON поверх HTTPS) с аутентификацией по Bearer-токену.
- Ошибки API возвращаются в едином формате `{ errorCode, message, details[], traceId }`.
- Раздел «Роли и права» реализуется только в веб-клиенте.
- Статус задачи отображается иконкой и текстом, а не только цветом.

2.5. Предположения и зависимости

Предположения:

- Система рассчитана на небольшие команды: сотни пользователей и тысячи задач на проект.
- Наступление срока определяется по календарной дате в едином часовом поясе системы.
- Роль пользователя действует на двух уровнях: глобальная роль определяет системные права, роль в проекте (менеджер, участник) - права внутри проекта.
- Пользователи физически не удаляются.

Зависимости от внешних систем: доставка push-уведомлений зависит от Firebase Cloud Messaging, доставка писем - от почтового сервиса. При их недоступности уведомление сохраняется в системе и отправляется повторно.

Порядок реализации (зависимости между историями):

- US-1 - перед всеми остальными;
- US-2 и US-3 - перед US-4, US-5, US-6;
- US-4 - перед US-8 и US-12;
- US-11 - перед US-8;
- US-1 и US-3 - перед US-9;
- US-2 и US-6 - перед US-10;
- US-1 - перед US-13.

3. Функции системы

Функции сгруппированы по ограниченным контекстам. Варианты использования показаны на рисунке 16, детальные сценарии подсистемы «Работа с задачами» приведены в Приложении Б.

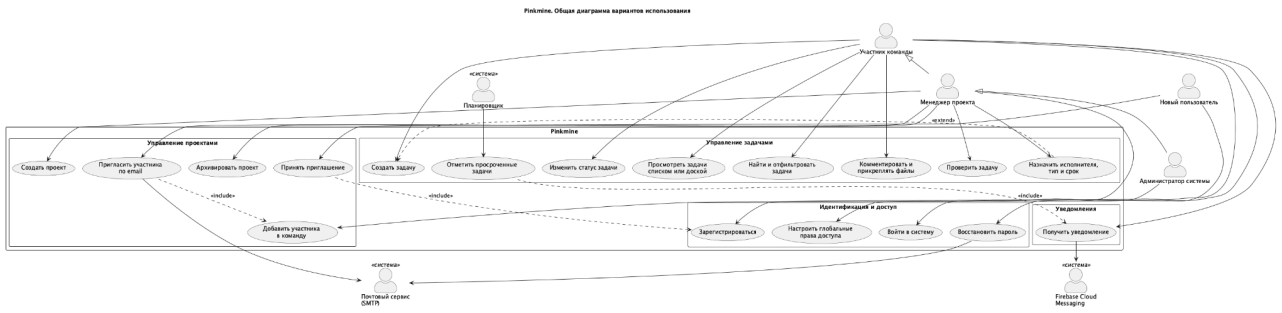


Рисунок 16 - Общая диаграмма вариантов использования

3.1. Идентификация и доступ

3.1.1. Описание

Регистрация и вход пользователей, подтверждение email, восстановление пароля, настройка глобальных прав ролей администратором. Приоритет: High (US-1), Medium (US-7, US-13). Без регистрации и входа недоступна ни одна другая функция.

3.1.2. Функциональные требования

Таблица 4 - Требования контекста «Идентификация и доступ»

ID	Требование	Источник	Приоритет
FR-01	Система должна регистрировать пользователя по логину, имени, фамилии, email и паролю; при незаполненном поле - показывать ошибку у этого поля и не создавать учётную запись	US-1	High
FR-02	Система должна отклонять регистрацию с email, который уже используется, с сообщением об этом	US-1	High
FR-03	Система должна требовать пароль не короче 8 символов и хранить его только в виде хеша	US-1, US-13	High
FR-04	Система должна отправлять после регистрации письмо со ссылкой подтверждения email; до подтверждения вход разрешён, а в профиле показывается баннер «Подтвердите email»	US-1	High
FR-05	Система должна выполнять вход по логину и паролю, при неверных данных - показывать сообщение, после входа - открывать список проектов; выход через меню профиля возвращает на экран входа	US-1	High
FR-06	Система должна по запросу «Забыли пароль?» отправлять письмо со ссылкой сброса и показывать одинаковое сообщение независимо от того, зарегистрирован ли адрес	US-13	Medium
FR-	Система должна делать ссылку сброса пароля одноразовой и ограниченной по времени и после смены пароля завершать все активные сессии пользователя	US-13	Medium

07			
FR-08	[Web] Система должна предоставлять администратору раздел «Роли и права» со списком ролей и их прав и сохранять изменения прав	US-7	Medium
FR-09	Система должна применять изменённые права при следующем действии пользователя и не показывать недоступные действия	US-7	Medium
FR-10	Система должна сообщать менеджеру и участнику при попытке открыть раздел «Роли и права», что действие недоступно	US-7	Medium

3.2. Управление проектами и командой

3.2.1. Описание

Создание проектов, формирование команды, приглашение по email, архивирование. Приоритет: High (US-3), Medium (US-9), Low (US-14). Без состава команды нельзя назначать исполнителей.

3.2.2. Функциональные требования

Таблица 5 - Требования контекста «Управление проектами»

ID	**Требование**	**Источник**	**Приоритет**
FR-11	Система должна позволять менеджеру создавать проект с названием и описанием в рабочем пространстве	S2	High
FR-12	Система должна позволять менеджеру добавлять в команду проекта зарегистрированного пользователя с ролью, находя его по имени или email	US-3	High

FR-13	Система должна запрещать повторное добавление пользователя, уже состоящего в проекте, с сообщением об этом	US-3	High
FR-14	Система должна показывать список команды с ролью и контактными данными каждого участника и не показывать участнику кнопку «Добавить участника…»	US-3	High
FR-15	Система должна позволять пригласить в проект человека по email, которого нет в системе: отправлять письмо со ссылкой, действующей 7 дней	US-9, H1	Medium
FR-16	Система должна показывать ожидающие приглашения с датой отправки и позволять отозвать приглашение; повторное приглашение на тот же email заменяет предыдущее и продлевает срок	US-9	Medium
FR-17	Система должна по ссылке приглашения открывать регистрацию с заполненным email и после регистрации делать пользователя участником проекта с указанной ролью; по истёкшей или отозванной ссылке - показывать «Приглашение недействительно, обратитесь к менеджеру проекта»	US-9	Medium
FR-18	Система должна позволять менеджеру архивировать проект с подтверждением и возвращать его из архива; архивный проект скрыт из списка и доступен через фильтр «Архив»	US-14, H4	Low
FR-19	Система должна запрещать создание и изменение задач в архивном проекте, оставляя их доступными для просмотра	US-14	Low
FR-	Система должна разрешать физическое удаление проекта только администратору системы	US-14, H4	Low

20			

3.3. Управление задачами

3.3.1. Описание

Постановка задач, смена статусов, просмотр и фильтрация, проверка и возврат на доработку, комментарии и вложения, офлайн-режим. Приоритет: High (US-2, US-4, US-5, US-8), Medium (US-11), Low (US-12). Постановка задач и смена статуса - ядро продукта и самые частые действия.



Рисунок 17 - Варианты использования подсистемы «Работа с задачами»

3.3.2. Функциональные требования

Таблица 6 - Требования контекста «Управление задачами»

ID	**Требование**	**Источник**	**Приоритет**
FR-21	Система должна создавать задачу с названием, описанием, типом, сроком и исполнителем; без названия задача не создаётся	US-2, UC-T1	High

FR-22	Система должна предлагать тип из пяти значений (задача, ошибка, улучшение, идея, встреча) и исполнителя только из участников проекта	US-2, UC-T2	High
FR-23	Система должна отклонять срок выполнения раньше текущей даты с сообщением «Срок не может быть в прошлом»	US-2, UC-T5	High
FR-24	Система должна присваивать новой задаче статус «Новая», показывать её в первой колонке доски и в разделе «Мои задачи» исполнителя	US-2	High
FR-25	Система должна запрещать участнику команды изменять исполнителя, тип и срок задачи	US-2	High
FR-26	Система должна поддерживать шесть статусов: новая, в работе, на проверке, отложена, готова, закрыта	US-4	High
FR-27	[Android] Система должна позволять сменить статус в два действия: касание текущего статуса и выбор нового в нижней панели	US-4	High
FR-28	[Web] Система должна позволять сменить статус выбором в карточке или перетаскиванием карточки между колонками доски	US-4	High
FR-	Система должна разрешать статус «Закрыта» только менеджеру проекта	US-4, H2	High

29			
FR-30	Система должна записывать каждое изменение задачи в историю: автор, время, старое и новое значение	US-4, US-8	High
FR-31	Система должна отклонять изменение задачи, основанное на устаревшей версии, и показывать пользователю актуальное состояние задачи с сообщением о конфликте	UC-T3, H3	High
FR-32	Система должна показывать задачи проекта вкладками «Доска» и «Список» без смены выбранного проекта	US-5	High
FR-33	Система должна фильтровать задачи одновременно по статусу, типу, исполнителю, сроку и тегу и предоставлять быстрые фильтры «Мои задачи», «Просроченные», «На проверке»	US-5	High
FR-34	Система должна сортировать список по сроку и дате обновления; при пустом результате - показывать сообщение и кнопку «Сбросить фильтры»	US-5	High
FR-35	[Web] Система должна отражать фильтр по статусу в адресе страницы	US-5	High
FR-36	Система должна при переводе задачи в «На проверке» уведомлять менеджера и показывать ему кнопки «Принять» и «Вернуть на доработку»	US-8, UC-T4	High

FR-37	Система должна по кнопке «Принять» переводить задачу в «Готова» и после этого позволять менеджеру закрыть её	US-8, UC-T4	High
FR-38	Система должна требовать комментарий с причиной для возврата на доработку, переводить задачу в «В работе» и уведомлять исполнителя с текстом комментария	US-8, UC-T4	High
FR-39	Система должна показывать в карточке блок «Обсуждение» с комментариями в хронологическом порядке; пустой комментарий не сохраняется	US-11	Medium
FR-40	Система должна позволять автору редактировать и удалять свой комментарий и помечать отредактированный комментарий «изменено»	US-11	Medium
FR-41	Система должна позволять прикрепить файл к задаче и показывать вложение с именем, размером и автором	US-11	Medium
FR-42	[Android] Система должна при отсутствии сети показывать баннер «Нет соединения» и задачи из локального кэша	US-12	Low
FR-43	[Android] Система должна сохранять смену статуса без сети в очереди, помечать карточку «ожидает синхронизации» и автоматически отправлять очередь при появлении связи; очередь сохраняется при закрытии приложения	US-12, H3	Low
FR-	[Android] Система должна делать недоступными в офлайне создание задач и приглашение участников, показывая подсказку	US-12	Low

44			

3.4. Уведомления и контроль сроков

3.4.1. Описание

Уведомления о назначении, смене статуса, комментариях и просрочках; ежедневный контроль сроков. Приоритет: Medium (US-6, US-10). Работать можно и без уведомлений, но они снижают риск пропустить задачу или срок.

3.4.2. Функциональные требования

Таблица 7 - Требования контекста «Уведомления»

ID	Требование	Источник	Приоритет
FR-45	Система должна при назначении исполнителя создавать уведомление только для него с названием проекта и задачи	US-6, UC-T2	Medium
FR-46	[Android] Система должна доставлять push-уведомление, касание которого открывает карточку задачи	US-6	Medium
FR-47	[Web] Система должна показывать по иконке колокольчика список последних событий с переходом к задаче	US-6	Medium
FR-48	Система должна помечать прочитанные уведомления и визуально выделять непрочитанные	US-6	Medium
FR-49	[Android] Система должна при запрете push-уведомлений в настройках телефона показывать подсказку с переходом в настройки	US-6	Medium

FR-50	Система должна уведомлять исполнителя и автора задачи о новом комментарии, кроме автора комментария	US-11	Medium
FR-51	Система должна считать задачу просроченной, если срок прошёл, а статус не «Готова» и не «Закрыта», и выделять её иконкой и текстом «Просрочено»	US-10	Medium
FR-52	Система должна ежедневно в 09:00 отмечать просроченные задачи и отправлять исполнителю одно уведомление по каждой задаче	US-10, H5	Medium
FR-53	Система должна отправлять менеджеру проекта один email-дайджест в день со списком просроченных задач проекта	US-10, H5	Medium
FR-54	Система должна снимать отметку «Просрочено» после переноса срока на будущую дату и уведомлять исполнителя об изменении срока	US-10, UC-T5	Medium

4. Требования к данным

4.1. Логическая модель данных

Логическая модель построена по агрегатам Event Storming: учётная запись, проект, задача, комментарий, уведомление. Ключевые правила модели:

- исполнитель задачи - участник проекта, а не произвольный пользователь;
- статусы и типы задач - справочники;
- связь задач и тегов - «многие ко многим»;
- история изменений хранится отдельно от текущего состояния задачи.

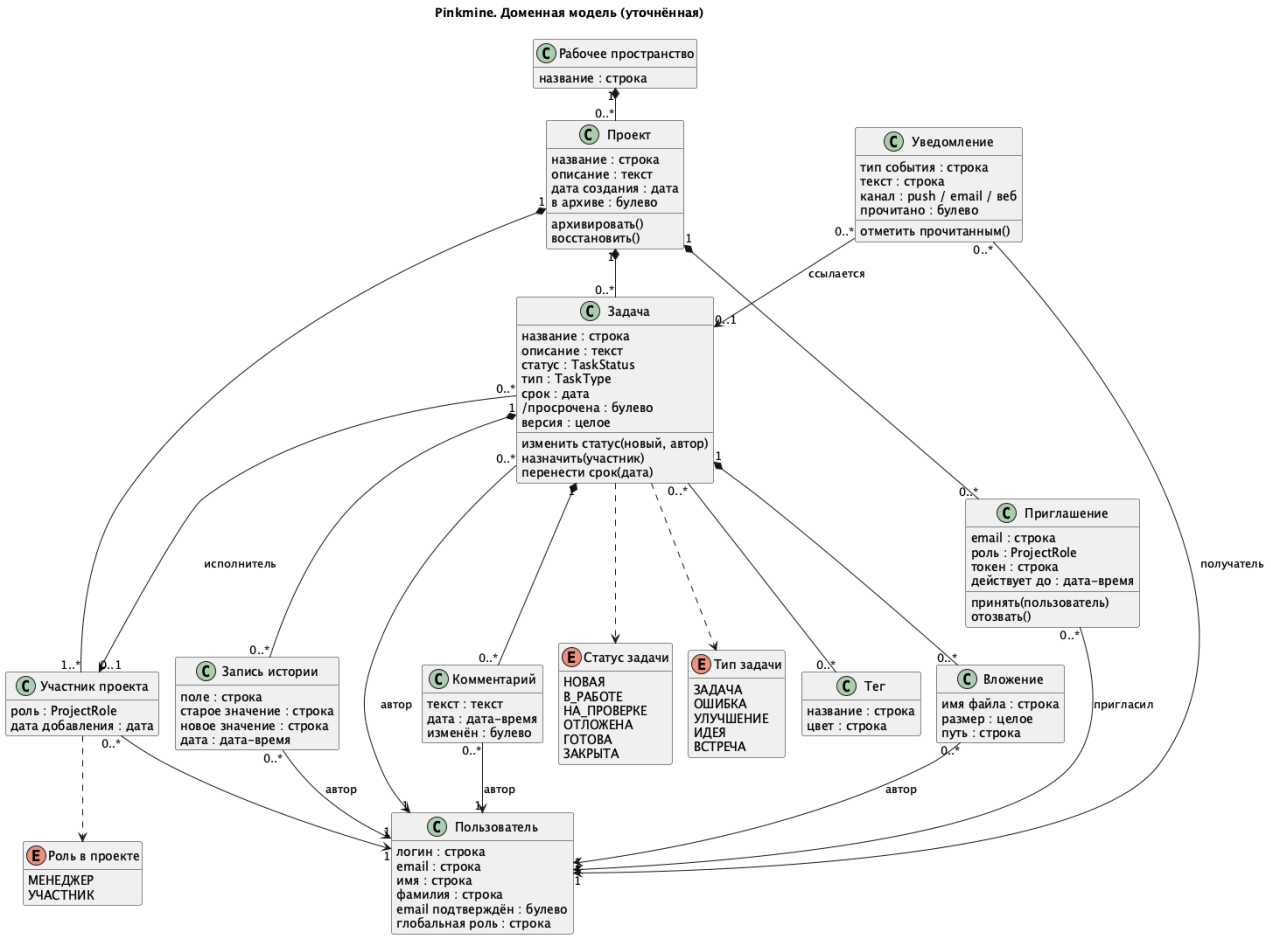


Рисунок 18 - Доменная модель

Физическая модель для PostgreSQL состоит из 19 таблиц (рисунок 19).

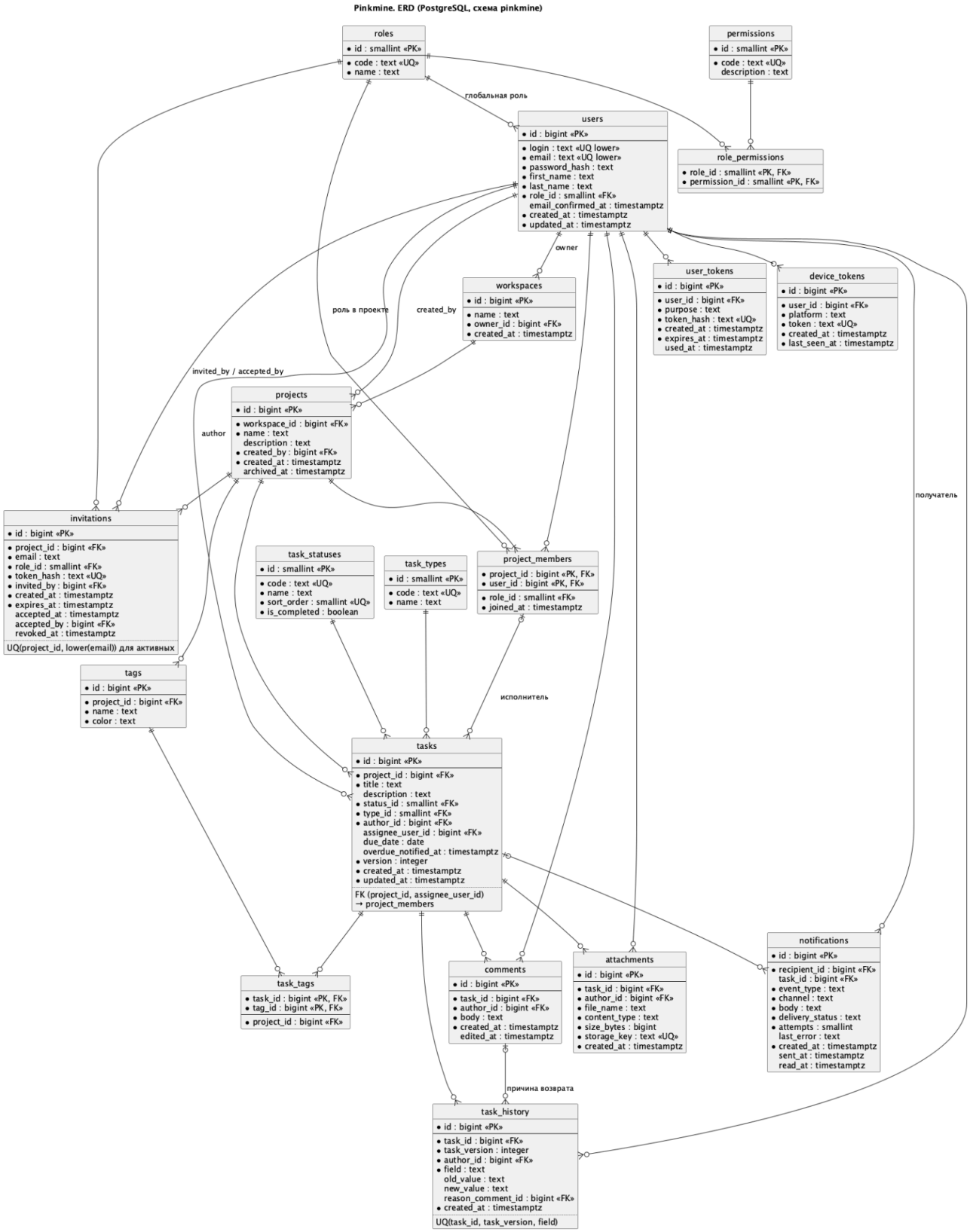


Рисунок 19 - ER-диаграмма базы данных

4.2. Словарь данных

Таблица 8 - Основные сущности

Таблица	Назначение	Ключевые поля
users	Учётная запись	login, email (уникальны без учёта регистра), password_hash, first_name, last_name, role_id (глобальная роль), email_confirmed_at
roles, permissions, role_permissions	Роли и их права	code роли (admin, manager, member), code права (например, task.close)
user_tokens	Подтверждение email, сброс пароля	purpose, token_hash, expires_at, used_at
device_tokens	Адреса push-уведомлений	platform, token
workspaces	Рабочее пространство	name, owner_id
projects	Проект	name, description, created_by, archived_at
project_members	Участник проекта	(project_id, user_id) - первичный ключ, role_id - роль в проекте
invitations	Приглашение по email	email, role_id, token_hash, expires_at, accepted_at, revoked_at
task_statuses	Справочник статусов	code (new, in_progress, review, postponed, done, closed), sort_order, is_completed
task_types	Справочник типов	code (task, bug, improvement, idea, meeting)
tasks	Задача	title, description, status_id, type_id, author_id, assignee_user_id, due_date, overdue_notified_at, version
tags, task_tags	Теги проекта и их привязка к задачам	name, color; (task_id, tag_id)

comments	Комментарий	task_id, author_id, body, edited_at
attachments	Метаданные вложения	file_name, content_type, size_bytes, storage_key
task_history	История изменений задачи	task_version, field, old_value, new_value, reason_comment_id
notifications	Уведомление	recipient_id, event_type, channel (push, email, in_app), delivery_status, attempts, read_at

4.3. Отчёты

- **Список задач проекта с фильтрами** (FR-33, FR-34) - основной источник для отчёта менеджера заказчику: задачи по статусу, типу, исполнителю, сроку и тегу.
- **Сводка по проекту** - представление Event Modeling, которое обновляется при приёмке задач.
- **Ежедневный email-дайджест просроченных задач** для менеджера проекта (FR-53).
- **История изменений задачи** - кто, когда и что изменил, число возвратов на доработку.

4.4. Получение, целостность, хранение и утилизация данных

- **Целостность** обеспечивается ограничениями БД:
  - составной внешний ключ не даёт назначить исполнителем пользователя из чужого проекта;
  - теги привязываются только к задачам своего проекта;
  - пользователь входит в проект не более одного раза;
  - на один email в проекте действует только одно активное приглашение.
- **Конкурентные изменения** задачи контролируются полем `version` (оптимистическая блокировка). Повторная отправка изменения из офлайн-очереди не создаёт дублей истории.
- **Вычисляемые значения** (признак просрочки) не хранятся; хранится только факт отправленного напоминания.
- **Хранение:** текущее состояние задачи и история изменений хранятся раздельно. Уведомление записывается в той же транзакции, что и вызвавшее его изменение.
- **Утилизация:**
  - проект не удаляется, а архивируется;

- физически проект удаляет только администратор, вместе с задачами, комментариями, вложениями и историей;
- пользователи физически не удаляются;
- файлы вложений, оставшиеся в хранилище после удаления задачи, удаляются фоновой очисткой.

5. Требования к внешним интерфейсам

5.1. Пользовательские интерфейсы

Таблица 9 - Различия пользовательских интерфейсов

Параметр	**Веб-клиент**	**Android-приложение**
Экран	1440 px, планшет 980 px, 12 колонок	375×670 px, сетка 8 pt
Навигация	Header: «Проекты», «Мои задачи», «Команда», поиск, уведомления, профиль; footer со справкой	Нижняя панель: «Главная», «Проекты», «Мои задачи», «Профиль»
Ввод	Мышь и клавиатура, отправка форм по Enter	Сенсорный ввод, клавиатура для email в поле email
Смена статуса	Выбор в карточке или перетаскивание по доске	Касание статуса и выбор в нижней панели
Поиск и фильтры	Строка поиска в header, панель фильтров над списком	Иконка лупы, полноэкранный поиск
Уведомления	Колокольчик в header	Push с переходом в карточку задачи
Создание задачи	Окно на Канбан-доске без перехода на другую страницу	Полноэкранная форма, срок через календарь
Вложения	Перетаскивание файла или кнопка	Галерея, камера, файловый менеджер
Офлайн-режим	Не поддерживается, при потере связи - сообщение	Кэш задач, очередь смены статусов

Роли и права	Раздел «Роли и права» для администратора	Отсутствует

5.2. Интерфейсы программного обеспечения

REST API. Базовый путь `/api`, формат JSON, аутентификация - заголовок `Authorization: Bearer <token>`. Основные маршруты определены срезами Event Modeling (таблица 10).

Таблица 10 - Основные маршруты REST API

Метод и путь	**Назначение**	**Срез / требование**
POST /api/auth/register	Регистрация	S1 / FR-01
POST /api/projects	Создание проекта	S2 / FR-11
POST /api/projects/{id}/invites	Приглашение участника	S3 / FR-15
POST /api/invites/{token}/accept	Принятие приглашения	S5 / FR-17
POST /api/projects/{id}/tasks	Создание задачи	S6 / FR-21
PATCH /api/tasks/{id}/assignee	Назначение исполнителя	S7 / FR-22
PATCH /api/tasks/{id}/status	Смена статуса, проверка, возврат на доработку	S9, S11, S12, S17 / FR-26…FR-31, FR-36…FR-38
POST /api/tasks/{id}/comments	Комментарий	S10 / FR-39
PATCH /api/tasks/{id}/deadline	Установка и перенос срока	S13, S16 / FR-23, FR-54

Метод `PATCH /api/tasks/{taskId}/status` полностью описан в постановке задачи, подготовленной навыком `getanalyst-restapi-designer-skill` (отчёт по заданию 4). Ключевые положения:

- в теле передаются новый статус и версия задачи;
- при устаревшей версии - 409 `TASK_VERSION_CONFLICT`;
- при закрытии задачи без права `task.close` - 403 `TASK_CLOSE_FORBIDDEN`;
- при возврате на доработку без комментария - 422 `REASON_COMMENT_REQUIRED`;
- при изменении задачи в архивном проекте - 409 `PROJECT_ARCHIVED`;
- при успехе - 200 с текущими статусом и версией задачи.

Firebase Cloud Messaging. Сервер отправляет push-уведомления по токенам устройств, которые Android-приложение регистрирует на сервере.

Почтовый сервис (SMTP). Сервер отправляет письма подтверждения email, сброса пароля, приглашения и ежедневный дайджест просрочек.

5.3. Интерфейсы оборудования

Специального оборудования система не требует. Android-приложение использует камеру и галерею смартфона для прикрепления файлов к задаче (FR-41) и механизм push-уведомлений операционной системы.

5.4. Коммуникационные интерфейсы

- Клиенты обмениваются данными с сервером по HTTPS в формате JSON.
- Сервер обращается к Firebase Cloud Messaging по HTTPS, к почтовому сервису - по SMTP.
- Сервер приложений и фоновый процесс обращаются к PostgreSQL по протоколу СУБД.
- Модули сервера обмениваются доменными событиями через внутреннюю шину событий (рисунок 20).

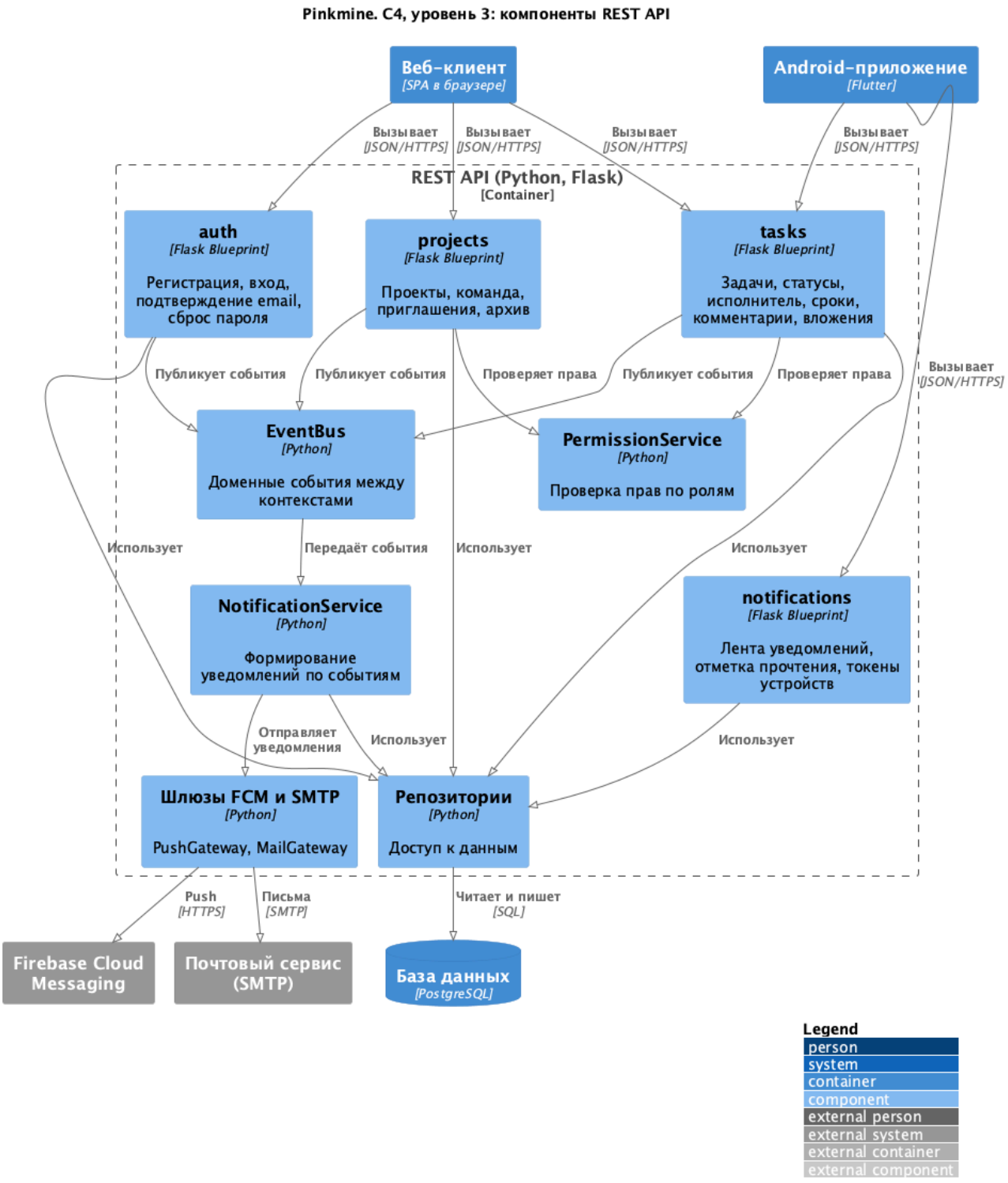


Рисунок 20 - Компоненты REST API (C4, уровень 3)

6. Атрибуты качества

6.1. Удобство использования

- Смена статуса задачи выполняется не более чем в два действия (FR-27).

- Задача создаётся на Канбан-доске без перехода на другую страницу [Web].
- Статусы обозначаются иконкой и текстом, используются контрастные цвета и шрифт без засечек.
- Ошибки ввода показываются у конкретного поля формы.
- [Android] Экраны помещаются на 375 px без горизонтальной прокрутки, пароль скрыт с кнопкой «показать».

6.2. Производительность

- Уведомления формируются асинхронно по доменным событиям: ответ пользователю не ожидает доставки через FCM или почтовый сервис.
- Ежедневная фиксация просрочек выполняется отдельным фоновым процессом и не замедляет обработку запросов пользователей.
- Списки задач и уведомлений выдаются постранично. Для ленты уведомлений используется пагинация по ключу.
- Запросы к задачам проекта, «Моим задачам», ленте уведомлений и проверке просрочек обслуживаются индексами БД.

6.3. Безопасность

- Пароли, токены приглашений, подтверждения email и сброса пароля хранятся только в виде хеша.
- Все методы API, кроме регистрации, входа и сброса пароля, требуют Bearer-токен. Субъект запроса определяется только по токену.
- Права проверяются единым сервисом по таблице прав ролей. Доступ к объекту проверяется отдельно от роли: на задачи чужих проектов сервер отвечает 404, не раскрывая их существование.
- Форма восстановления пароля не позволяет проверить, зарегистрирован ли адрес (FR-06).
- Ответы API не содержат SQL, трассировок стека и внутренних адресов. Токены и пароли не записываются в журналы.

6.4. Техника безопасности (сохранность данных)

- Изменения, сделанные в офлайне, не теряются: очередь сохраняется при закрытии приложения (FR-43).
- Одновременные изменения одной задачи не перезаписывают друг друга (FR-31, H3).
- Архивирование проекта требует подтверждения. Физическое удаление доступно только администратору (FR-18, FR-20).
- Закрыть задачу может только менеджер после проверки результата (FR-29, H2).

6.5. Надёжность

- При недоступности FCM или почтового сервиса уведомление остаётся в системе со статусом доставки и отправляется повторно. Число попыток и последняя ошибка сохраняются.
- Повторный запуск проверки просрочек не создаёт повторных уведомлений.
- Каждое успешное изменение задачи фиксируется в истории и в журнале с идентификатором трассировки.

7. Требования по интернационализации и локализации

- Интерфейс веб-клиента и Android-приложения - на русском языке, включая тексты уведомлений и писем.
- Даты в интерфейсе отображаются в формате ДД.ММ.ГГГГ.
- В API дата и время передаются в формате RFC 3339 в UTC, срок задачи - календарной датой.
- Коды статусов, типов, ролей и ошибок в API - латиницей в нижнем регистре (`in_progress`) или верхнем регистре для кодов ошибок (`TASK_VERSION_CONFLICT`). Пользователю показываются русские названия.
- Наступление срока определяется в едином часовом поясе системы.

8. Остальные требования

- **Миграции БД:** структура БД создаётся миграциями, справочники ролей, прав, статусов и типов загружаются отдельной миграцией. Несовместимые изменения выполняются поэтапно (expand-and-contract).
- **Тестовые данные:** для разработки и тестирования используется отдельный набор данных на основе персонажей ЛР2. В тестовой среде не хранятся реальные пароли и токены.
- **Персональные данные:** система хранит минимум персональных данных - логин, имя, фамилию, email.

Приложение А. Словарь терминов

Таблица 11 - Словарь терминов

Термин	**Определение**
Рабочее пространство	Группа проектов одного владельца
Проект	Контейнер задач с собственной командой

Участник проекта	Пользователь, добавленный в команду проекта с ролью «менеджер» или «участник»
Глобальная роль	Роль пользователя в системе (администратор, менеджер, участник), определяющая системные права
Задача	Единица работы в проекте с типом, статусом, исполнителем и сроком
Канбан-доска	Представление задач проекта колонками по статусам
Просроченная задача	Задача, срок которой прошёл, а статус не «Готова» и не «Закрыта»
Возврат на доработку	Перевод задачи из «На проверке» в «В работе» с комментарием о причине
Версия задачи	Счётчик изменений задачи для обнаружения одновременных изменений
Офлайн-очередь	Сохранённые на устройстве изменения статусов, ожидающие отправки на сервер
Push-уведомление	Уведомление, доставляемое на смартфон через Firebase Cloud Messaging
Дайджест	Ежедневное письмо менеджеру со списком просроченных задач проекта
Ограниченный контекст	Часть предметной области с собственной моделью (Event Storming, DDD)
Доменное событие	Факт, произошедший в системе, например «Статус задачи изменён»
Flask Blueprint	Модуль серверного приложения Flask, объединяющий маршруты одного контекста

Приложение Б. Модели анализа

Б.1. Сценарии вариантов использования

Таблица 12 - Сценарии подсистемы «Работа с задачами»

Сценарий	Актёр	Основной поток (кратко)	Альтернативы и исключения
UC-T1. Создать задачу	Участник, менеджер	Форма «Новая задача…» → название и описание → менеджер выбирает тип, срок, исполнителя → проверка полей → задача «Новая» в первой колонке доски	Участник создаёт задачу без типа, срока и исполнителя; пустое название; срок в прошлом; ошибка связи
UC-T2. Назначить исполнителя	Менеджер	Выбор участника проекта → сохранение и запись в историю → уведомление исполнителю → задача в «Моих задачах»	Снятие или смена исполнителя; пользователь исключён из проекта; FCM недоступен
UC-T3. Изменить статус задачи	Участник, менеджер	Выбор статуса → запрос с версией задачи → проверка прав и версии → новый статус в карточке и на доске → при «На проверке» уведомление менеджеру	Офлайн-очередь; участник выбирает «Закрыта» (403); устаревшая версия (409)
UC-T4. Проверить выполненную задачу	Менеджер	Открыть карточку → «Принять» → «Готова» → «Закрыть» → «Закрыта», уведомление исполнителю	Возврат на доработку с обязательным комментарием; задачу успели изменить
UC-T5. Перенести срок выполнения	Менеджер	Выбор новой даты → сохранение и запись в историю → снятие отметки «Просрочено» → уведомление исполнителю	Очистка срока; дата в прошлом

Б.2. Диаграммы деятельности

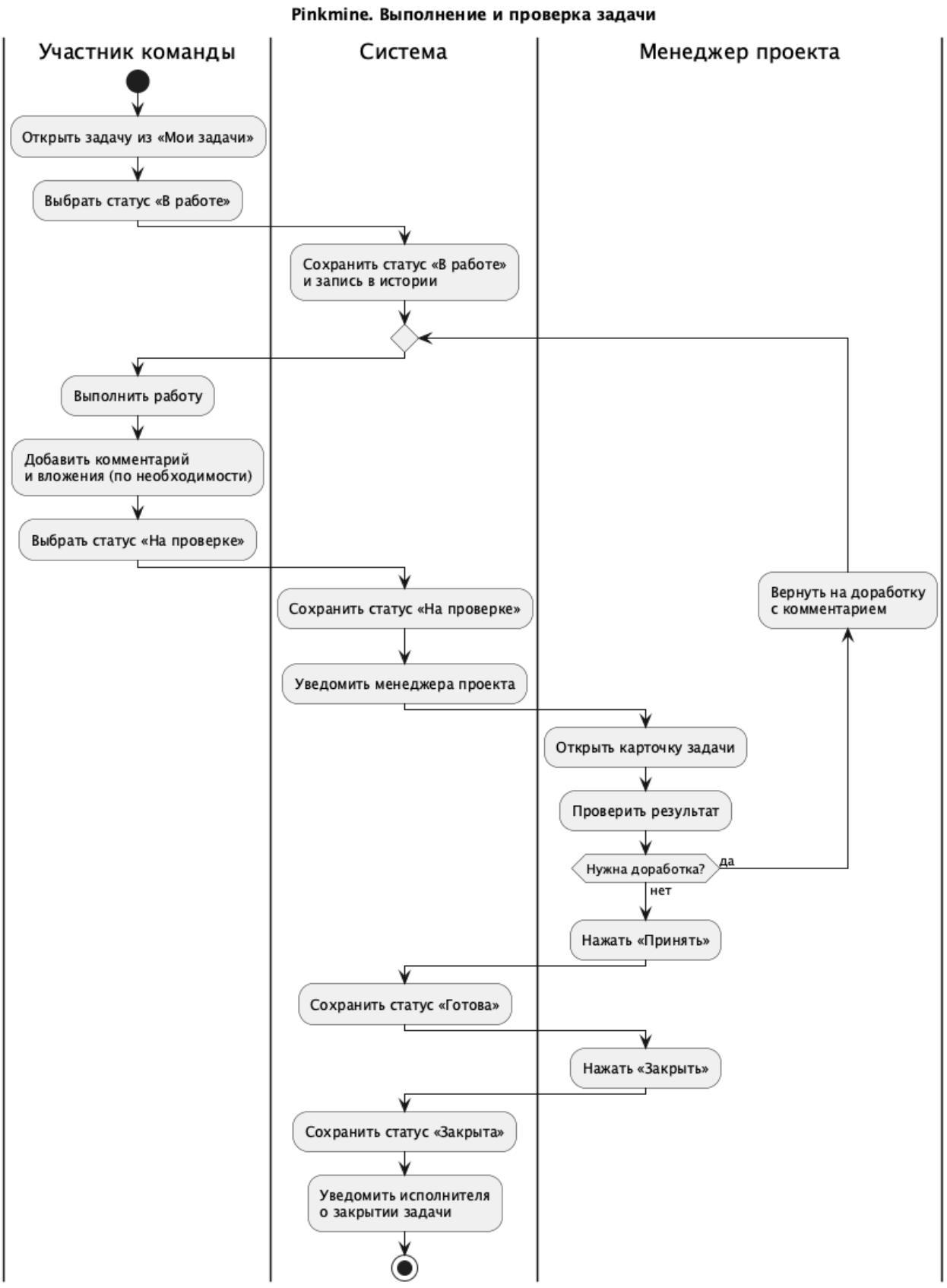


Рисунок 21 - Выполнение и проверка задачи

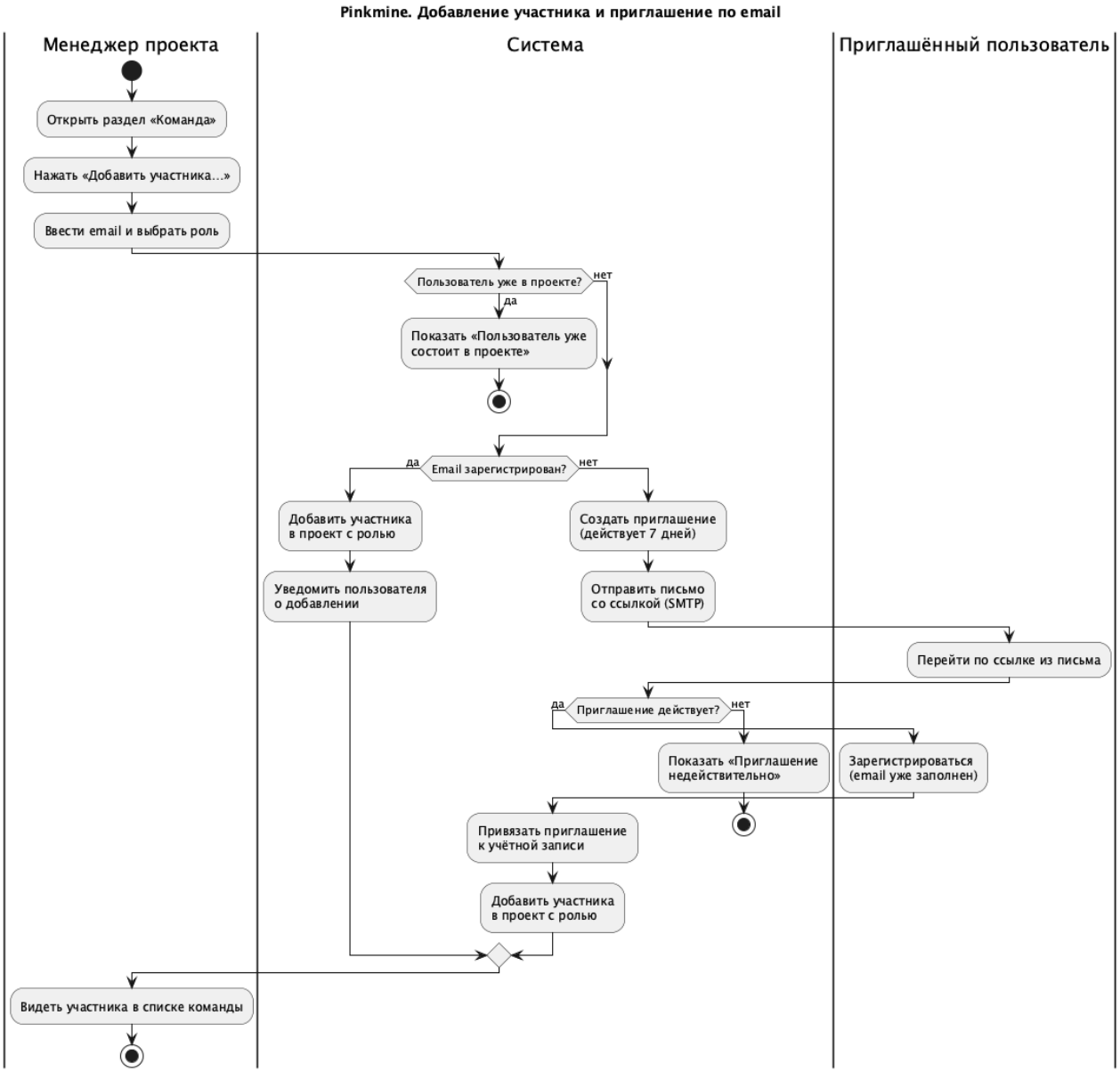


Рисунок 22 - Добавление участника и приглашение по email

Б.3. Диаграммы классов и объектов

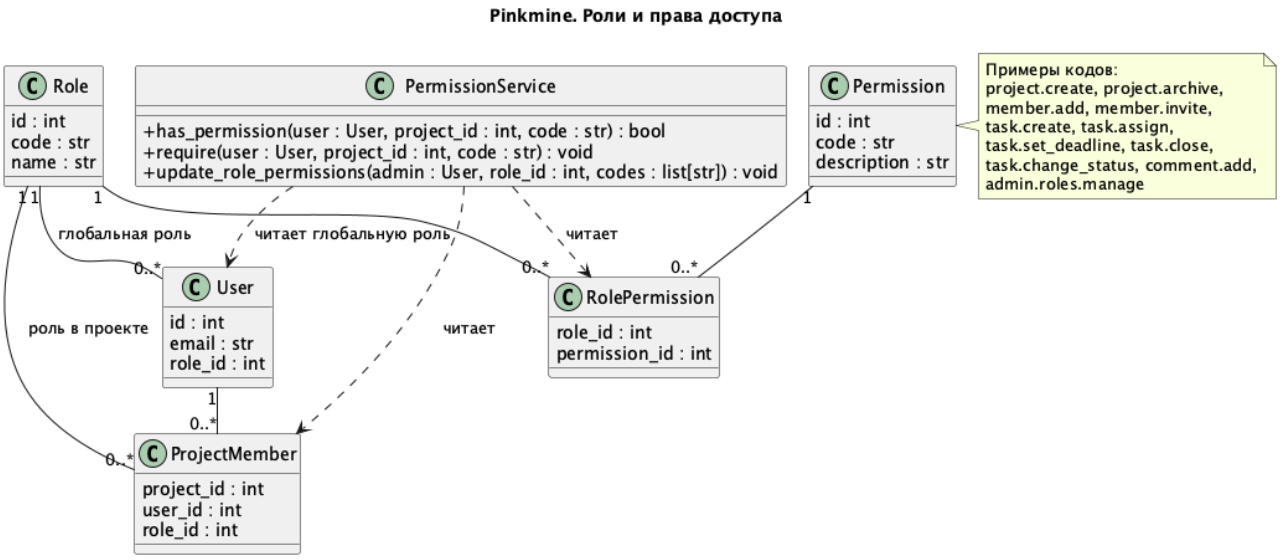


Рисунок 23 - Модель ролей и прав доступа

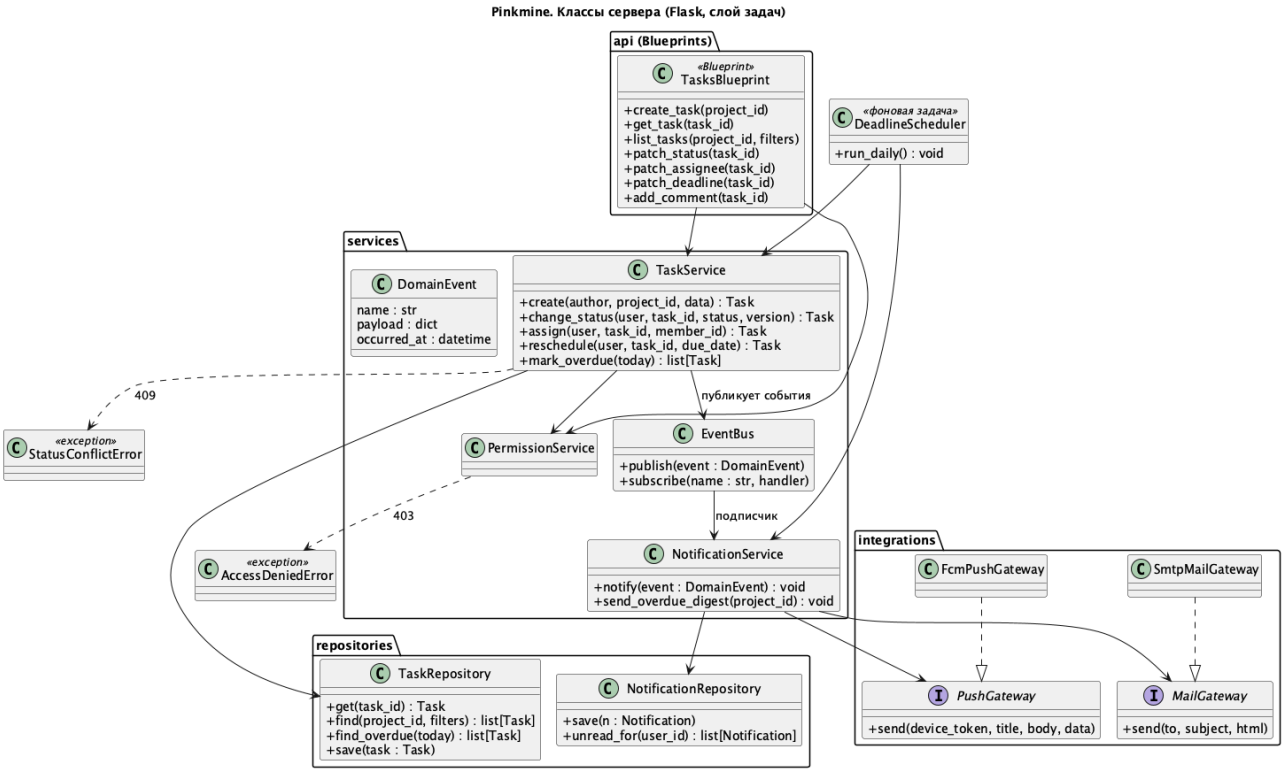


Рисунок 24 - Классы сервера (Flask)

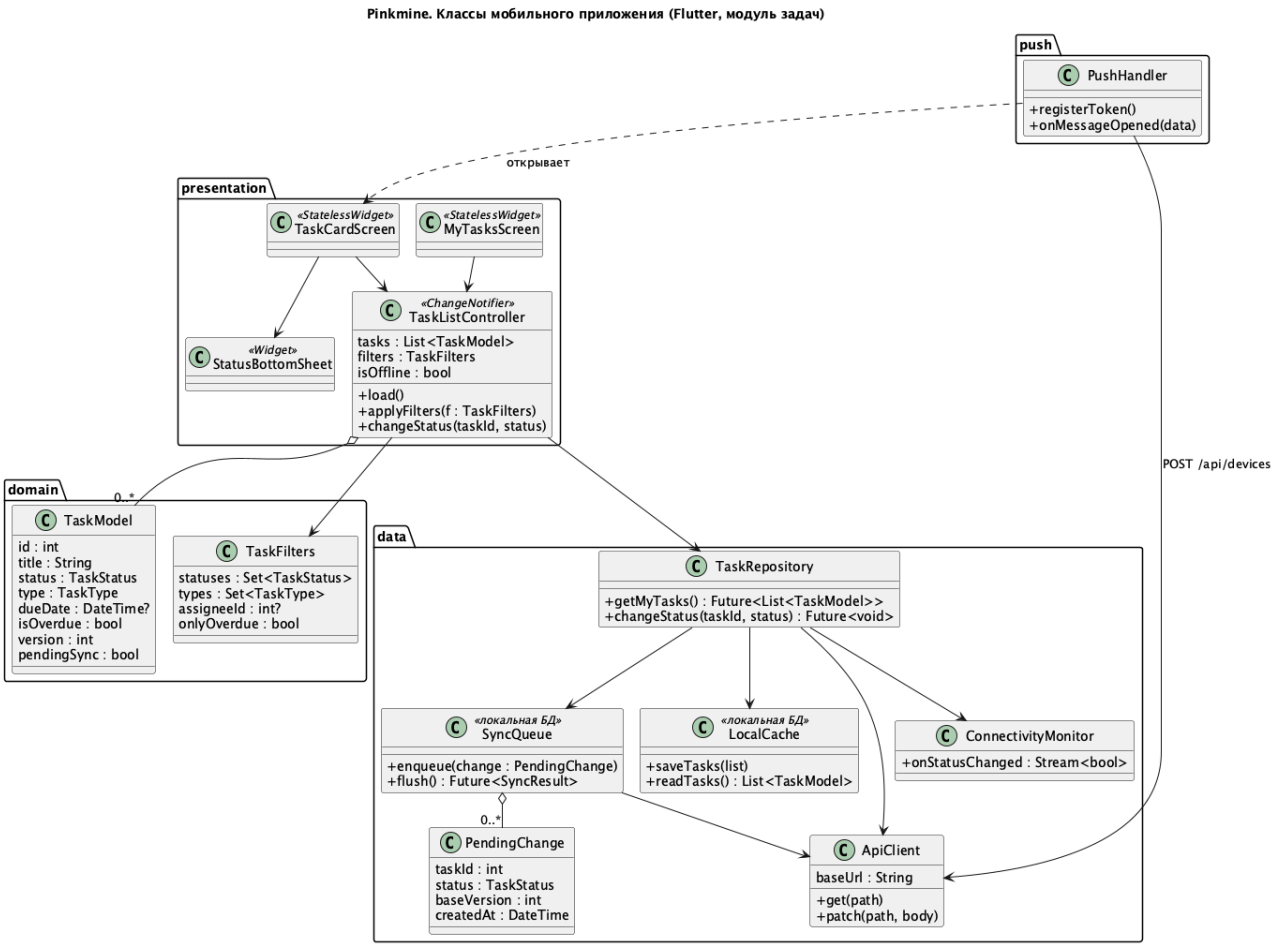


Рисунок 25 - Классы мобильного приложения (Flutter)

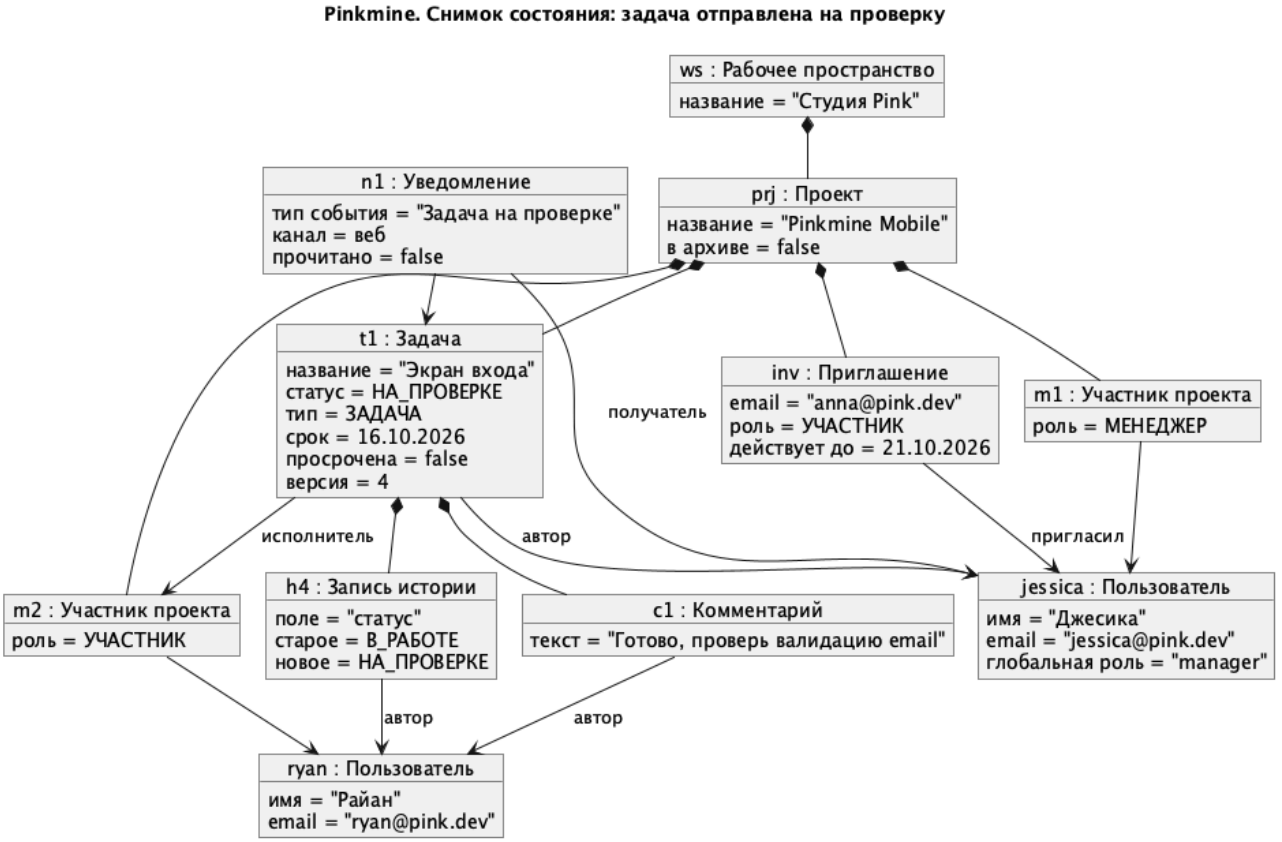


Рисунок 26 - Объектная диаграмма: задача отправлена на проверку

Б.4. Диаграммы последовательности

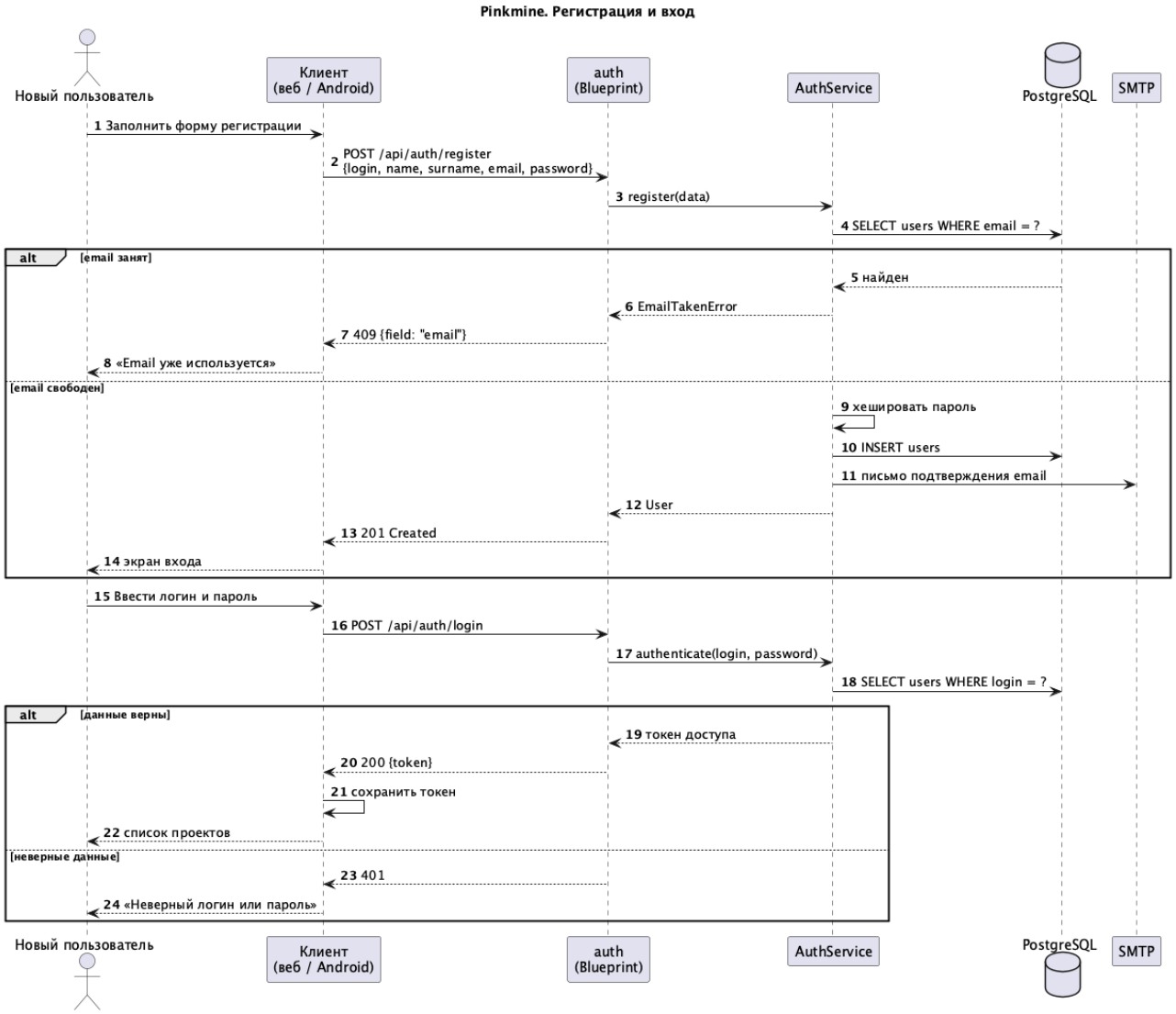


Рисунок 27 - Регистрация и вход

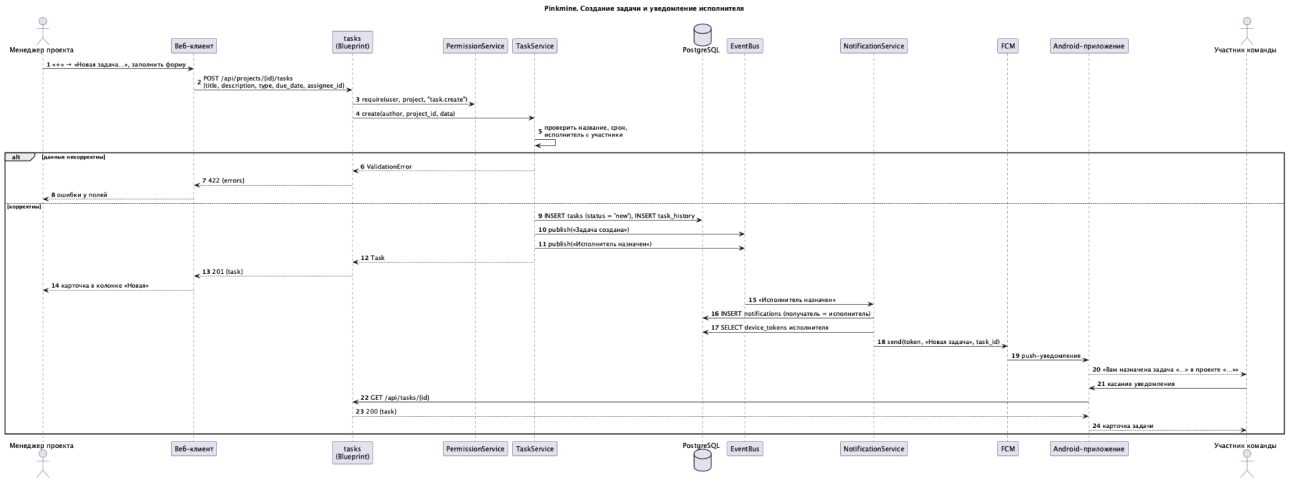


Рисунок 28 - Создание задачи и уведомление исполнителя

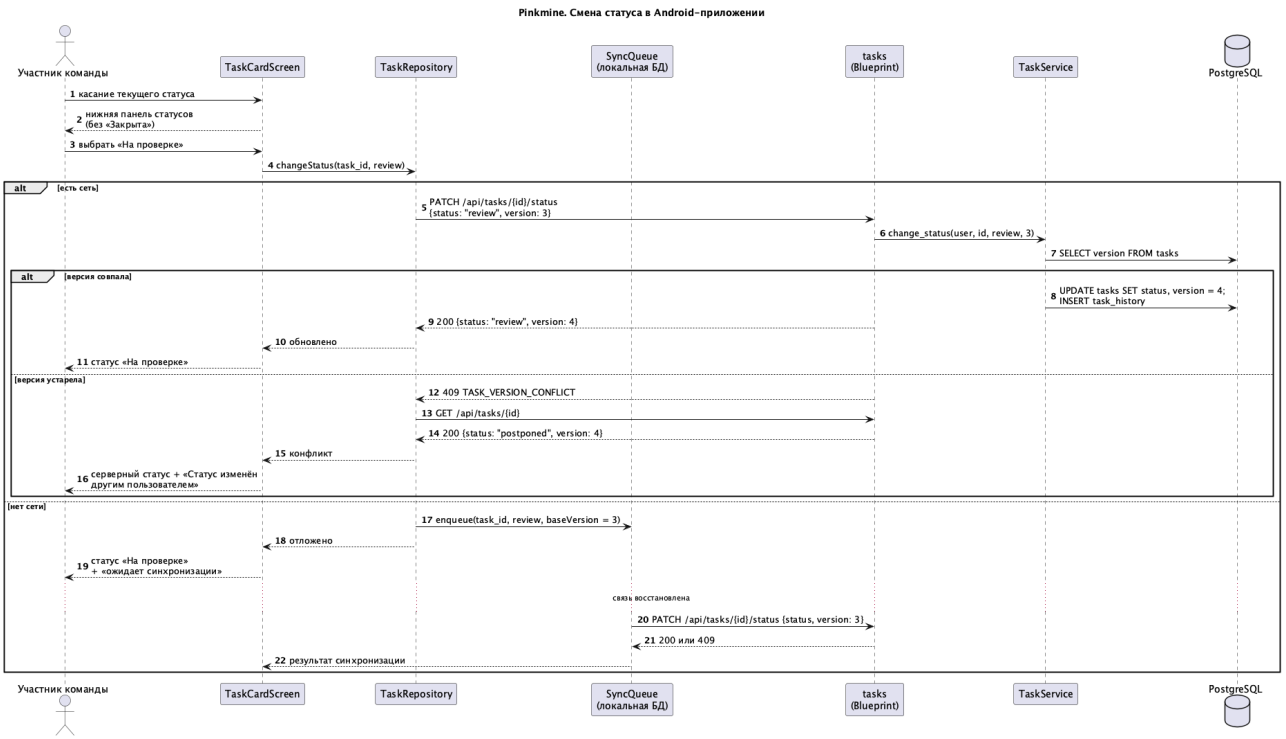


Рисунок 29 - Смена статуса в Android-приложении (онлайн и офлайн)

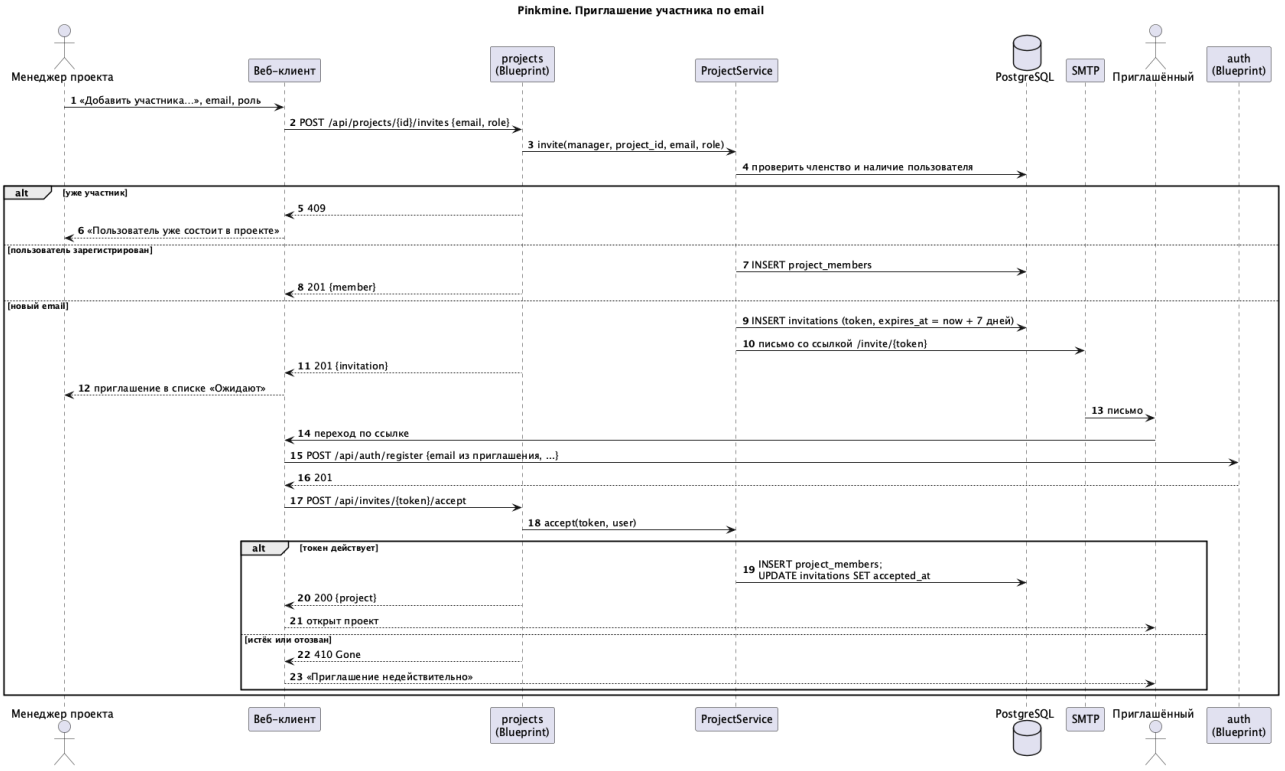


Рисунок 30 - Приглашение участника по email

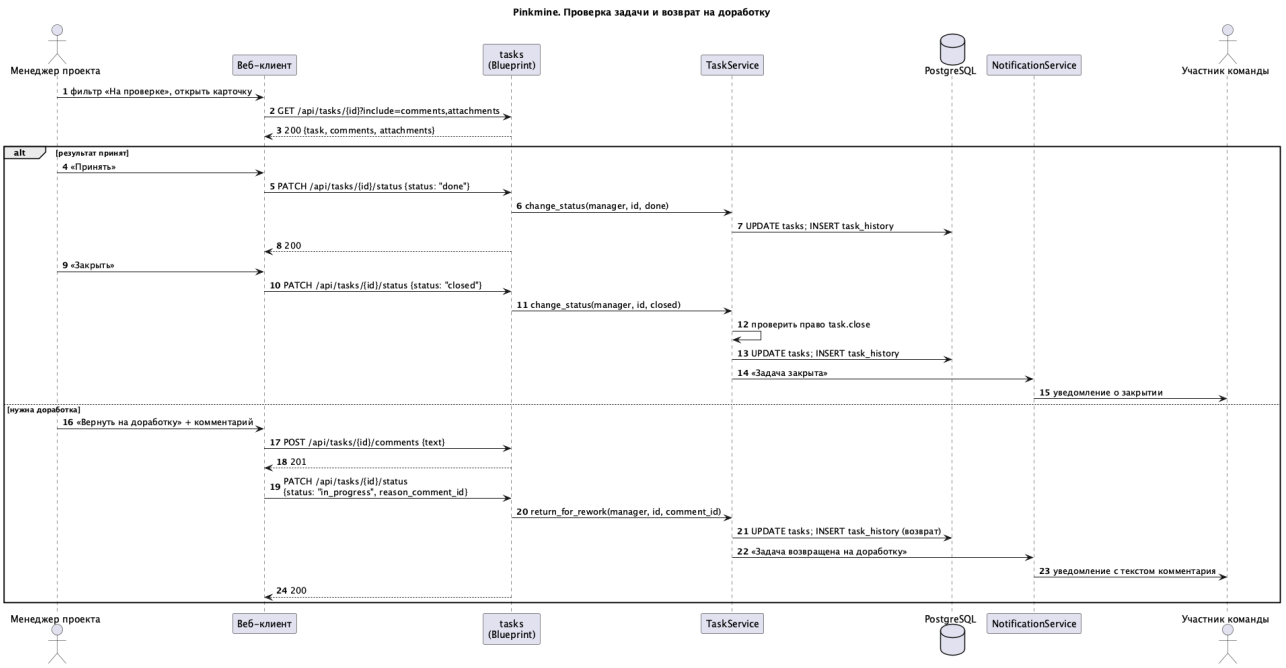


Рисунок 31 - Проверка задачи и возврат на доработку

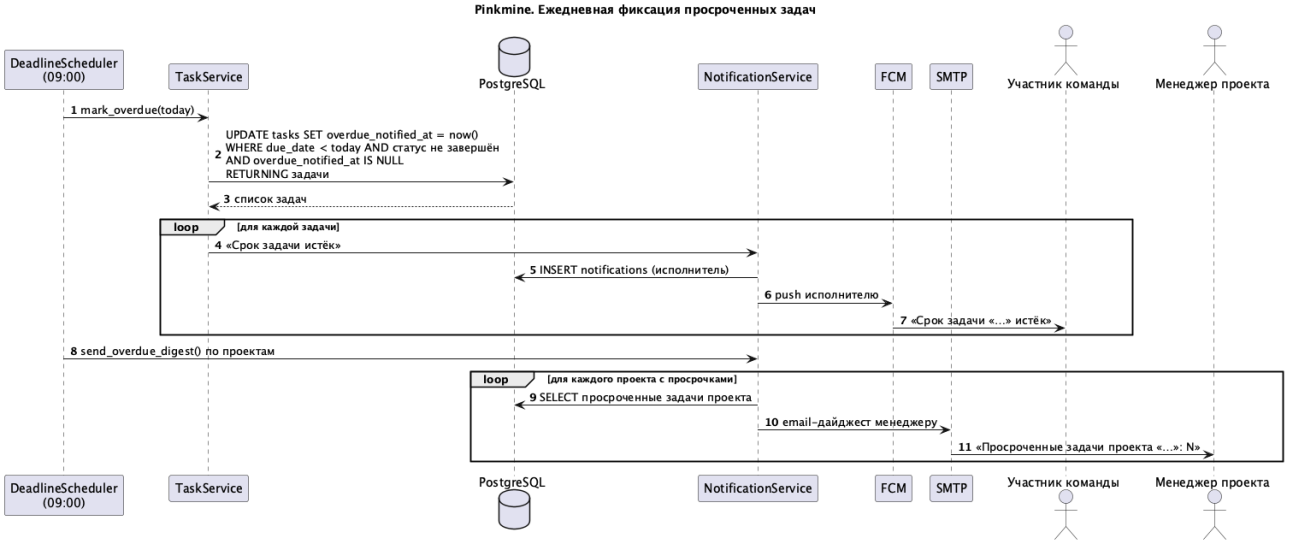


Рисунок 32 - Ежедневная фиксация просроченных задач

Б.5. Компоненты, пакеты, развёртывание

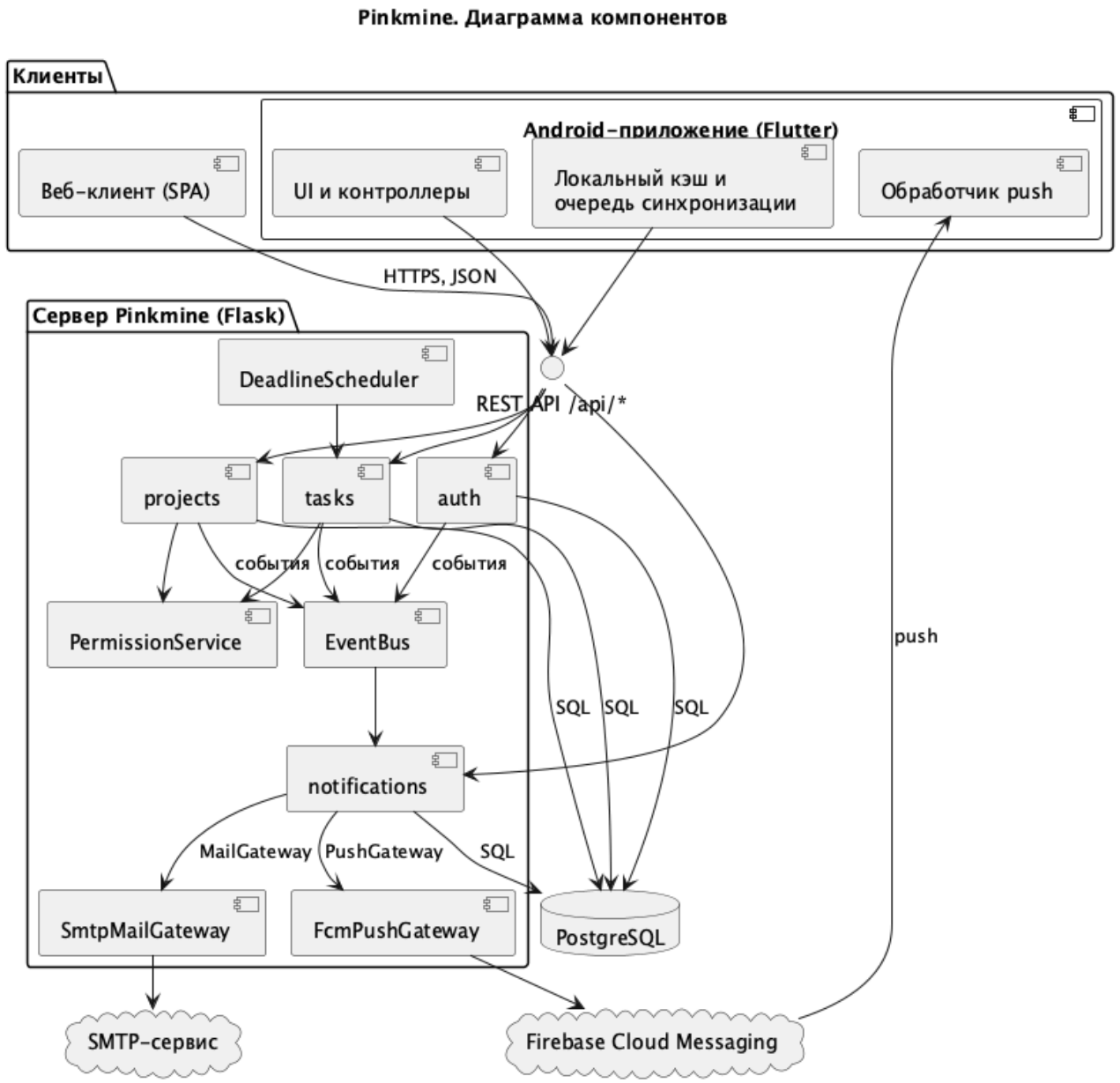


Рисунок 33 - Диаграмма компонентов

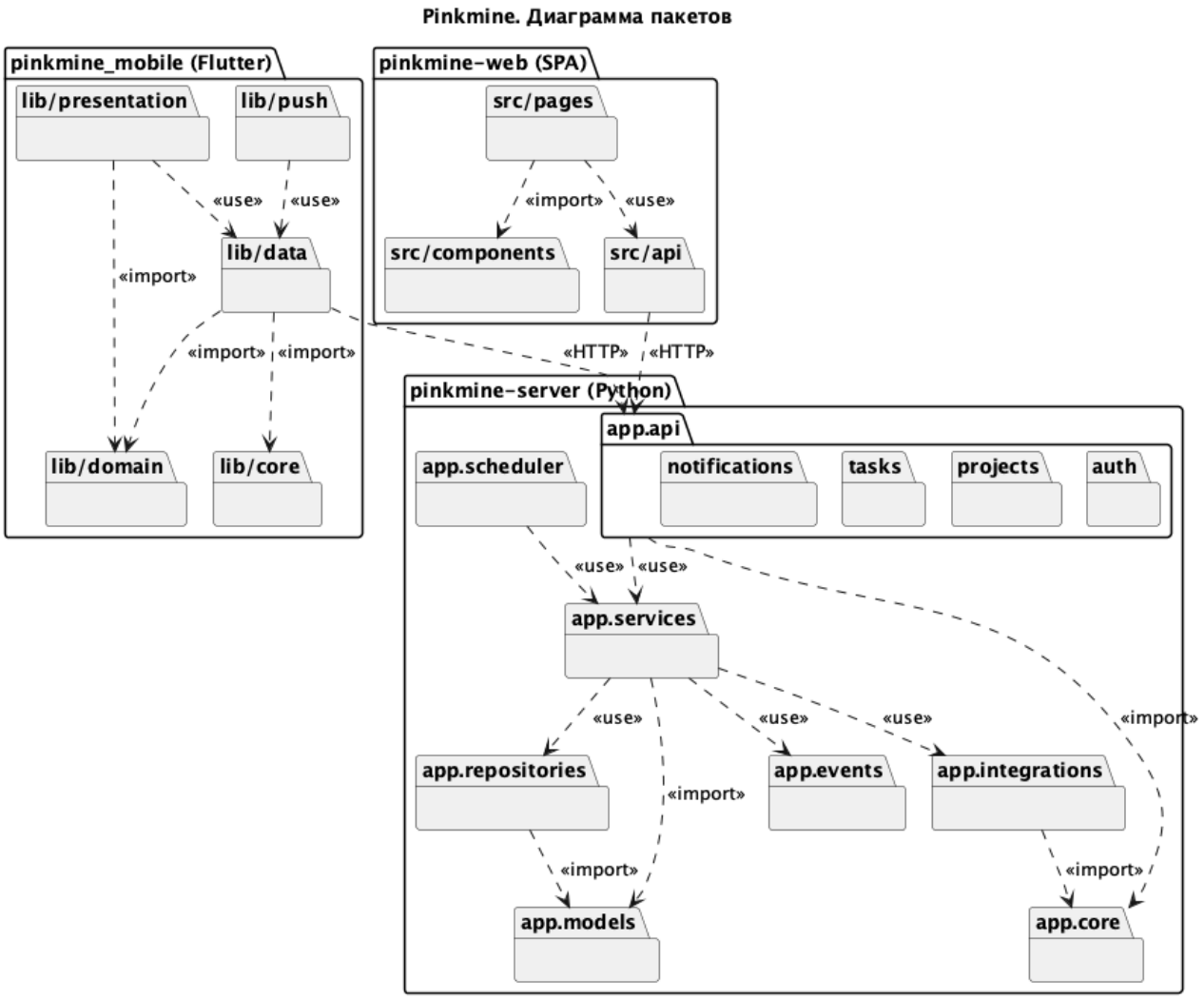


Рисунок 34 - Диаграмма пакетов

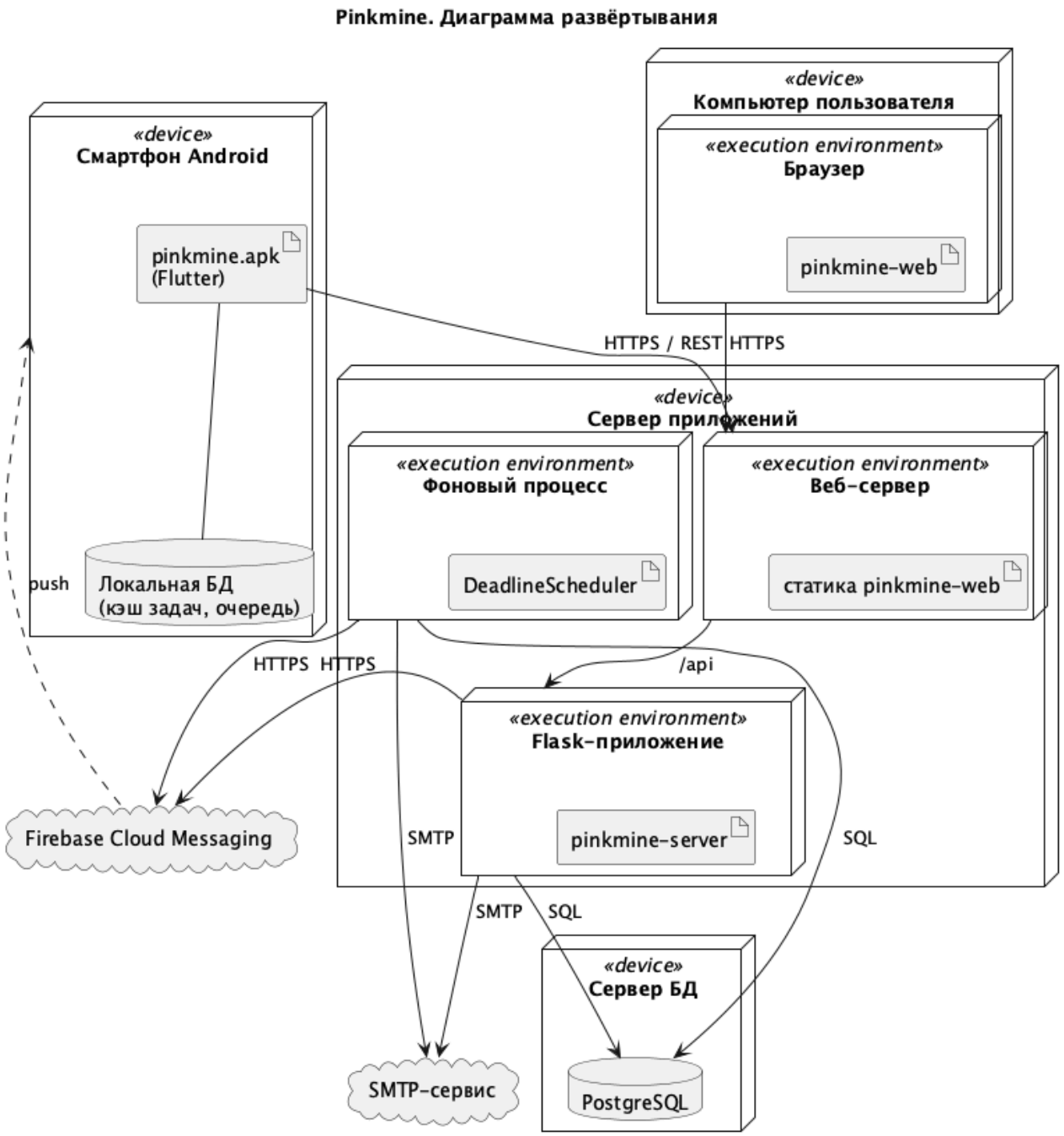


Рисунок 35 - Диаграмма развёртывания (UML)

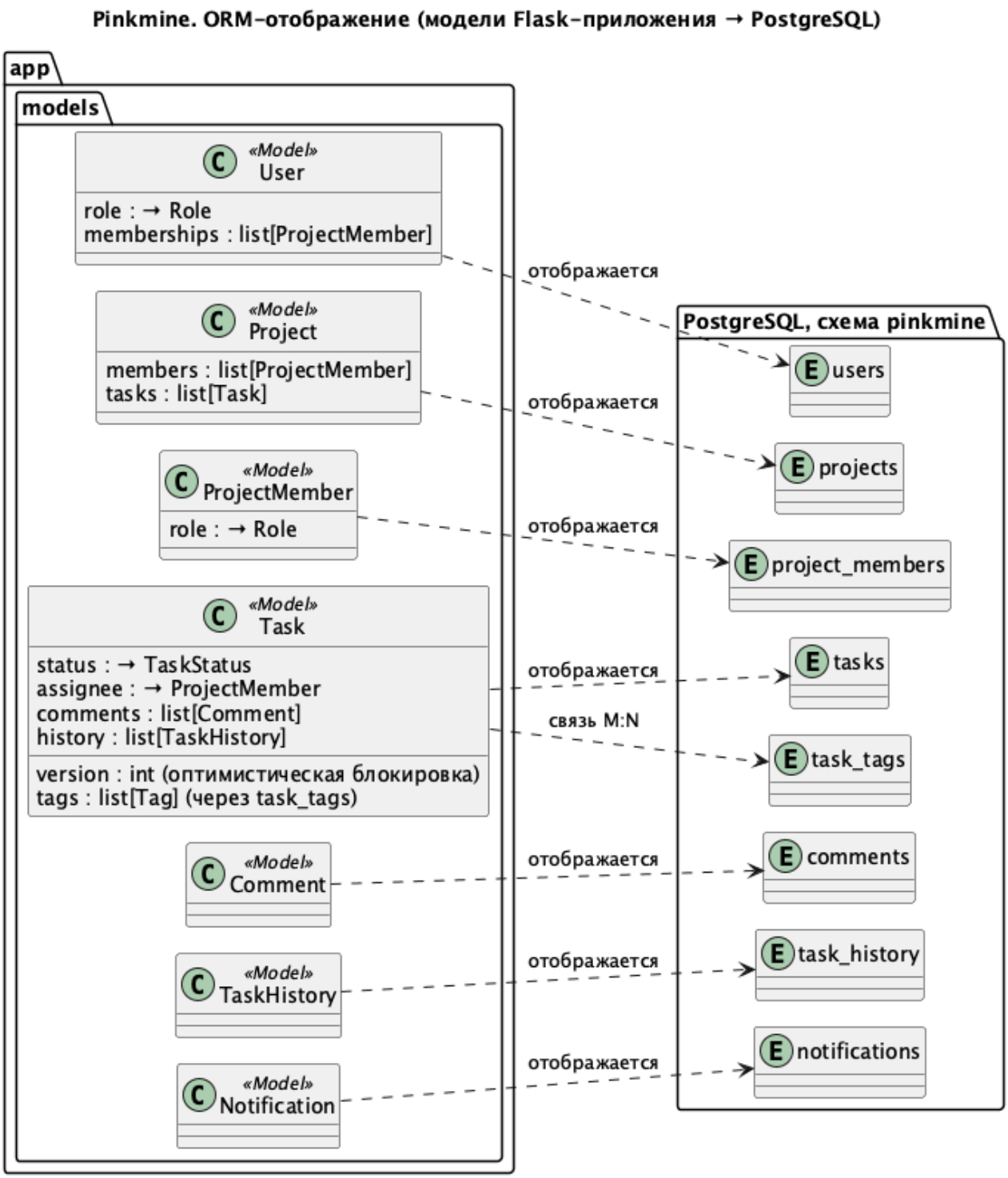


Рисунок 36 - ORM-отображение

Б.6. Event Storming и Event Modeling

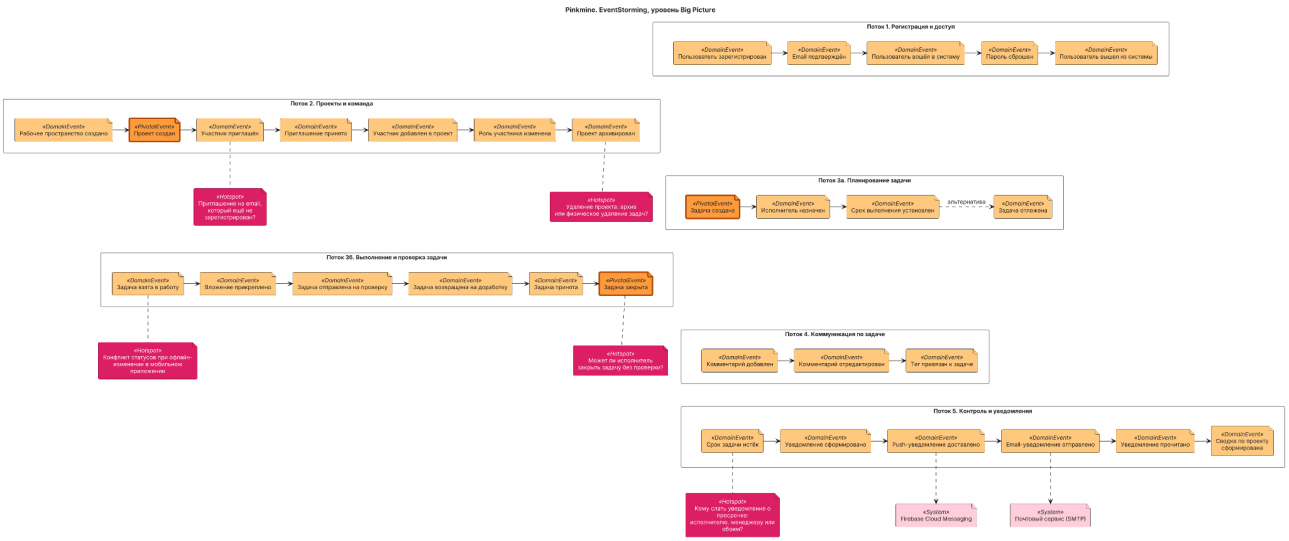


Рисунок 37 - Event Storming, Big Picture: временная линия событий

Pinkmine. EventStorming, Software Design: ограниченные контексты и интеграционные события

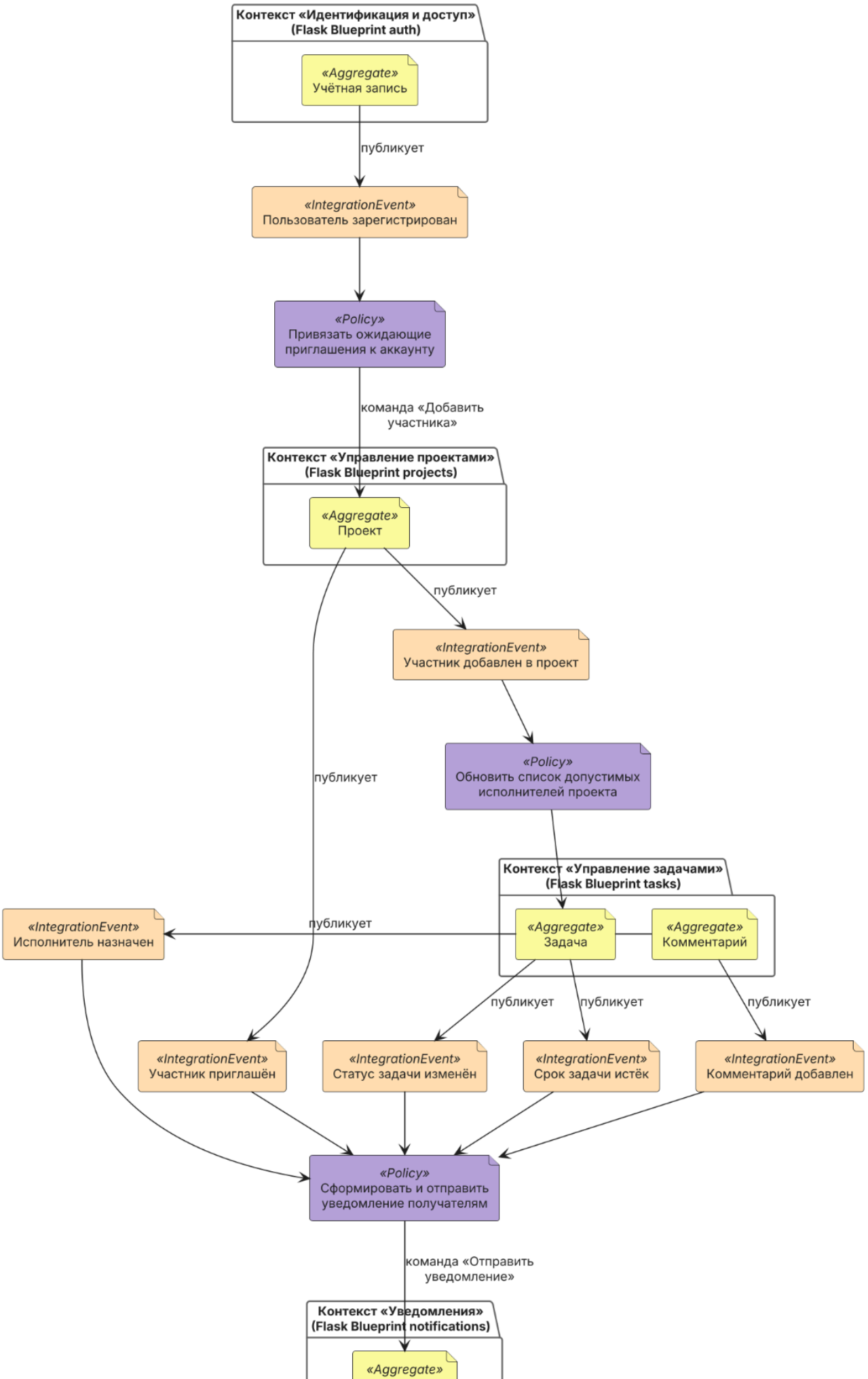


Рисунок 38 - Event Storming, Software Design: ограниченные контексты

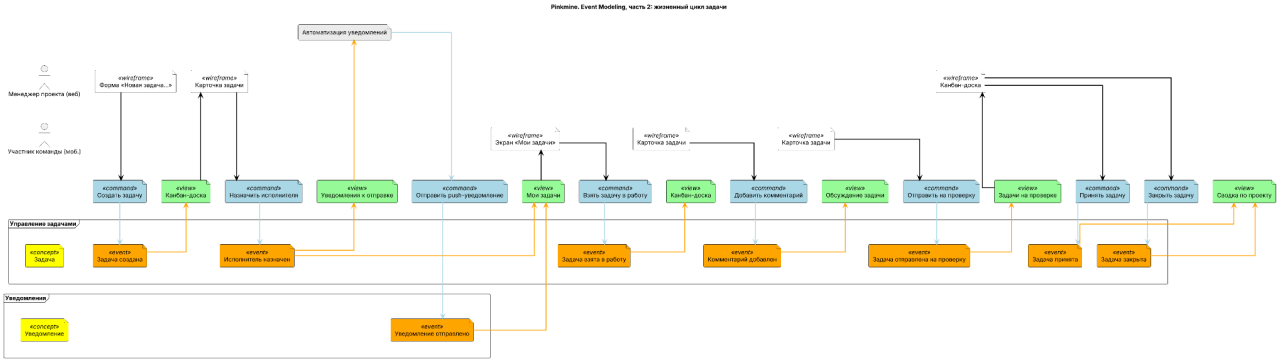


Рисунок 39 - Event Modeling: жизненный цикл задачи

Б.7. Динамическая диаграмма С4

Pinkmine. C4, динамическая диаграмма: смена статуса задачи

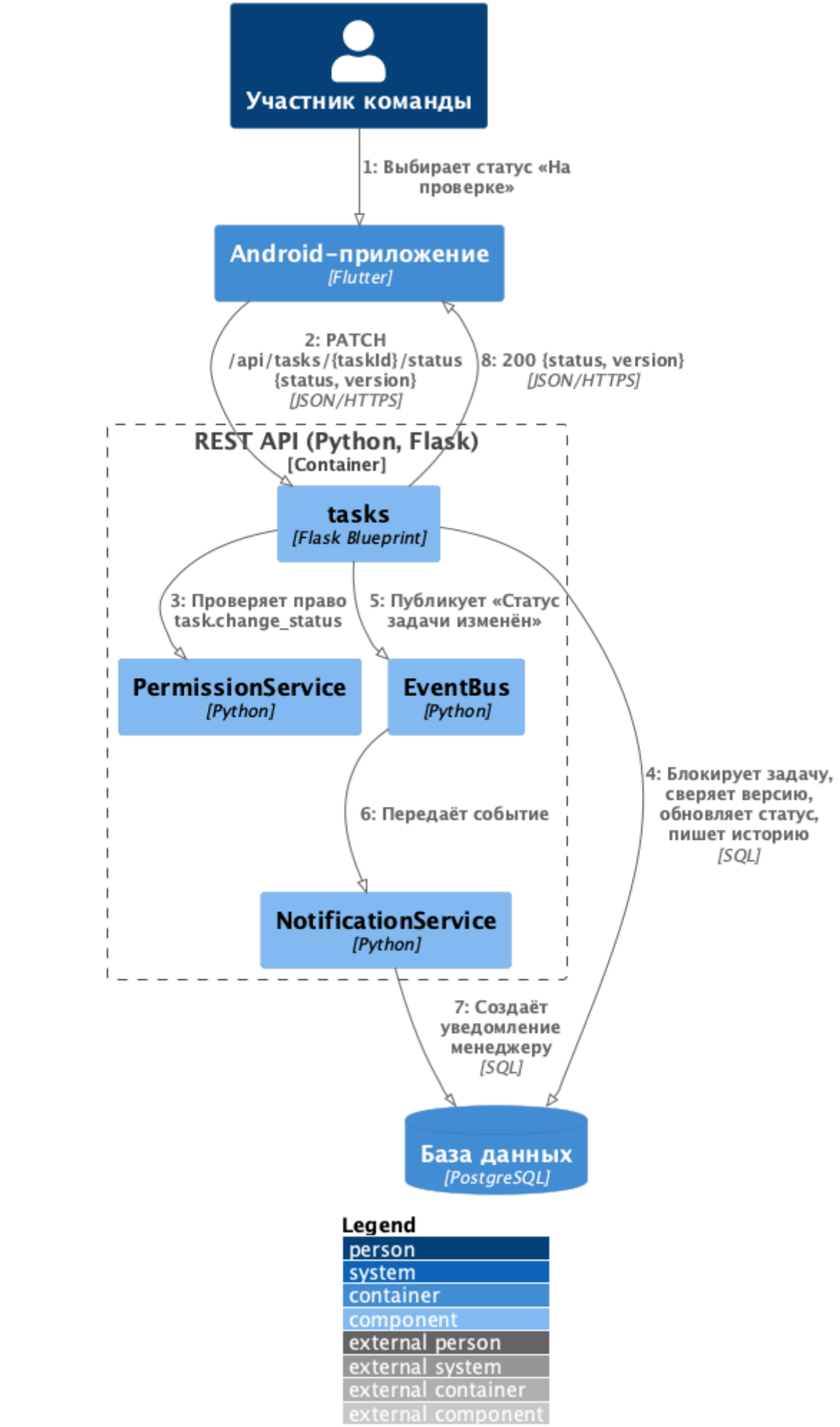


Рисунок 40 - Смена статуса задачи (C4, динамическая диаграмма)

Б.8. Матрица трассируемости

Таблица 13 - Трассируемость требований

История	**Вариант использования**	**Требования**	**Модели**
US-1	Зарегистрироваться, Войти в систему	FR-01…FR-05	Рис. 27
US-2	UC-T1, UC-T2	FR-21…FR-25	Рис. 28
US-3	Добавить участника в команду	FR-12…FR-14	Рис. 22, 30
US-4	UC-T3	FR-26…FR-31	Рис. 29, 40
US-5	Найти и отфильтровать задачи	FR-32…FR-35	Рис. 25
US-6	Получить уведомление	FR-45…FR-49	Рис. 28
US-7	Настроить глобальные права доступа	FR-08…FR-10	Рис. 23
US-8	UC-T4	FR-36…FR-38	Рис. 21, 31
US-9	Пригласить участника по email, Принять приглашение	FR-15…FR-17	Рис. 22, 30
US-10	UC-T5, Отметить просроченные задачи	FR-51…FR-54	Рис. 32
US-11	Комментировать и прикреплять файлы	FR-39…FR-41, FR-50	Рис. 31
US-12	UC-T3 (офлайн)	FR-42…FR-44	Рис. 25, 29
US-13	Восстановить пароль	FR-06, FR-07	Рис. 19
US-14	Архивировать проект	FR-18…FR-20	Рис. 18

Общие выводы по лабораторной работе

В ходе выполнения лабораторной работы разработаны требования к системе управления проектами “Pinkmine” и спроектирована её архитектура. Работа выполнена последовательно, от пользовательских требований к моделям предметной области, архитектуре и техническому заданию, причём каждый следующий этап опирался на результаты предыдущих и на материалы лабораторных работ №1 и №2.

Требования сформулированы в виде семи пользовательских историй для четырёх ролей, нового пользователя, участника команды, менеджера проекта и администратора системы. Для каждой истории определены проверяемые критерии приёмки, приоритет с обоснованием и зависимости, а различия веб-интерфейса и Android-приложения сведены в отдельную таблицу. На основе историй и объектной модели ЛР2 построены диаграммы вариантов использования, деятельности и классов на PlantUML, а также диаграммы вариантов использования, последовательности и классов на Mermaid.

С помощью ИИ-агентов разработано 12 промптов, по одному на каждый вид артефакта, и применены навыки getanalyst-database-designer и getanalyst-restapi-designer-skill. В результате список пользовательских историй расширен до 14, спецификация дополнена сценариями вариантов использования, диаграммами деятельности, классов и объектов, последовательности, компонентов, пакетов и развёртывания, спроектирована база данных из 19 таблиц с ERD и ORM-отображением и подготовлена полная постановка метода изменения статуса задачи. Ревью навыков выявило ошибки черновой модели данных, которые были исправлены во всех связанных диаграммах.

Метод EventStorming позволил описать предметную область через события, выделить поворотные события «Проект создан», «Задача создана» и «Задача закрыта», выявить и разрешить пять противоречий, а также определить пять агрегатов и четыре ограниченных контекста, соответствующих модулям Flask-сервера. Метод Event Modeling дополнил эту модель экранами-триггерами и представлениями и разбил систему на 17 срезов, каждый из которых связывает экран, команду, маршрут REST API, событие и представление и описан сценариями Given / When / Then. Архитектура системы описана по модели C4 на уровнях контекста, контейнеров и компонентов, дополнительно построены диаграммы развёртывания и динамическая диаграмма смены статуса задачи.

Итогом работы стала спецификация требований по шаблону SRS К. Вигерса и Д. Битти, которая объединяет результаты всех заданий: 54 функциональных требования, сгруппированных по ограниченным контекстам, требования к данным, интерфейсам и атрибутам качества, а также матрицу трассируемости от пользовательских историй к вариантам использования, требованиям и моделям. Совместное применение UML, событийных методов и модели C4 показало, что эти подходы дополняют друг друга, а разработанная спецификация является основой для реализации веб-сайта, API-сервера и мобильного приложения Pinkmine.